

Adversaria

A Specification for a Public Claim Graph

Flyxion

Independent Researcher

September 2026

Scope and Conventions

This volume specifies Adversaria in five parts. Part I states the problem and the treatment of provenance, and Part II places the design among its predecessors. Part III, Chapters 4 to 12, is the formal model: the claim and its scope algebra, evidence and warrant, arguments, objections and defeat semantics, distinction and dependency with the gluing analysis, the append-only ledger, the typed relation algebra, constraint, and the status vector. Part IV, Chapters 13 to 18, is the protocol: it separates routing from adjudication and specifies the pipeline, ontologies and mappings, capability gates and restrictions, imports and negative results, and federation and queries. Part V works two cases. The parts after the formal model add no semantic rule to it; each chapter states which of its results are proved in Part III and which are design requirements.

Conventions. Statements of three kinds are kept apart throughout. A *theorem*, *proposition* or *lemma* is a mathematical fact about the model and is proved. A *definition* or *design rule* is a choice; it is argued for but not proved, and no theorem is claimed for it. A *counterexample* shows why a tempting alternative is rejected. Nothing in the text is presented as a theorem when it is a normative choice about what counts as good evidence.

Illustrative material. Examples are schematic. They exercise the formalism and do not report empirical findings about any real study, population or page.

Contents

Scope and Conventions	ii
I Problem	1
1 The Unit of a Public Knowledge Base	2
1.1 A family of pages that disagree	2
1.2 Three candidate units	3
1.3 What is stored and what is computed	3
1.4 What the specification guarantees and what it leaves open	4
1.5 Organization	4
2 Traces, Agents and Independence	5
2.1 Attribution as an unresolved trace	5
2.2 Independence is a property of origins	5
2.3 Inspectable work and performed agency	6
II Predecessors	7
3 Predecessors and Departures	8
3.1 Article encyclopedias	8
3.2 Large ontologies	8
3.3 Statement stores	9
3.4 Argument maps and abstract argumentation	9
3.5 Summary of departures	9
III Formal Model	10
4 The Claim	11
4.1 Units and the unit space	11
4.2 Boxes, regions and the scope algebra	12
4.3 Kernels, negation and grain	12

4.4	The claim	13
4.5	What a claim says	14
4.5.1	Scope monotonicity	14
4.5.2	Entailment is exactly restriction	15
4.5.3	Conflict is local	15
4.5.4	Rules that follow	16
5	Evidence and Provenance	17
5.1	Evidence items	17
5.2	What a source can and cannot supply	18
5.3	Independence	18
5.4	Which kinds may ground which modalities	19
6	Warrants and Arguments	21
6.1	Warrants	21
6.2	Arguments	22
6.3	What well-formedness guarantees	22
6.4	Chains need bridges	23
6.5	What objections will target	24
7	Objections and Defeat	25
7.1	Objections are arguments	25
7.2	Sub-arguments inherit attacks	26
7.3	Policies and the split between attack and defeat	27
7.4	Labeling an argument at a unit	27
7.5	Three structural guarantees	29
7.6	Disagreement, cycles and reinstatement	30
7.7	Policies, and objects that fall out of play	32
7.8	A worked example	32
8	Distinction, Dependency and Gluing	34
8.1	Overloaded terms and senses	34
8.2	Restriction and pooling preserve status	35
8.3	Dependency	36
8.4	Pages as local sections	37
8.4.1	The general gluing problem	37
8.4.2	Signed identifications, exactly	38
8.4.3	The cohomological reading, and its limits	39
8.4.4	What Adversaria does with an obstruction	39
9	The Ledger	41
9.1	Records and the pipeline	41

9.1.1	Three identifiers	42
9.2	Trace completeness	42
9.3	Dispositions and the active set	43
9.4	Validity and merging	44
9.5	Revision lineage	44
9.6	Withdrawal	45
9.7	Replay, merging and traceability	46
9.8	Evaluation depends on arguments only	47
10	The Relation Algebra	48
10.1	Ten relation types	48
10.2	Scoped relations form an algebra	50
10.3	Composition is licensed, not assumed	51
11	Constraint	53
11.1	Constraint as region removal	53
11.2	Constraint on scopes is a commutative monoid	54
11.3	Preservation	54
11.4	The surviving claim	55
12	The Status Vector	57
12.1	Two layers: what is on the table, and what stands	57
12.2	The vector	58
12.3	Dominance	59
12.4	Scalar summaries are not determined by the data	60
12.5	Weighted sums	62
12.6	The Pareto frontier	63
12.7	Policy-indexed evaluation	63
12.8	Scoreboards and commensurability	64
IV	Protocol	66
13	Routing Is Not Adjudication	67
13.1	Four functions	67
13.2	What routing can and cannot change	68
13.3	Acyclic dependence	68
13.4	What each architecture is for	69
13.5	Routing as a surface	69
14	The Pipeline	71
14.1	Records	71
14.2	Five stages	72

14.3	Every submission receives a disposition	72
14.4	Proof objects and certificates	73
14.5	Idempotence and order	74
15	Ontologies and Mappings	75
15.1	Plural ontologies	75
15.2	Mapping claims	75
15.3	Transport	76
15.4	Consequences	77
16	Gates, Restrictions and Dispositions	78
16.1	Ledger time	78
16.2	Two planes	79
16.3	Capabilities	80
16.4	Gates authorize acts, not conclusions	81
16.5	Testing capability without testing belief	82
16.6	Restrictions and controversiality	84
16.7	Dispositions with reasons	85
16.8	Gate capture	85
17	Imports, Automated Origin and Negative Results	87
17.1	Origin modes and batches	87
17.2	Reversible activation	88
17.3	Negative and failed work	88
17.4	Credit as a vector	91
17.5	Attention is the scarce resource	91
18	Federation, Queries and the Constitution	93
18.1	Shared events, divergent projections	93
18.2	Queries over arguments and scopes	94
18.3	The dossier	95
18.4	The ten decisions	95
18.5	Limits of the specification	96
V	Cases	97
19	Three Pages and One Name	98
19.1	Setup	98
19.2	What page-by-page checking sees	99
19.3	What the specification reports	99
19.4	Ways the obstruction can change	99
19.5	What the case shows	100

20 A Contested Effect	101
20.1 Setup	101
20.2 Status before any objection	102
20.3 An objection	102
20.4 A reply, and two policies	102
20.5 An equivalence result	103
20.6 A withdrawal	103
20.7 What the case shows	103
A Inventory of Results	105

Part I

Problem

Chapter 1

The Unit of a Public Knowledge Base

A public knowledge base is organized around a unit, and the unit decides which failures the system can express and which it must hide. Encyclopedias take the article as the unit. Knowledge graphs take the statement, usually a subject, a property and a value with some annotation. This monograph specifies a system whose unit is the *argued proposition*: a claim, the grounds offered for it, the warrant that connects grounds to claim, the scope over which it is asserted, the objections raised against it, and the status these currently yield. The system is called Adversaria.

The specification follows one principle. It does not store what a community says is true. It stores what has been claimed, why it may be true, what limits it, and what would make it fail. Everything else, including any display of what currently stands, is computed from that record under a stated policy and can be recomputed.

1.1 A family of pages that disagree

A familiar failure of article-based encyclopedias is structural and not editorial. Take a figure discussed in several traditions: a page on the figure as presented in one scripture, a page on the same figure in another religious tradition, and a separate language edition with its own page. Each page is internally maintained, cited and reviewed. Each is locally coherent. Yet the pages make statements about what is nominally one subject, and some of those statements conflict. No page is wrong by its own standards, and the family has no place where the conflict is stated, because the conflict lies between pages and every editorial process operates within one.

The specification treats this as a gluing problem. Each page is a local section of what would be a global description; the local sections agree wherever they overlap in what they assert, or they do not. If they do not, the failure is not in any one section, and no amount of local editing removes it. Chapter 8 gives the exact form for a family of pages whose overlaps are binary identifications: there is an obstruction if and only if a certain cycle exists, and the region on which the obstruction lives is itself a scope (Theorem 8.16). What a distinction between senses can then do is dissolve the obstruction by declaring that the pages were not about one thing; what it can never do is create one (Proposition 8.18). Chapter 19 works the case through.

The point for the present chapter is only that the page is the wrong unit for detecting

this. The unit must be something two pages can both bear on, with a scope, so that the overlap is computable.

1.2 Three candidate units

The three units can be compared by asking what each can say about a disputed proposition.

	page	statement	argued proposition
carries	prose about a topic	subject, property, value, with qualifiers and references	claim, scope, grounds, warrant, objections
scope	implicit in the prose	optional qualifiers	mandatory, region-valued
conflict	editorial dispute, one winner or hedged prose	two values, ranks	localized to overlapping scope, both preserved
why supported	citations at paragraph level	references attached to a statement	explicit argument with warrant
what fails	the page is edited	the value is deprecated or deleted	a component is attacked; a narrower claim can survive

None of the three is wrong for every purpose. A page is a good way to read. A statement store is a good way to serve a stable lookup. The argued proposition is the unit that the machinery of disagreement needs, and the other two can be produced from it as presentations. The reverse is not true: a page does not determine which claims it contains or their scopes.

1.3 What is stored and what is computed

The specification keeps three layers apart and never lets an interface label move between them.

1. The *ledger* records what happened: which records were inscribed, by which acts, referencing which earlier records. It is append-only (Chapter 9).
2. The *argument graph* represents inferential relations among admitted records: which arguments have which conclusions, which attack which (Chapters 6 and 7).
3. The *presentation*, or dossier, is a policy-relative reading of the argument graph: labels, regions of standing, and the status vector (Chapters 7 and 12).

A label such as “accepted” is easily read at once as a historical fact (someone accepted it), a logical consequence (it follows) and a community consensus (most agree). Here these are three different objects. Only the first is in the ledger; the second is a relation in the graph; the third is a computation, and Theorem 9.13 says the computation depends on the set of records and the policy and on nothing else.

1.4 What the specification guarantees and what it leaves open

The specification separates results that are proved from choices that are argued. The proved results, collected in the appendix, include that narrowing a scope never breaks a valid claim (Theorem 4.11); that conflict between opposite-kernel claims is confined to the overlap of their scopes (Proposition 4.16); that a label can be computed regionally and is consistent (Theorems 7.7 and 7.8); that contradictory arguments do not license arbitrary conclusions (Corollary 7.12); that replicas holding the same records under the same policy hold the same projection (Theorem 9.13); and that no scalar represents the ordering of statuses (Theorem 12.12).

The choices are of a different kind, and are not dressed as theorems. They include what counts as an admissible warrant for a given kind of evidence, how strong a rung of the identification ladder each method reaches, which preferences a policy may adopt, and how scarce attention is allocated. These are set out as definitions, design rules and design arguments. A reader who disagrees with a choice can replace it and keep the results that do not depend on it. Where a result depends on a choice, the dependence is stated.

The specification also leaves some things unspecified on purpose. It does not say which proposition is true. It does not rank contributors, claims or dossiers by a single figure. It does not define a universal ontology. Each refusal is argued in the chapter where it arises, and each is a decision about what a shared record ought to promise.

1.5 Organization

Part I (this chapter and Chapter 2) states the problem and the treatment of provenance. Part II (Chapter 3) places the design among its predecessors. Part III, Chapters 4 to 12, is the formal model and the source of every proved result. Part IV, Chapters 13 to 18, is the protocol: it separates routing from adjudication, and specifies admission, ontologies, capability gates, imports and federation, each in terms the formal model supports. Part V works two cases. Later parts depend on earlier ones and never the reverse: nothing in the protocol part adds a semantic rule to the model.

Chapter 2

Traces, Agents and Independence

A record of an argument must say where the argument came from. The question is what “where” can mean. The obvious answer names an author, and the obvious reading of the author is an agent with beliefs and intentions. This chapter argues that the specification can do without that reading, and that doing without it is what lets it handle automated contributors, collaborations and duplicated origin in one framework.

2.1 Attribution as an unresolved trace

To say that an agent produced a contribution is to say that something upstream of the contribution, unobserved in detail, is being treated as one source. The word “agent” compresses a history: prompts, tools, training, edits, reviews, the decision to submit. For a person the compression is habitual and usually harmless. For a system of automated contributors it is not: ten instances of one model, given the same instruction and the same sources, are ten records but very nearly one origin, and any accounting that counts them as ten has counted the compression boundary and not the sources behind it.

The specification therefore records *traces* and not intentions. A trace of an act is what can be inspected about how it came to be: the records it refers to, the sources it used, the method applied, the production mode, the actor identifier, and the sequence position. Trace completeness has six components, and an act with any component missing is trace-null (Definition in Chapter 9). A trace-null act is not an act by an unknown agent; it is an act whose origin is unresolved, and the ledger says so. Such an act is quarantined and inert: it is stored, addressable and citable as a record, but takes no part in evaluation, and adds nothing to any count.

Design rule 2.1 (Unresolved origin is a state, not a default). An act whose trace is incomplete is recorded as such and is never assigned a default origin. Assigning one (a default human author, say) would convert an absence into an assertion, and later counts of independent sources would depend on it.

2.2 Independence is a property of origins

Evidence counts as independent support only when it has independent origins, and origin is determined by shared methodological dependence, not by the number of records.

Chapter 5 defines independence families as classes of a signed identification and proves that duplicates do not raise the independent count (Proposition 5.5). The consequences for automated and imported material are direct.

- A batch of records generated by one procedure from one source is one family, however many records it contains.
- Repeated runs of one model with one instruction over one corpus are one family unless the runs differ in a way that a declared equivalence claim, itself open to objection, recognizes as producing independent evidence.
- Agreement among records of one family is displayed, since it is information, but never counted as replication.

The rule is deliberately conservative. A false merge of two independent sources understates support and is repaired by a distinction claim; a false separation of two dependent sources overstates support and is repaired by an equivalence claim. Both repairs are ordinary acts in the ledger, contestable like any other, and both are visible in the status vector through the coordinate that counts families.

2.3 Inspectable work and performed agency

A public record can reward the appearance of purposeful activity, such as many posts, many replies and many actors, or it can reward acts whose reasons can be checked. The specification takes the second course. Credit, standing and influence attach to argumentative acts, and an act counts only through the record of what it did to the argument graph. Goal-directedness is attributed by observers and is not stored. This has one practical consequence that matters for automated contributors: a system that produces a great deal of fluent activity gains nothing from the volume, since volume neither adds independent families (Section 2.2) nor carries any defeat power (Proposition 12.5), and gains nothing from apparent agency, since agency is not an input to evaluation (Proposition 9.15).

What an automated contributor can do is what any contributor can do: add evidence with a checked anchor, supply a warrant with backing, raise an objection whose active region is nonempty, or restrict an overbroad claim. Each of these is a legible act with an effect on the graph. The specification does not decide whether such an act is wise. It decides that the act is recorded with its origin mode and evaluated by the same rules as every other.

Part II

Predecessors

Chapter 3

Predecessors and Departures

The design draws on four traditions, and departs from each at a specific place. This chapter states the departures. It describes each predecessor only as far as the comparison needs, and it treats their failure modes as design constraints and not as verdicts on their authors' aims.

3.1 Article encyclopedias

The article is a strong unit for reading and a weak one for adjudication. Its scope is carried by prose, its evidence by citation at the level of a paragraph or sentence, and its disagreements by editorial process: one version at a time is current, and a dispute is settled by editing or by hedged wording. The modularity that makes an encyclopedia scalable, with each topic maintained in its own page and language edition, is the same feature that lets conflicting statements coexist without any place that records the conflict (Chapter 1). Adversaria keeps the article as a presentation of a set of claims, and stores the claims.

3.2 Large ontologies

Cyc attempted to encode common knowledge in a single logical ontology with general inference. Two design problems are usually identified with that program. Concepts in ordinary use are context-sensitive, purpose-relative and contested at their boundaries, and a single engineered vocabulary must either fix them or localize them; the microtheory mechanism localizes them, but then the system must decide which microtheory governs a statement and how conclusions cross between microtheories. And unrestricted inference over a large, imperfectly consistent theory grows quickly and is difficult to audit. Adversaria's responses are that ontologies are versioned objects in the graph and that mappings between them are argued claims (Chapter 15), and that inference is local, through explicit warrants, with no closure under support (Theorem 6.6).

3.3 Statement stores

Wikidata-style statement stores improve on bare triples with qualifiers, references and ranks. The center remains a subject, a property and a value. The qualifiers can describe a statement but do not naturally say why a cited source supports it, which warrant connects them, what the strongest undercutter is, whether the source merely reports the claim, or which operationalization was used. References can therefore act as markers of authority and not as parts of an argument. A rank is a scalar per statement and resolves disagreement by preference within one property. Adversaria replaces the triple by the argument as the carrier of epistemic force, makes scope mandatory and region-valued (Design rule 4.18), and replaces the rank by a status vector with no scalar aggregation (Theorem 12.12). Large automated imports in statement stores raise a further problem, that record count can look like consensus; the treatment is by independence families (Chapter 17).

3.4 Argument maps and abstract argumentation

The design owes most to the literature on argument. Toulmin’s layout of claim, grounds, warrant, backing, qualifier and rebuttal is the source of the components of an argued proposition. Dung’s abstract argumentation frameworks and the grounded labeling supply the evaluation, and the structured-argumentation literature, notably ASPIC-style systems, supplies the distinction among rebutting, undermining and undercutting attacks. Belnap’s four values supply the informational layer of the status.

The departures are these. First, every element is scoped: an attack has an active region, and labels are computed at each unit of a product space, so that a defeat can remove part of a claim and leave the rest (Theorem 7.8). Second, attack is structural and derived from what an objection concludes, while defeat is policy-relative (Theorem 7.18), so that different communities can share one graph and differ only in a declared preference. Third, the store is an append-only ledger, and the evaluation is a function of it, which gives replay determinism and traceable divergence (Theorems 9.13 and 9.14). Fourth, the design refuses a scalar summary and proves why (Theorem 12.12), where argument-mapping tools commonly present one.

3.5 Summary of departures

predecessor	shared assumption declined	replacement
article encyclopedia	the page is the unit of maintenance and of truth	claims stored, pages projected
large ontology	one canonical vocabulary with global inference	plural versioned ontologies, argued mappings, local inference
statement store	a value with annotations is the unit, a rank settles disputes	argued proposition, status vector
argument map	evaluation is unscoped and score-like	regional labeling, policy-relative defeat, no scalar

Part III

Formal Model

Chapter 4

The Claim

Wikipedia stores sentences inside articles. Wikidata stores triples with qualifiers. Neither stores what a reader most needs in order to disagree usefully: the exact proposition being asserted, the units over which it is asserted, the meanings its terms carry, and the conditions under which it would fail. Adversaria’s basic object is therefore a *claim*: a scoped, typed, versioned proposition. This chapter defines the space in which claims are scoped, the algebra of scopes, and the claim itself. The next chapter states what a claim means and proves the first structural guarantees.

4.1 Units and the unit space

Every claim is about some collection of things: people, materials, periods, settings, instruments. The model does not enumerate these collections. It describes them by coordinates.

Definition 4.1 (Unit space). A *unit space* is a finite set D of *dimensions* together with a nonempty set U_d of *values* for each $d \in D$. Its elements are *units*:

$$\Omega = \prod_{d \in D} U_d.$$

Four kinds of dimension recur in practice, and the schema reserves names for them without treating them differently in the mathematics:

Dimension	Typical values
population	adults, adolescents, a clinical sample, a species
context	laboratory, daily life, a jurisdiction, a historical period
time	an interval, an epoch, a version of a standard
measurement regime	an instrument, a protocol, an estimand, a threshold

A unit is one combination of values, for example (adults, daily life, 2010–2020, momentary sampling). The space is a modelling device: contributors state scopes in these terms, and nothing requires the values to be finite or to be enumerated in advance. Where the mathematics needs finiteness it derives it from the finitely many scopes actually present in a dossier (Chapter 7).

4.2 Boxes, regions and the scope algebra

Definition 4.2 (Box, region). A *box* is a set $\prod_d S_d$ with $S_d \subseteq U_d$ for every d . A *region* is a finite union of boxes. Write $\mathcal{B}(\Omega)$ for the set of regions.

A contributor's declared scope is usually a box: "adults, in daily life, under momentary sampling." Boxes alone are not enough, because the operation the system most needs is subtraction. A claim that survives an objection survives *outside* the region the objection removes, and the difference of two boxes is generally not a box.

Example 4.3. Let $D = \{\text{population, context}\}$ with $\text{population} \in \{a, c\}$ (adults, children) and $\text{context} \in \{\ell, f\}$ (laboratory, field). Remove the box $\{c\} \times \{\ell\}$ from $\{a, c\} \times \{\ell, f\}$. What remains is $\{(a, \ell), (a, f), (c, f)\}$, which is not a box: a box containing (a, ℓ) and (c, f) would also contain (c, ℓ) . It is, however, a region: $\{a\} \times \{\ell, f\} \cup \{a, c\} \times \{f\}$.

Proposition 4.4 (Regions form a Boolean algebra). $\mathcal{B}(\Omega)$ contains $\emptyset$ and Ω and is closed under finite union, finite intersection and complement in Ω . Consequently $(\mathcal{B}(\Omega), \subseteq)$ is a Boolean algebra, a subalgebra of the power set of Ω .

Proof. Closure under finite union is the definition of a region, and $\Omega = \prod_d U_d$ is a box. The intersection of two boxes $\prod_d S_d$ and $\prod_d T_d$ is the box $\prod_d (S_d \cap T_d)$. Since intersection distributes over union, the intersection of two finite unions of boxes is a finite union of pairwise intersections of boxes, hence a region.

For complements, let $B = \prod_d S_d$ be a box. A unit x lies outside B exactly when $x_d \notin S_d$ for some d , so

$$\Omega \setminus B = \bigcup_{d \in D} B_d, \quad B_d = \left(\prod_{e \neq d} U_e \right) \times (U_d \setminus S_d),$$

a finite union of boxes. The complement of a finite union of boxes is the intersection of the complements of the boxes; each complement is a region, and regions are closed under intersection, so the result is a region. $\square$

Definition 4.5 (Scope). A *scope* is a region. Scopes are ordered by inclusion, $\sigma \preceq \tau$ iff $\sigma \subseteq \tau$. Meet is intersection, join is union, and relative complement $\sigma \setminus \tau$ is defined for all scopes. A scope is *void* if it is empty.

Remark 4.6 (Hulls are not needed). If the schema admitted only boxes, the join of two scopes would have to be a representable *hull* (the smallest box containing both) and subtraction would be unavailable. Working in $\mathcal{B}(\Omega)$ avoids both problems: join is exact, and every surviving remainder of a constrained claim is itself a scope. The user-facing interface can still present scopes as boxes, or unions of a few boxes, without the algebra changing.

4.3 Kernels, negation and grain

Two sentences with the same words can express different propositions, and two sentences with different words can express the same one. The model therefore individuates the *content* of a claim by more than its wording.

Definition 4.7 (Kernel). A *kernel* κ consists of

- (i) an exact statement, a string that states one proposition about a unit;
- (ii) *operational meanings* μ : for each term of the statement that needs one, a reference to a definition object fixing how the term is applied;
- (iii) an *ontology version* naming the vocabulary in which the statement and μ are written;
- (iv) a *negation* $\bar{\kappa}$, another kernel with $\bar{\bar{\kappa}} = \kappa$ and $\bar{\kappa} \neq \kappa$, stating that the proposition fails at the unit;
- (v) a *grain* $\Gamma(\kappa)$, a partition of Ω into blocks.

Two kernels are equal iff they agree in all five components.

Kernel identity includes μ and the ontology version. “Emotional differentiation improves regulation” with differentiation defined as within-person granularity of momentary reports, and the same sentence with differentiation defined as a score on a self-report scale, are different kernels. A dispute that turns on which definition is meant is then visible as such and does not masquerade as a factual disagreement (Chapter 8).

The grain records the level at which a kernel is evaluated. A kernel about individuals has the finest grain, in which every unit is its own block. A kernel that states an aggregate, such as “the mean effect is positive,” is evaluated only for a whole cell of a population design and cannot be asked of an individual or of half a cell. Its blocks are those cells.

Definition 4.8 (Admissible scope). A scope σ is *admissible* for κ if it is a union of blocks of $\Gamma(\kappa)$. Admissible scopes for κ form a Boolean subalgebra of $\mathcal{B}(\Omega)$.

4.4 The claim

Definition 4.9 (Claim). A *claim* is a tuple

$$c = (\kappa, \sigma, m, \pi, \nu)$$

with

- κ a kernel;
- $\sigma = \text{scope}(c)$ a nonempty scope admissible for κ ;
- $m \in \{\text{descriptive, associational, causal, mechanistic, normative, speculative}\}$ the epistemic modality;
- π a provenance record naming the ledger record that introduced c (Chapter 9);
- ν a version identifier, the identifier of that record.

The pair (κ, σ) is the *core* of c .

The core carries the meaning; the other three fields govern how the claim may be argued for and how it is stored. The modality does not change what the claim says. It changes which evidence kinds may ground it and which warrants may connect them

(Chapter 6), and those are design rules, not theorems. Provenance and version make every relation in the system a relation between claim *versions*: an objection targets the version it was written against, never an unversioned sentence.

4.5 What a claim says

A claim asserts its kernel of every unit in its scope. The semantics below makes that precise and fixes what follows from a claim by logic alone. Anything that follows only because of further assumptions must enter the graph as an argument (Chapter 6).

Definition 4.10 (Model, truth). Assume, as a standing convention, that a kernel and its negation have the same grain. A *model* M assigns to every kernel κ a set $[\kappa]_M \subseteq \Omega$ such that

- (i) $[\bar{\kappa}]_M = \Omega \setminus [\kappa]_M$, and
- (ii) $[\kappa]_M$ is a union of blocks of $\Gamma(\kappa)$.

Otherwise the assignments are unconstrained: distinct kernel pairs $\{\kappa, \bar{\kappa}\}$ are assigned independently. A claim $c = (\kappa, \sigma, \dots)$ is *true in* M , written $M \models c$, iff $\sigma \subseteq [\kappa]_M$.

This is a universal reading: the claim holds of *every* unit in its scope. The choice is deliberate and has a consequence worth stating at once. An existential statement (“there is a population in which the intervention harms”) is not a claim in this sense. It enters the graph as *evidence* about a witness scope, asserting the negated kernel throughout the units actually observed (Chapter 5). A single counterexample is then a universal statement about its witness scope, and it conflicts with a broader claim exactly where the two scopes overlap (Proposition 4.16).

4.5.1 Scope monotonicity

For a claim $c = (\kappa, \sigma, m, \pi, \nu)$ and a nonempty scope $\tau \preceq \sigma$ admissible for κ , write $c|_\tau = (\kappa, \tau, m, \pi, \nu)$ for the *restriction* of c to τ . (In the ledger a restriction is a new claim version that cites c ; Chapter 9.)

Theorem 4.11 (Scope monotonicity and additivity). *Let κ be a kernel and M a model.*

- (a) (Downward monotone.) *If $M \models (\kappa, \sigma)$ and $\tau \subseteq \sigma$ is an admissible scope, then $M \models (\kappa, \tau)$.*
- (b) (Additive.) *For admissible scopes $\sigma_1, \dots, \sigma_n$, $M \models (\kappa, \sigma_1 \cup \dots \cup \sigma_n)$ iff $M \models (\kappa, \sigma_i)$ for every i .*

Proof. (a) $\tau \subseteq \sigma \subseteq [\kappa]_M$. (b) A union is contained in a set iff each member is. $\square$

Part (b) is the sheaf-like condition on which distinction and gluing rest (Chapter 8): a universal claim over a union of scopes is exactly the conjunction of its claims over the parts.

Corollary 4.12 (Iterated restriction). *If $\sigma_1 \supseteq \sigma_2 \supseteq \dots$ is a chain of admissible scopes and $M \models (\kappa, \sigma_1)$, then $M \models (\kappa, \sigma_n)$ for every n .*

Counterexample 4.13 (Why grain is required). Drop the grain condition and read an aggregate kernel κ_{agg} , “the mean effect is positive,” as follows. Let a model carry weights $w : \Omega \rightarrow (0, \infty)$ and effects $e : \Omega \rightarrow \mathbb{R}$, and say the claim holds on σ iff $\sum_{x \in \sigma} w(x)e(x) > 0$. Take $\Omega = \{a, b\}$ with $w(a) = 9, e(a) = 1, w(b) = 1, e(b) = -5$. On $\sigma = \{a, b\}$ the sum is $9 - 5 = 4 > 0$, so the claim holds; on $\tau = \{b\} \subset \sigma$ the sum is $-5 < 0$, so it fails. Aggregate claims are not downward monotone.

Adversaria does not adopt this reading. It gives κ_{agg} the grain $\Gamma(\kappa_{\text{agg}}) = \{\Omega\}$, one block, so that $\{b\}$ is not an admissible scope and the claim $(\kappa_{\text{agg}}, \{b\})$ cannot be formed. A statement about subpopulation b is a claim about a *different kernel*, “the mean effect in b is positive,” whose grain separates b from a . Passing from the coarse kernel to the finer one is a distinction (Chapter 8); it is never a restriction, and Theorem 4.11 is never invoked across it.

4.5.2 Entailment is exactly restriction

Definition 4.14. For claims c, d write $c \models d$ if every model satisfying c satisfies d .

Theorem 4.15 (Completeness of the restriction order). *Let $c = (\kappa, \sigma, \dots)$ and $d = (\kappa', \tau, \dots)$ be claims. Then $c \models d$ iff $\kappa' = \kappa$ and $\tau \subseteq \sigma$.*

Proof. $(\Leftarrow)$ is Theorem 4.11(a). For $(\Rightarrow)$ suppose it is not the case that $\kappa' = \kappa$ and $\tau \subseteq \sigma$. We build a model satisfying c but not d . Recall that $\tau \neq \emptyset$.

Case 1: $\kappa' = \kappa$ and $\tau \not\subseteq \sigma$. Set $[\kappa]_M = \sigma$ and $[\bar{\kappa}]_M = \Omega \setminus \sigma$, both unions of blocks since σ is admissible. Then $M \models c$, and a unit of $\tau \setminus \sigma$ shows $\tau \not\subseteq [\kappa]_M$, so $M \not\models d$.

Case 2: $\kappa' = \bar{\kappa}$. Set $[\kappa]_M = \Omega$ and $[\bar{\kappa}]_M = \emptyset$. Then $M \models c$, while $\tau \neq \emptyset$ is not contained in $\emptyset$, so $M \not\models d$.

Case 3: $\kappa' \notin \{\kappa, \bar{\kappa}\}$. Set $[\kappa]_M = \Omega$ and $[\kappa']_M = \emptyset$ (with $[\bar{\kappa}]_M = \Omega$), the two pairs being independent. Then $M \models c$ and $M \not\models d$ as in Case 2. $\square$

The theorem says that a claim entails nothing beyond its own restrictions. In particular it entails nothing about other kernels, however related they look in ordinary language. Every inference from one kernel to another must therefore be made through an explicit warrant, which is itself a claim-like object with a scope (Chapter 6). This is the formal reason support is not implicitly transitive (Theorem 6.6).

4.5.3 Conflict is local

Two claims can appear opposed and still be compatible. The semantics locates conflict exactly.

Proposition 4.16 (Conflict localization). *Let $c = (\kappa, \sigma, \dots)$ and $d = (\bar{\kappa}, \tau, \dots)$. Then c and d are jointly satisfiable, that is, some model satisfies both, iff $\sigma \cap \tau = \emptyset$.*

Proof. If $x \in \sigma \cap \tau$ and $M \models c, d$, then $x \in [\kappa]_M$ and $x \in [\bar{\kappa}]_M = \Omega \setminus [\kappa]_M$, a contradiction. Conversely if $\sigma \cap \tau = \emptyset$, take $[\kappa]_M = \sigma$ and $[\bar{\kappa}]_M = \Omega \setminus \sigma \supseteq \tau$. $\square$

Proposition 4.17 (The conflict region is minimal). *Let $\rho = \sigma \cap \tau \neq \emptyset$. For any admissible scope $\rho' \subseteq \rho$, the claims $(\kappa, \sigma \setminus \rho')$ and $(\bar{\kappa}, \tau)$ are jointly satisfiable iff $\rho' = \rho$ (when $\sigma \setminus \rho'$ is nonempty).*

Proof. Their scopes meet in $(\sigma \setminus \rho') \cap \tau = \rho \setminus \rho'$, which is empty iff $\rho' = \rho$. Apply Proposition 4.16. $\square$

So two claims never contradict one another globally. They contradict each other on $\sigma \cap \tau$, and only there, and removing exactly that region from either one restores consistency. This is what allows rival positions to coexist in one dossier without rival edits to a single sentence: each is a claim with its own scope, and the system computes where they collide. Chapter 7 turns this location into a defeat relation.

4.5.4 Rules that follow

Design rule 4.18 (Scope and modality are mandatory). A submission that declares no scope, a void scope, or no modality may be inscribed in the ledger but is not admitted as a claim; it carries no status and takes part in no argument. It is marked incomplete, with the missing field named. This is a schema-level protection against exaggeration, not a style recommendation.

Remark 4.19 (Parameter families and boundary conditions). Many disputes arise because a sentence stands for a family $c(\theta)$ with θ specifying a threshold, an exposure, an outcome or an estimand. Treat such parameters as dimensions of Ω . Evidence for the sentence then supports a region Θ_E of the parameter space; contrary evidence removes a region Θ_D ; and the surviving claim is $\Theta_E \setminus \Theta_D$, a scope by Proposition 4.4. Boundary conditions are regions. The apparatus of this chapter requires nothing else.

Chapter 5

Evidence and Provenance

A reference in an encyclopedia is usually a pointer that a reader either trusts or does not. In Adversaria a reference is an object with parts, because the ways a citation can fail are different and each needs a place to be recorded. A source may not say what is attributed to it. A source may say it but merely report someone's belief. A study may supply data that do not bear on the estimand in question. An inference from sound data may be bad. This chapter defines evidence items so that each of these is a separately addressable failure.

5.1 Evidence items

Definition 5.1 (Evidence item). An *evidence item* is a tuple

$$e = (\text{src}, \text{loc}, \text{ext}, \text{kind}, \kappa_e, \sigma_e, \text{fam})$$

with

- *src* the content hash of an immutable source object (a paper, dataset, recording, proof script);
- *loc* a *locator* inside the source, precise to a page, figure, table, dataset row set, or analysis identifier;
- *ext* the *extract*: the text, values or output found at the locator;
- *kind* $\in \{\text{observation}, \text{experiment}, \text{dataset}, \text{quotation}, \text{derivation}\}$;
- κ_e a kernel and σ_e a nonempty scope admissible for it: what the item reports, and of which units;
- *fam* an *independence family* (Section 5.3).

The item *reports* (κ_e, σ_e) : within the source, κ_e was found to hold of the units in σ_e . Its *witness scope* is σ_e .

The scope of an evidence item is the scope of what was observed, not of what the observer would like to conclude. A trial run in one laboratory on one sample has a small witness scope even if its authors claim more. Claiming more is the business of an argument, and it is checked there (Chapter 6).

Definition 5.2 (Anchoring). An evidence item is *anchored* if *loc* resolves inside the source object addressed by *src* and the content found there equals *ext*. Anchoring is decidable from the source object. An item that is not anchored is a malformed record, not a weak one.

5.2 What a source can and cannot supply

The kinds are not interchangeable, and one of them needs particular care.

A *quotation* reports what a source *says*. Its kernel κ_e is of the form “source *S* states *P*,” not *P*. That is a different proposition, and the gap between the two is filled by a warrant about testimony: that the source is reliable on *P*, or that its author had access to what they describe. An argument that goes from “the source says *P*” to *P* therefore carries an explicit warrant, and the warrant can be undercut without disputing that the source says it. The same holds for a source that reports another party’s belief. In the vocabulary of the model:

Situation	Representation
a source contains a sentence	quotation item; $\kappa_e = \text{“}S \text{ contains this sentence”}$
a source reports someone believes <i>P</i>	quotation item; $\kappa_e = \text{“}S \text{ reports that } X \text{ believes } P\text{”}$
a study supplies data	observation, experiment or dataset item
an inference from the data	warrant inside an argument
the conclusion attributed to the study	the claim the argument concludes

Consequently “no source” and “a bad inference from a valid source” are different failure modes: the first defeats an evidence item, the second defeats a warrant (Chapter 7).

An *existence report* is an evidence item whose kernel is a negated kernel: it records that a proposition failed at the observed units.

Proposition 5.3 (A counterexample is localized). *Let e be an existence report with $\kappa_e = \bar{\kappa}$ and let $c = (\kappa, \sigma, \dots)$ be a claim. The report and the claim are jointly satisfiable iff $\sigma_e \cap \sigma = \emptyset$. If they overlap, the conflict lies exactly in $\sigma_e \cap \sigma$.*

Proof. Proposition 4.16 applied to $(\bar{\kappa}, \sigma_e)$ and (κ, σ) , and Proposition 4.17 for the location. $\square$

This is why a single counterexample need not refute a broad claim. It refutes the claim on the units it witnessed. What survives outside that region is the subject of Chapter 11.

5.3 Independence

Whether several evidence items are corroboration or repetition depends on whether they are independent. Record counts cannot decide that: one dataset re-analysed twenty times and twenty datasets collected separately produce the same number of records.

Definition 5.4 (Independence family). An *independence family* groups evidence items that share a methodological dependence: the same laboratory, dataset, instrument, pipeline, or automated import. Membership is declared when an item is inscribed, and the declaration is itself a contestable claim that any contributor may object to. The *independent count* of a set E of items is the number of distinct families among them.

Proposition 5.5 (Duplicates do not raise the independent count). *If E' is obtained from E by adding items from families already represented in E , the independent count is unchanged.*

Proof. The independent count is the number of distinct values of *fam* on the set, and adding items with existing values does not add a value. $\square$

The proposition is immediate, and the rule it protects is not. It applies with particular force to automated contributors. Several acts produced by one model, one prompt lineage, one script or one import share a methodological dependence, so by default they belong to a single independence family whatever number of personas, accounts or voices produced them. Ten automated agents repeating one inference are one dependent lineage, not ten confirmations. An act earns evidential standing only through what it identifiably does (extract evidence, supply a warrant, find a counterexample, restrict a scope), never through the appearance of a distinct contributor or through agreement with another one. Splitting a family requires a declared and contestable reason. Independence, replication and contributor count must never be inferred from record count. In particular one automated import of a million rows is one family, and cannot masquerade as a million confirmations. The status vector of Chapter 12 uses independent counts and no others.

5.4 Which kinds may ground which modalities

Not every kind of evidence can ground every kind of claim. A dataset can ground an association, but grounding a causal claim needs an identification strategy that the data alone do not supply. The admissibility of a kind for a modality is a *policy choice* and is not derived.

Design rule 5.6 (Admissibility table). A policy p supplies a relation $\text{Adm}_p \subseteq \text{Kinds} \times \text{Modalities}$. A well-formed argument may attach an evidence item of kind k directly to a claim of modality m only if $(k, m) \in \text{Adm}_p$. The default policy is as follows.

modality	obs.	expt.	dataset	quotation	derivation
descriptive	•	•	•	•	•
associational	•	•	•		•
causal		•			•
mechanistic	•	•			•
normative				•	•
speculative	•	•	•	•	•

A dataset may ground a causal claim only through a derivation that supplies the identification strategy. No theorem is claimed for this table; it records a defensible default, it is versioned, and communities may adopt a different one.

Admissibility limits what may be *attached*. It never certifies that what is attached is persuasive. Whether the attachment survives is decided by objections and the defeat semantics of Chapter 7.

Chapter 6

Warrants and Arguments

Evidence does not conclude anything by itself. A dataset shows what was found. What follows from it depends on a bridge: a reason why findings of that kind bear on a claim of that kind, over that range of units. In Toulmin’s vocabulary the bridge is the warrant. Adversaria makes it an explicit, scoped object, because it is where most disagreement about evidence actually lives, and because a claim can inherit scope from evidence only through it.

6.1 Warrants

Definition 6.1 (Warrant). A *warrant* is a tuple $w = (\kappa_1, \dots, \kappa_k \Rightarrow \kappa_0; \sigma_w; r; B)$ with $k \geq 1$ pairwise distinct *premise kernels* $\kappa_1, \dots, \kappa_k$, a *conclusion kernel* κ_0 , a *scope* σ_w , a *rung* r and a set B of *backing* (references to evidence items or claims that support the warrant itself, possibly empty). The rung is drawn from the ordered ladder

none < associational < adjusted < quasi-experimental < randomized < interventional,

and records how strongly the bridge identifies a causal relation; for non-causal warrants it is none. A model M *validates* w if

$$\forall x \in \sigma_w : \quad x \in [\kappa_1]_M \cap \dots \cap [\kappa_k]_M \implies x \in [\kappa_0]_M.$$

A warrant is thus a pointwise rule, licensed only on σ_w . It says nothing outside that region. The ladder is a design choice; a community can adopt a different one. The rung is part of the warrant, so a dispute over whether a study really identifies a causal effect is a dispute over a warrant, and it is fought by undercutting that warrant (Chapter 7).

Two *structural warrants* are built into the system and need no backing. RESTRICT takes one filler and yields the same kernel over a smaller scope. POOL takes fillers for one kernel κ over scopes $\sigma_1, \dots, \sigma_n$ and yields κ over their union. Both are valid in every model by Theorem 4.11, and both leave nothing to undercut: they can be attacked only through their fillers.

6.2 Arguments

Definition 6.2 (Argument). An *argument* a is a finite tree given by a warrant $w = (\kappa_1, \dots, \kappa_k \Rightarrow \kappa_0; \sigma_w; r; B)$, one *filler* f_i for each premise kernel, and a *conclusion claim* $c = \text{concl}(a)$ with kernel κ_0 , where each filler f_i is either an evidence item e with $\kappa_e = \kappa_i$ or an argument a_i with $\kappa(\text{concl}(a_i)) = \kappa_i$. Write $\sigma(e) = \sigma_e$ for evidence and $\sigma(a) = \text{scope}(\text{concl}(a))$ for arguments. The *sub-arguments* $\text{Sub}(a)$ are a together with the sub-arguments of its argument fillers. The *leaves* of a are the evidence items occurring as fillers anywhere in the tree.

An argument is *well-formed* when:

- (W1) *Scope discipline.* $\text{scope}(c) \subseteq \sigma_w \cap \sigma(f_1) \cap \dots \cap \sigma(f_k)$. For RESTRICT, $\text{scope}(c) \subseteq \sigma(f_1)$; for POOL, $\text{scope}(c) \subseteq \sigma(f_1) \cup \dots \cup \sigma(f_n)$.
- (W2) *Admissibility.* Each evidence filler of kind $\text{kind}(e)$ satisfies $(\text{kind}(e), m(c)) \in \text{Adm}_p$ (Design rule 5.6). If $m(c) = \text{causal}$ the warrant's rung is not none.
- (W3) *Anchoring.* Every leaf is anchored.
- (W4) *Recursion.* Every argument filler is well-formed.
- (W5) *No self-undermining.* The kernel of $\text{concl}(a)$ is not a denial kernel (Definition 7.1) of any target in any sub-argument of a . An argument whose own conclusion denies part of its own support is refused with that reason recorded.

The tree is finite and each sub-argument must already exist when the argument is inscribed, so arguments are well-founded by construction (Chapter 9).

Remark 6.3 (Joint and alternative support). Premises of one warrant support the conclusion *jointly*: a filler failing removes the argument. Two distinct arguments concluding the same claim support it *alternatively*: either surviving suffices. An argument is the hyper-edge $\{f_1, \dots, f_k, w\} \rightarrow c$. Flattening it to pairwise links would lose exactly the distinction between joint and alternative support.

6.3 What well-formedness guarantees

Say that a model *validates* an argument a if every leaf e of a is true in M (that is, $\sigma_e \subseteq [\kappa_e]_M$) and every ordinary warrant in a is validated by M .

Theorem 6.4 (Soundness of well-formed arguments). *If a is well-formed and M validates a , then $M \models \text{concl}(a)$.*

Proof. Induction on the height of the tree. Let $c = \text{concl}(a)$ have kernel κ_0 . By the induction hypothesis every argument filler a_i satisfies $M \models \text{concl}(a_i)$, that is, $\sigma(a_i) \subseteq [\kappa_i]_M$; every evidence filler satisfies $\sigma(e_i) \subseteq [\kappa_i]_M$ by hypothesis. So $\sigma(f_i) \subseteq [\kappa_i]_M$ for all i .

If w is ordinary, let $x \in \text{scope}(c)$. By (W1), $x \in \sigma_w$ and $x \in \sigma(f_i) \subseteq [\kappa_i]_M$ for each i . Since M validates w , $x \in [\kappa_0]_M$. Hence $\text{scope}(c) \subseteq [\kappa_0]_M$. If w is RESTRICT or POOL, the conclusion follows from Theorem 4.11 (a) or (b). $\square$

The scope discipline in (W1) is not merely sufficient. For ordinary warrants it is the exact boundary of what the premises license.

Theorem 6.5 (Tightness of scope discipline). *Let $w = (\kappa_1, \dots, \kappa_k \Rightarrow \kappa_0; \sigma_w; \dots)$ be an ordinary warrant whose kernels $\kappa_0, \overline{\kappa_0}, \kappa_1, \overline{\kappa_1}, \dots, \kappa_k, \overline{\kappa_k}$ are pairwise distinct, and suppose κ_0 has the finest grain (each unit its own block). Let $e_1, \dots, e_k$ be evidence items with $\kappa_{e_i} = \kappa_i$, and let $\mu = \sigma_w \cap \sigma_{e_1} \cap \dots \cap \sigma_{e_k}$. If S is a scope with $S \not\subseteq \mu$, there is a model in which all e_i are true, w is validated, and (κ_0, S) is false.*

Proof. Pick $x \in S \setminus \mu$. Set $[\kappa_i]_M = \sigma_{e_i}$ and $[\kappa_0]_M = \mu$, with negations complementary; distinct kernel pairs are independent, and σ_{e_i} is admissible for κ_i while μ is a union of singleton blocks. Each e_i is true. The set $\sigma_w \cap [\kappa_1]_M \cap \dots \cap [\kappa_k]_M$ equals $\mu \subseteq [\kappa_0]_M$, so M validates w . But $x \notin \mu = [\kappa_0]_M$, so (κ_0, S) is false. $\square$

For a conclusion kernel of coarser grain the rule is sound but not tight: a block meeting μ is entirely true, so $\text{scope}(c)$ could safely extend to the grain closure of μ . The specification keeps (W1) in its strict form. The conservative choice costs nothing a contributor cannot recover by stating a finer claim.

6.4 Chains need bridges

The most common informal inference form is the chain: e supports c_1 , and c_1 supports c_2 , so e supports c_2 . The model refuses to treat this as automatic.

Theorem 6.6 (No implicit transitivity). *Let a_1 be a well-formed argument with conclusion $c_1 = (\kappa_1, \rho_1, \dots)$.*

- (a) (Chaining intersects scopes.) *For an ordinary warrant $w_2 = (\kappa_1 \Rightarrow \kappa_2; \sigma_2; \dots)$, the arguments with warrant w_2 and filler a_1 that conclude a claim (κ_2, τ) are exactly those with $\tau \subseteq \rho_1 \cap \sigma_2$. Inductively, along a chain from an evidence item e through warrants $w_1, \dots, w_n$, every conclusion scope is contained in $\sigma_e \cap \sigma_{w_1} \cap \dots \cap \sigma_{w_n}$.*
- (b) (Claims alone license nothing.) *If $\kappa_2 \neq \kappa_1$, then $(\kappa_1, \rho_1) \not\models (\kappa_2, \tau)$ for every τ .*
- (c) (Disjoint scopes license nothing.) *If $\rho_1 \cap \sigma_2 = \emptyset$, no claim about κ_2 is licensed by the chain, although each link may be individually well-formed.*

Proof. (a) is (W1) applied at each step, with induction on the chain length. (b) is Theorem 4.15. (c) By (W1) a conclusion scope τ must be nonempty and contained in $\rho_1 \cap \sigma_2 = \emptyset$, which is impossible. $\square$

Remark 6.7 (Derived support is a separate inscription). Suppose b_2 and b'_2 are two well-formed arguments concluding the same claim c_2 , resting on different evidence, and b_3 is an argument for c_3 whose filler is b'_2 . Then c_2 supports c_3 and e , a leaf of b_2 , supports c_2 ; but e is not a leaf of b_3 , and nothing about b_3 depends on e . To have e support c_3 someone must inscribe a further argument $b_3[b_2]$, choosing b_2 as the filler and citing the bridging warrant explicitly. This matters for defeat: attacks propagate through sub-arguments (Chapter 7), so which argument fills a slot decides what can defeat the result. A graph that closed support under transitivity automatically would let an objection to e silently

damage arguments that never used it, and let e lend credit to conclusions its scope never reached.

6.5 What objections will target

Every element of an argument is a place where it can fail, and a different failure needs a different repair. The next chapter treats them formally. In summary, an objection may attack an evidence filler (the item is not anchored, is retracted, or fails to replicate), the warrant (the bridge does not hold in some region, or the claimed rung is too high), the conclusion (an incompatible claim has its own argument), or the scope (the conclusion is asserted beyond the region the premises reach). Keeping these apart in storage is what lets the system say not merely that an argument is contested but where.

Chapter 7

Objections and Defeat

Disagreement in a shared text is usually settled by whoever edits last. In *Adversaria* it is settled by a structure that anyone can inspect: which arguments attack which, on which units, and which attacks succeed under which published policy. This chapter defines that structure. The formal core is a contextual version of Dung’s argumentation frameworks in which every attack carries a region, so that an objection can defeat an argument on some units and leave it standing on others. The sections below give the definitions, and the rest of the chapter proves the guarantees they support.

A word on order. The defeat semantics is developed here, ahead of the chapters on distinction, versioning and relations, because those chapters rest on it: constraint is defeat located in a region, dependency is inheritance of defeat through sub-arguments, and the status vector reads the labels defined here.

7.1 Objections are arguments

An objection is not a comment attached to a sentence. It is an argument whose conclusion denies some component of another argument. This keeps the burden symmetric: to object, one must argue.

Every argument a offers three kinds of *target*, one for each way it can fail.

Definition 7.1 (Targets and denial kernels). Let a be an argument with top-level warrant w and evidence fillers $e_1, \dots, e_j$ (the fillers that are evidence items, not sub-arguments). Its targets, each with a *target scope*, are:

target	type	denied by a conclusion of kernel	target scope
concl	rebut	$\overline{\kappa_0}$, where κ_0 is the kernel of $\text{concl}(a)$	$\text{scope}(\text{concl}(a))$
$\text{ev}(e_i)$	undermine	$\overline{\kappa_{e_i}}$ or $\text{acc}(e_i)$	σ_{e_i}
$\text{war}(w)$	undercut	$\overline{\text{valid}(w)}$	σ_w (ordinary warrants only)

Here $\text{acc}(e)$ is a designated kernel, “the report of e is accurate and applicable at the unit” (the ground on which retraction, fabrication, a failed replication or a mismatch with the estimand is asserted), and $\text{valid}(w)$ is the kernel with $[\text{valid}(w)]_M = \{x : x \in [\kappa_1]_M \cap \dots \cap [\kappa_k]_M \Rightarrow x \in [\kappa_0]_M\}$, so that M validates w iff $\sigma_w \subseteq [\text{valid}(w)]_M$.

The kernel $\text{acc}(e)$ is a name for the ground of an objection to a filler. No semantic constraint ties it to κ_e . An objection concluding $\overline{\text{acc}(e)}$ is a reason to withhold e from an argument; it is not a proof that κ_e is false, which would be a separate rebutting claim.

Definition 7.2 (Structural attack). Let a, b be arguments in the ledger and t a target of a with target scope σ_t . Then b attacks a at t iff $\kappa(\text{concl}(b))$ is a denial kernel for t and the active region

$$\text{reg}(b, a, t) = \sigma(b) \cap \sigma_t \cap \sigma(a)$$

is nonempty. The attack has the type of t .

Each factor of the active region is a reason. The attacker exists only on $\sigma(b)$; it bites the target only on the target's scope; and the attacked argument exists only on $\sigma(a)$. The requirement that the region be nonempty says that an objection must name where it bites: one that never overlaps its target is rejected. Attacks are *derived*, not declared. A contributor filing an objection names the intended target as an audit field, and the record is malformed if the named target does not match the structure of the conclusion.

Three consequences are worth stating.

- *Rebuttal is symmetric.* If b rebuts a then $\kappa(\text{concl}(a)) = \overline{\kappa(\text{concl}(b))}$ and the regions coincide, so a rebuts b with the same region (Proposition 4.16 is the reason the region is $\sigma(a) \cap \sigma(b)$).
- *Undercutting is shared.* A warrant is a reusable object. An argument for $\overline{\text{valid}(w)}$ attacks every argument that uses w , on the region each of them reaches.
- *Two informal objection types need no separate machinery.* A *scope defeater* ("this does not generalize outside a region") is an undercut or a rebut with the region of concern as its active region; (W1) already forbids overreach beyond the premises' scopes. A *relevance attack* ("this evidence does not bear on the stated estimand") is an undercut of the bridging warrant, or an undermine on $\text{acc}(e)$.

7.2 Sub-arguments inherit attacks

An argument is only as secure as its parts. If a filler is defeated, everything built on the filler is exposed.

Definition 7.3 (Hereditary attack). Suppose b attacks a' at t with region $\rho = \text{reg}(b, a', t)$ and $a' \in \text{Sub}(a)$. Then b hereditarily attacks a with region $\rho \cap \sigma(a)$, of the same type and through the same t .

For an ordinary warrant or RESTRICT, (W1) gives $\sigma(a) \subseteq \sigma(a')$; for POOL the sub-argument covers only part of $\sigma(a)$. In every case the hereditary region is the part of ρ where a exists at all. This is the mechanism that makes dependency real: whatever defeats a premise argument defeats every argument that stands on it, on the units they share. The leaves used matter (Remark following Theorem 6.6): an argument that fills a slot with a different sub-argument inherits a different set of attacks.

7.3 Policies and the split between attack and defeat

An attack is a structural fact about the ledger. Whether it *defeats* is a further question, and different communities can reasonably answer it differently. A methodological attack on an underpowered study might defeat it for a causal claim and not for a descriptive one. The model separates the two and makes the second explicit.

Definition 7.4 (Policy). A *policy* p is a versioned ledger object consisting of the admissibility relation Adm_p of Design rule 5.6 and a *preference*: for each ordered pair of arguments (b, a) a scope $\langle b \prec_p a \rangle \in \mathcal{B}(\Omega)$, the units at which b is strictly less preferred than a . It is required that $\langle a \prec a \rangle = \emptyset$ and that $\langle b \prec a \rangle \cap \langle a \prec b \rangle = \emptyset$. No other property, such as transitivity, is required, and preferences need not be totally defined.

The default policy p_0 has $\langle b \prec a \rangle = \emptyset$ for all pairs: no argument is strictly preferred to another. Preferences may encode methodological relevance, directness, temporal priority, preregistration or replication; the specification does not choose. Because they are objects in the ledger, they are versioned, citable and open to objection like everything else. They are also *regional*: an experiment may be preferred to a survey in one population and not in another.

Definition 7.5 (Defeat). Fix a policy p . An attack of b on a' at t *succeeds at x under p* if $x \in \text{reg}(b, a', t)$ and either t is an undermine or undercut target, or $t = \text{concl}$ and $x \notin \langle b \prec_p a' \rangle$. The *defeat region* of b on a is

$$D_p(b, a) = \sigma(a) \cap \bigcup \{ \{x : \text{the attack of } b \text{ on } a' \text{ at } t \text{ succeeds at } x\} : \\ a' \in \text{Sub}(a), t \text{ a target of } a' \},$$

and we write $b \triangleright_p^x a$ when $x \in D_p(b, a)$.

Only rebuttals are preference-sensitive. Undermining and undercutting attack a component of an argument, not its standing relative to a rival, and succeed whenever they apply. This follows the usual convention of structured argumentation. With the default policy every attack is a defeat.

The definition is deliberately *not* numerical. Preferences enter as a regional strict relation on arguments, never as a score. Chapter 12 explains why no scalar is defined anywhere in the system, and Theorem 7.18 shows that the same ledger can support different labelings under different policies without any inconsistency.

7.4 Labeling an argument at a unit

Fix a ledger, a policy p and the finite set $\mathcal{A}$ of *active* arguments (Chapter 9 says which records are active; until then read $\mathcal{A}$ as all admitted arguments). At a unit $x \in \Omega$ the arguments present are $\mathcal{A}_x = \{a \in \mathcal{A} : x \in \sigma(a)\}$. Every defeat at x connects members of $\mathcal{A}_x$, because a defeat region is contained in both the attacker's and the target's scope. So at each unit the ledger yields an ordinary finite defeat graph $(\mathcal{A}_x, \triangleright_p^x)$, and the model applies Dung's grounded semantics to it, unit by unit.

Definition 7.6 (Grounded labeling at x). For $S \subseteq \mathcal{A}_x$ let

$$\Phi_x(S) = \{a \in \mathcal{A}_x : \text{for every } b \triangleright_p^x a \text{ there is } s \in S \text{ with } s \triangleright_p^x b\}.$$

Let in_x be the least fixed point of Φ_x , $\text{out}_x = \{a : \exists s \in \text{in}_x, s \triangleright_p^x a\}$ and $\text{und}_x = \mathcal{A}_x \setminus (\text{in}_x \cup \text{out}_x)$. The label $L(a, x) \in \{\text{in}, \text{out}, \text{und}\}$ is the class of a in this partition. The regions of a are

$$\text{In}(a) = \{x \in \sigma(a) : L(a, x) = \text{in}\}, \quad \text{Out}(a) = \{x : \dots = \text{out}\}, \quad \text{Und}(a) = \{x : \dots = \text{und}\}.$$

The labels record standing under a policy and a ledger. They are not truth. An argument that is in at x is one whose every defeater has itself been defeated; nothing in the semantics of Chapter 4 makes that a guarantee that the conclusion holds, and no theorem below says so.

Theorem 7.7 (Well-definedness and label consistency). *For every unit x :*

- (a) Φ_x is monotone, the iteration $\Phi_x^n(\emptyset)$ increases and stabilizes, and its limit is the least fixed point in_x .
- (b) in_x is conflict-free: no member of in_x defeats another. Hence in_x , out_x , and und_x are pairwise disjoint and cover $\mathcal{A}_x$.
- (c) $a \in \text{in}_x$ iff every defeater of a at x is in out_x .
- (d) $a \in \text{out}_x$ iff some defeater of a at x is in in_x .
- (e) If $a \in \text{und}_x$ then no defeater of a is in in_x and some defeater of a is in und_x .
- (f) (Minimality.) If (I, O, U) is any labeling satisfying (c) and (d) with I for in and O for out, then $\text{in}_x \subseteq I$. The grounded labeling is thus the most skeptical consistent one.

Proof. (a) If $S \subseteq S'$ then the condition “some $s \in S$ defeats b ” implies the same with S' , so $\Phi_x(S) \subseteq \Phi_x(S')$. Hence $\emptyset \subseteq \Phi_x(\emptyset)$ gives an increasing chain, which stabilizes because $\mathcal{A}_x$ is finite, at a set S^* with $\Phi_x(S^*) = S^*$. Any fixed point T contains every $\Phi_x^n(\emptyset)$ by induction, so S^* is the least.

(b) First, each $S_n = \Phi_x^n(\emptyset)$ is conflict-free, by induction: $S_0 = \emptyset$. Suppose S_n is conflict-free and $a, b \in S_{n+1}$ with $b \triangleright_p^x a$. Since $a \in \Phi_x(S_n)$ there is $s \in S_n$ with $s \triangleright_p^x b$; since $b \in \Phi_x(S_n)$ there is $t \in S_n$ with $t \triangleright_p^x s$, contradicting conflict-freeness of S_n . Then in_x is conflict-free as a union of a chain. If $a \in \text{in}_x \cap \text{out}_x$, some $s \in \text{in}_x$ defeats $a \in \text{in}_x$, a conflict. So the three classes are disjoint, and they cover by definition of und_x .

(c) If $a \in \text{in}_x = \Phi_x(\text{in}_x)$, each defeater b of a is defeated by a member of in_x , so $b \in \text{out}_x$. Conversely, if every defeater of a is in out_x , each is defeated by a member of in_x , so $a \in \Phi_x(\text{in}_x) = \text{in}_x$. (d) is the definition of out_x .

(e) If $a \in \text{und}_x$ then $a \notin \text{out}_x$ gives that no defeater is in in_x . Since $a \notin \text{in}_x$, by (c) some defeater b is not in out_x ; it is not in in_x either, so $b \in \text{und}_x$.

(f) Condition (c) for (I, O, U) says every defeater of a is in O iff $a \in I$; and (d) says O is the set of arguments defeated by a member of I . So

$$\begin{aligned} \Phi_x(I) &= \{a : \text{every defeater of } a \text{ is defeated by } I\} \\ &= \{a : \text{every defeater is in } O\} = I \end{aligned}$$

Thus I is a fixed point and contains the least fixed point in_x . $\square$

Theorem 7.8 (Regions). *For every active argument a , the sets $\text{In}(a)$, $\text{Out}(a)$, $\text{Und}(a)$ partition $\sigma(a)$ and each is a scope. If the ledger and policy use k distinct scopes (argument scopes, target scopes and preference regions), each is a union of atoms of the partition of Ω by membership in those k regions, and there are at most 2^k atoms.*

Proof. Let $R_1, \dots, R_k$ be the regions and call the set $\{i : x \in R_i\}$ the *pattern* of x . The set $\mathcal{A}_x$ is determined by the pattern of x , since $x \in \sigma(a)$ is one of the memberships. Each defeat region $D_p(b, a)$ is a finite union, over sub-arguments a' and targets t , of sets of the form $\sigma(a) \cap \sigma(b) \cap \sigma_t \cap \sigma(a')$ possibly minus $\langle b \prec a' \rangle$; whether x belongs to it is again determined by the pattern. So the defeat graph at x , and hence $L(\cdot, x)$, depends on x only through its pattern. Each pattern class is a Boolean combination of $R_1, \dots, R_k$, hence a scope by Proposition 4.4, and $\text{In}(a)$, $\text{Out}(a)$, $\text{Und}(a)$ are unions of such classes. There are at most 2^k patterns. $\square$

The theorem is what makes the status of a claim computable and finitely describable although Ω may be infinite: one evaluates one representative per atom. The bound is exponential in the worst case, and implementation notes address it; ordinary dossiers touch few regions.

Proposition 7.9 (Grain is respected). *Let Γ be a partition of Ω . If every scope and region used by the ledger and policy is a union of blocks of Γ , then $L(a, x) = L(a, x')$ whenever x, x' lie in the same block.*

Proof. Units in one block have the same pattern. $\square$

This is the operational meaning of the grain of Chapter 4. A record that would cut an indivisible cell of an aggregate kernel is not admitted as it stands; its author is directed to state a finer kernel first (Chapter 8), after which the cut falls between blocks.

7.5 Three structural guarantees

Theorem 7.10 (Objections are local). *Let Σ be a scope and let the ledger be extended by a set N of new arguments, each with scope contained in Σ . Then for every unit $x \notin \Sigma$ and every argument a already present, $L(a, x)$ is unchanged. In particular this holds for a single objection b with $\Sigma = \sigma(b)$, together with any restrictions of b or arguments built on b by an ordinary warrant.*

Proof. No new argument is present at $x \notin \Sigma$. Every defeat involving a new argument, as attacker or as target, has region contained in that argument's scope, hence in Σ . New arguments cannot be sub-arguments of arguments already present, so no existing argument acquires a new hereditary defeater. Thus for $x \notin \Sigma$ the defeat graph $(\mathcal{A}_x, \triangleright^x)$ is identical before and after, and so are its labels. $\square$

(An argument built on b with POOL is not covered, since it may extend beyond $\sigma(b)$ using other fillers; it is a new claim with new scope and is treated as such.)

This is the formal content of “an objection cannot reach beyond the scope it declares.” It also bounds the damage of a careless or hostile contribution: it can only ever alter labels on the units where it has standing.

Theorem 7.11 (Directionality and non-explosion). *Fix x . The backward closure $B_x(a)$ is the smallest set containing a and containing every $b \in \mathcal{A}_x$ that defeats a member of the set. Then $L(a, x)$ is determined by the defeat graph restricted to $B_x(a)$. Consequently, if new arguments and defeats are added but no new defeater of any member of $B_x(a)$ is introduced, $L(a, x)$ is unchanged.*

Proof. Write $U = B_x(a)$; it is closed under taking defeaters. Let Φ_U be the operator of the framework restricted to U . We show $\Phi_x^n(\emptyset) \cap U = \Phi_U^n(\emptyset)$ by induction on n ; $n = 0$ is trivial. Suppose it holds for n and put $T = \Phi_U^n(\emptyset)$. For $c \in U$, $c \in \Phi_x^{n+1}(\emptyset)$ iff each defeater b of c is defeated by some $s \in \Phi_x^n(\emptyset)$. Such a b lies in U , and s , defeating b , lies in U too, so $s \in \Phi_x^n(\emptyset) \cap U = T$. Conversely any $s \in T$ lies in $\Phi_x^n(\emptyset)$. So the condition is equivalent to $c \in \Phi_U(T)$. Hence the least fixed points agree on U , and so do the out-sets, since an out member of U is defeated by an in member, which lies in U . $\square$

Corollary 7.12 (Contradiction does not explode). *Suppose the ledger contains arguments for a claim and for its opposite with overlapping scopes. Any argument a whose backward closure at x contains neither of them keeps its label at x , and no new argument for any other conclusion comes into existence.*

Proof. The first assertion is Theorem 7.11. For the second, arguments exist only when someone inscribes them with explicit fillers and warrants; nothing in the semantics generates arguments from the mere presence of a conflict. $\square$

Remark 7.13 (Where explosion is avoided). The model theory of Chapter 4 is classical: two universal claims on opposite kernels with overlapping scopes have no common model and would entail everything. The system avoids explosion by never asking that theory to reason from the ledger's total contents. Truth in models is used only to justify a single argument (Theorem 6.4), whose hypotheses concern that argument's own leaves and warrants; the status of a claim is computed by the local labeling above. A contested claim is a labeled state, not an inconsistency of the store.

Lemma 7.14 (Dependency soundness). *Let $a' \in \text{Sub}(a)$ and $x \in \sigma(a) \cap \sigma(a')$. (i) If $L(a, x) = \text{in}$ then $L(a', x) = \text{in}$. (ii) If $L(a', x) = \text{out}$ then $L(a, x) = \text{out}$.*

Proof. By Definition 7.3, every defeat of a' at x is a defeat of a at x (the region is intersected with $\sigma(a) \ni x$, and $\text{Sub}(a') \subseteq \text{Sub}(a)$). (i) If a is in, all its defeaters at x are out by Theorem 7.7(c); the defeaters of a' are among them, so a' is in. (ii) If a' is out, some $s \in \text{in}_x$ defeats a' , hence defeats a , so a is out. $\square$

A conclusion is never better established than what it stands on, and it falls with what it stands on, at exactly the units they share.

7.6 Disagreement, cycles and reinstatement

The guarantees above hold whatever the defeat graph looks like. The results below say what particular shapes of disagreement look like once labeled. Each is stated at a single unit x ; regions follow by reading them across all units.

Proposition 7.15 (Acyclic disagreement is always decided). *If the defeat graph at x has no directed cycle, then $\text{und}_x = \emptyset$.*

Proof. Define the depth of a as the length of the longest defeat path ending at a ; it is finite because the graph is finite and acyclic. Induct on depth. Every defeater of a has smaller depth, so by hypothesis lies in $\text{in}_x \cup \text{out}_x$. If some defeater is in in_x then $a \in \text{out}_x$. Otherwise all defeaters are in out_x , and $a \in \text{in}_x$ by Theorem 7.7(c). $\square$

Proposition 7.16 (Symmetric defeat stays open; reinstatement resolves it). *Let a, b defeat one another at x .*

- (a) *If neither has any other defeater at x , then $a, b \in \text{und}_x$.*
- (b) *If a has no defeater other than b and $s \in \text{in}_x$ defeats b , then $a \in \text{in}_x$ and $b \in \text{out}_x$.*
- (c) *More generally (reinstatement), if every defeater of a is in out_x then $a \in \text{in}_x$.*

More generally, a defeat cycle in which no member has a defeater outside the cycle leaves all its members in und_x .

Proof. (a) $\Phi_x(\emptyset)$ consists of the unattacked arguments, which excludes a and b . On $\{a, b\}$ the least fixed point is therefore empty, so neither is in in_x , and neither is defeated by a member of $\text{in}_x = \emptyset$. The same argument covers any cycle without outside defeaters: no member is unattacked, so in_x misses the cycle and out_x does too. (b) $b \in \text{out}_x$ by definition, and its being out makes every defeater of a out, so $a \in \text{in}_x$ by Theorem 7.7(c). (c) is Theorem 7.7(c). $\square$

The grounded semantics never resolves a symmetric conflict by fiat. If two rebutting arguments stand on equal footing, both remain unresolved until someone adds a defeater, a reply or a preference. This is the behavior the dossier needs: an honest “contested” in place of a winner chosen by whoever edited last.

Combining this with conflict localization (Proposition 4.16) gives the basic picture of scoped disagreement.

Proposition 7.17 (Scoped disagreement). *Let p be an argument for $c = (\kappa, \sigma)$ and q an argument for $d = (\bar{\kappa}, \tau)$ with $\sigma(p) = \sigma$ and $\sigma(q) = \tau$, and assume that they are the only arguments in the ledger and have no sub-arguments. Under the default policy,*

$$\text{In}(p) = \sigma \setminus \tau, \quad \text{Und}(p) = \sigma \cap \tau, \quad \text{In}(q) = \tau \setminus \sigma, \quad \text{Und}(q) = \sigma \cap \tau.$$

If a policy makes q strictly less preferred than p exactly on $\rho \subseteq \sigma \cap \tau$, then in addition $\text{In}(p) \supseteq \rho$ and $\text{Out}(q) = \rho$, and $\text{Und}(p) = \text{Und}(q) = (\sigma \cap \tau) \setminus \rho$.

Proof. By Definition 7.2 the two rebut one another with active region $\sigma \cap \tau$. Outside that region no attack applies, so each is unattacked and in on its own scope by Theorem 7.7(c). On $\sigma \cap \tau$ apply Proposition 7.16(a). Under the stated policy p defeats q on all of $\sigma \cap \tau$ while q defeats p only off ρ : at units of ρ , q has no successful attack on p , so p is unattacked and in, and q is out; at the remaining units the defeat is still symmetric. $\square$

Two rival positions therefore coexist in one dossier, each standing on the part of its scope the other does not reach, and each showing exactly which units remain contested.

7.7 Policies, and objects that fall out of play

Theorem 7.18 (Policy-relative defeat). (a) *Theorems 7.7, 7.8, 7.10, 7.11 and Lemma 7.14 hold for every policy.*

(b) *Two policies can assign different labels to the same ledger with identical structural attacks.*

Proof. (a) Each proof uses only that, for each unit, the defeat graph is finite (and, for Theorem 7.8, that preference regions are scopes, which Definition 7.4 requires). (b) Let $\Omega = \{x\}$, and let a, b be arguments with opposite conclusion kernels and scope Ω . Structurally each rebuts the other. Under p_0 both are und by Proposition 7.16(a). Under a policy with $\langle b \prec a \rangle = \Omega$, the attack of b on a fails while that of a on b succeeds, so a is in and b is out. $\square$

Since policies are ledger objects, two communities can share every record and differ only in a policy, and the difference in their dossiers is then traceable to that policy (Theorem 9.13).

Lemma 7.19 (Arguments that are out are inert). *If $b \in \text{out}_x$, deleting b and all defeats involving it leaves the labels of all other arguments at x unchanged.*

Proof. Let Φ, Φ' be the operators before and after deletion, $\text{in} = \text{in}_x$, and in' the least fixed point of Φ' . Note $b \notin \text{in}$.

$\text{in}' \subseteq \text{in}$. Let $a \in \Phi'(\text{in})$: each defeater of a other than b is defeated by a member of in . Since $b \in \text{out}_x$, b is also defeated by a member of in . So every defeater of a is defeated by in , and $a \in \Phi(\text{in}) = \text{in}$. Thus $\Phi'(\text{in}) \subseteq \text{in}$, and by monotonicity $\Phi'^n(\emptyset) \subseteq \text{in}$ for all n .

$\text{in} \subseteq \text{in}'$. Put $S_n = \Phi^n(\emptyset)$ and $S'_n = \Phi'^n(\emptyset)$. We show $S_n \subseteq S'_n$ by induction. If $a \in S_{n+1}$, each defeater of a is defeated by a member of $S_n \subseteq \text{in}$; members of in differ from b , so the defeaters of a remaining after deletion are defeated by members of $S_n \subseteq S'_n$, giving $a \in S'_{n+1}$.

So $\text{in} = \text{in}'$. An argument is out after deletion iff a member of $\text{in}' = \text{in}$ defeats it, and b was never such a member, so the out sets agree too. $\square$

The lemma is what makes withdrawal safe (Chapter 9): an argument that has fallen out of play can be removed from consideration without disturbing anything else, and nothing it attacked stays attacked.

7.8 A worked example

The example is schematic. It exercises the definitions and reports no empirical result. Take two dimensions, population (adults A , adolescents K) and context (daily life D , laboratory L), giving four units AD, AL, KD, KL . Let κ be a kernel “differentiation improves regulation,” with a fixed operational meaning. There are three arguments p_1, p_2, p_3 , each from one evidence item, concluding κ over $\{AD\}, \{AL\}, \{KD\}$ respectively, and a pooling argument a (structural warrant POOL) concluding $c = (\kappa, \{AD, AL, KD\})$ with those three as sub-arguments. Nothing addresses KL .

An objection q from a different evidence item concludes $(\bar{\kappa}, \{KD\})$. It rebuts p_3 and a , both symmetrically, over $\{KD\}$. A reply r concludes $\text{acc}(e_q)$ over $\{KD\}$: it undermines the evidence of q . Under a second policy, q is strictly less preferred than p_3 and than a over $\{KD\}$.

stage	AD	AL	KD	KL
1. no objection	a in	a in	a in	no argument
2. q filed, default policy	a in	a in	a, p_3, q undecided	no argument
3. reply r filed	a in	a in	a, p_3 in; q out; r in	no argument
2'. q filed, $q \prec p_3, a$ on KD	a in	a in	a, p_3 in; q out	no argument

At stage 2 the claim c is supported on the adult cells and open on KD : $\text{In}(a) = \{AD, AL\}$, $\text{Und}(a) = \{KD\}$. A single objection has narrowed what stands without deleting anything and without touching the other cells (Theorem 7.10). At stage 3 the reply defeats q , and p_3 and a are reinstated on KD (Proposition 7.16(b) and Lemma 7.14(ii) in the other direction). Stage 2' reaches the same standing on KD by a different route: the policy, not a reply, decides which rebuttal succeeds. Both routes are visible in the record, and the difference between them is a fact about the ledger and the policy, not about the claim. The labels above were also computed independently by a short program implementing Definitions 7.3–7.6 and agree with the table.

Chapter 8

Distinction, Dependency and Gluing

Most long disputes are not disputes about a fact. They are disputes about which of several things a sentence means, about how much of the truth of one claim rests on another, or about pages that were written separately and never checked against one another. This chapter gives each a precise form. Distinction splits an overloaded term into its senses without rewriting anyone’s words. Dependency records that one conclusion stands on another and makes the dependence enforceable. Gluing asks when separately written local accounts can be assembled into one, and names the obstruction when they cannot.

8.1 Overloaded terms and senses

A phrase such as “emotional differentiation” can stand for several constructs with different operational meanings. Claims made about the phrase then look like one dispute and are really several. Adversaria does not resolve this by choosing a meaning. It records the overload and lets each sense carry its own dossier.

Let $s \in D$ be a dimension of *sense* with $U_s = \{s_1, \dots, s_k\}$, $k \geq 2$, and write $\Omega_i = \{x \in \Omega : x_s = s_i\}$ for the slice of units at sense s_i .

Definition 8.1 (Pooling declaration). A *pooling declaration* is a tuple $\Delta = (\kappa^*; s; \kappa_1, \dots, \kappa_k)$ of kernels sharing a grain that refines the partition $\{\Omega_1, \dots, \Omega_k\}$. Here κ_i is the kernel with the operational meaning μ_i of sense s_i , and κ^* is the *umbrella* kernel, whose statement is the overloaded phrase. A model M *satisfies* Δ if

$$[\kappa^*]_M = \bigcup_{i=1}^k ([\kappa_i]_M \cap \Omega_i).$$

The umbrella kernel holds at a unit exactly when the kernel of that unit’s sense does. A declaration is a ledger record: an argument that the term is overloaded, contestable like any other claim, and it changes no existing record.

Theorem 8.2 (Decomposition). *Let M satisfy Δ and let σ be a scope admissible for κ^* . Then $M \models (\kappa^*, \sigma)$ iff $M \models (\kappa_i, \sigma \cap \Omega_i)$ for every i with $\sigma \cap \Omega_i \neq \emptyset$.*

Proof. $\sigma \subseteq [\kappa^*]_M$ iff for each i , $\sigma \cap \Omega_i \subseteq [\kappa^*]_M \cap \Omega_i = [\kappa_i]_M \cap \Omega_i$, iff $\sigma \cap \Omega_i \subseteq [\kappa_i]_M$. The scope $\sigma \cap \Omega_i$ is admissible for κ_i because the common grain refines the slices. $\square$

A claim about the umbrella term is therefore exactly the conjunction of its claims about the senses, each over its own slice. No content is created or destroyed by a distinction.

Theorem 8.3 (No generalization across senses). *Let $I \subsetneq \{1, \dots, k\}$ and let σ be an admissible scope meeting Ω_j for some $j \notin I$. No set of claims about kernels κ_i with $i \in I$ entails (κ^*, σ) in the models of Δ .*

Proof. Set $[\kappa_i]_M = \Omega$ for $i \in I$, $[\kappa_j]_M = \emptyset$ for $j \notin I$, and define $[\kappa^*]_M$ by Δ , other kernels arbitrary. Every claim about a κ_i with $i \in I$ is true in M . A unit of $\sigma \cap \Omega_j$ lies outside $[\kappa^*]_M$, so (κ^*, σ) is false in M . $\square$

This is the formal reason a finding about one construct cannot silently stand in for another under a shared name. Evidence for one sense supports the umbrella claim only on that sense's slice, through an explicit LIFT step that takes fillers for $\kappa_1, \dots, \kappa_k$ over slices to κ^* over their union. LIFT is a structural warrant like POOL: valid in all models of Δ by Theorem 8.2, with $\text{scope}(c)$ contained in the union of the filler scopes and no filler required for a slice the conclusion does not reach.

8.2 Restriction and pooling preserve status

A distinction, a constraint or a pooling step makes new claims out of old ones. It matters whether the new claims inherit the standing of the old ones or acquire standing they have not earned. The first requirement is a condition on policies.

Design rule 8.4 (Restriction-coherent preference). If a' is a restriction of a (the argument $\text{RESTRICT}(a)$ with $\sigma(a') \subseteq \sigma(a)$), a policy must treat a' as it treats a on the units a' covers: $\langle a' \prec b \rangle \cap \sigma(a') = \langle a \prec b \rangle \cap \sigma(a')$ and $\langle b \prec a' \rangle \cap \sigma(a') = \langle b \prec a \rangle \cap \sigma(a')$ for every b .

Without the rule, a policy could rank an argument and its own restriction differently and thereby manufacture a preference out of a notational change.

Lemma 8.5 (Twins). *Fix x and suppose $a, a' \in \mathcal{A}_x$ neither defeat each other, have the same defeaters at x , and defeat the same arguments at x . Then $L(a, x) = L(a', x)$, and deleting a' leaves the labels of all other arguments at x unchanged.*

Proof. Let Φ, Φ_0 be the operators of the framework with and without a' , and put $S_n = \Phi^n(\emptyset)$, $S_n^0 = \Phi_0^n(\emptyset)$. We show by induction that $S_n \setminus \{a'\} = S_n^0$ and that $a' \in S_n$ iff $a \in S_n^0$. The case $n = 0$ is trivial. Assume it for n .

Let $c \neq a'$. The defeaters of c in the larger framework are those in the smaller one, together with a' if a' defeats c ; and then a defeats c too. For a defeater $d \neq a'$, “some member of S_n defeats d ” holds iff some member of S_n^0 does: a witness a' can be replaced by a (twins defeat the same arguments) and $a' \in S_n$ iff $a \in S_n^0$. For the defeater $d = a'$, “some member of S_n defeats a' ” holds iff some member of S_n^0 defeats a (same defeaters, and a' itself is not one of them). Hence $c \in \Phi(S_n)$ iff $c \in \Phi_0(S_n^0)$. The same reasoning applied to a' in place of c gives $a' \in \Phi(S_n)$ iff $a \in \Phi_0(S_n^0)$.

Thus the least fixed points correspond in the same way. An argument is out iff a member of in defeats it, and a' contributes nothing new as a defeater since a does the same, so the out sets correspond too. $\square$

Theorem 8.6 (Restriction preserves status). *Let policies be restriction-coherent. Let a be a well-formed argument, $\tau \subseteq \sigma(a)$ an admissible nonempty scope and $a' = \text{RESTRICT}(a)$ the argument concluding $\text{concl}(a)|_\tau$. Then for every $x \in \tau$, $L(a', x) = L(a, x)$, and adding a' to the ledger changes no label of any argument already present.*

Proof. Fix $x \in \tau$. We check the twin hypotheses of Lemma 8.5. Defeaters of a' at x : any attack on a' is on its conclusion, its filler a , or a sub-argument of a ; the first is a rebuttal of the same kernel as $\text{concl}(a)$ with a region that at x coincides with that of the corresponding rebuttal of a ; the others are hereditary attacks that are also hereditary attacks on a . RESTRICT has no warrant or evidence of its own to target. Success under the policy agrees by Design rule 8.4. Conversely every defeater of a at x is a defeater of a' at x because $x \in \tau = \sigma(a')$. The same reasoning applies to what a and a' defeat. Neither defeats the other: they have the same conclusion kernel, not opposite ones, and by (W5) that kernel denies no component of the tree they share, since a' has a as its filler. Lemma 8.5 gives both assertions at $x \in \tau$. For $x \notin \tau$, a' is absent, so nothing changes. $\square$

The theorem is the foundation for constraint (Chapter 11): the surviving remainder of a challenged claim, stated as a restriction, is in exactly where the original was, and no standing appears from the act of restating.

8.3 Dependency

Every argument depends on the sub-arguments that fill its slots, and a claim depends on the claims those sub-arguments conclude.

Definition 8.7 (Dependence between claims). Claim c *depends on* claim d over τ when some well-formed argument for c has a sub-argument concluding d , τ being the intersection of the two scopes.

Lemma 7.14 is the dependency guarantee: an argument is in at a unit only if every sub-argument present there is in, and it is out whenever one is. Two further facts follow from the definitions and are worth stating, because both concern circularity.

Proposition 8.8 (Every argument grounds out in evidence). *Every well-formed argument has at least one leaf, and all its leaves are anchored evidence items.*

Proof. Induction on height. An ordinary warrant has $k \geq 1$ premises, and each filler is an evidence item, which is a leaf, or a smaller argument, which has a leaf by hypothesis. RESTRICT , POOL and LIFT have at least one filler. Anchoring is (W3). $\square$

Proposition 8.9 (Cycles among claims do not bootstrap support). *The dependence relation between claims may contain cycles, while every argument is a finite well-founded tree.*

Proof. Let c_1 have an argument whose sub-argument concludes c_2 , and let c_2 have a different argument whose sub-argument concludes c_1 . Both are finite trees, and each must be inscribed after its sub-arguments; the two arguments have different roots and share no path. Every tree has anchored leaves by Proposition 8.8, so neither claim is supported by the other's support alone. $\square$

Mutual dependence between claims is a legitimate feature of coherent bodies of knowledge, which is why it is permitted. What is never permitted is a proof that supports itself: an argument cannot have itself among its sub-arguments. Claim-level cycles are flagged in the dossier as a fact about its structure, and the labeling of Chapter 7 is unaffected by them.

8.4 Pages as local sections

Encyclopedic knowledge is written in modules. A page treats one topic, one tradition or one language edition, and its authors check it against the sources they know. Nobody checks the collection. Suppose separate pages, in separate traditions or language editions, each discuss a figure under the same name, and each makes claims about how its figure relates to the figure discussed elsewhere. Each page may be locally coherent and every pair of pages may look compatible on its own. The collection can still fail to assemble into one account, and no editor of any single page is in a position to notice. The name for that situation is a gluing failure, and this section gives the problem and the one case in which its obstruction can be characterized completely.

8.4.1 The general gluing problem

Definition 8.10 (Pages and gluing). Let $\mathcal{K}$ be a finite set of binary variables (kernels, each read as true or false at a unit). A *page* is a triple $P_i = (U_i, \mathcal{K}_i, C_i)$ with a scope U_i , a sub-vocabulary $\mathcal{K}_i \subseteq \mathcal{K}$, and, for each $x \in U_i$, a set $C_i(x) \subseteq \{0,1\}^{\mathcal{K}_i}$ of *locally admissible* assignments: what the page permits at x . A *global solution* at x is an assignment $\tau \in \{0,1\}^{\mathcal{K}}$ with $\tau|_{\mathcal{K}_i} \in C_i(x)$ for every page with $x \in U_i$. Pages P_i, P_j are *compatible* at x if the projections of $C_i(x)$ and $C_j(x)$ to $\mathcal{K}_i \cap \mathcal{K}_j$ intersect. The family is *pairwise compatible* if every pair is compatible at every unit both cover. An *obstruction* at x is a pairwise compatible family with no global solution at x .

Whether pairwise compatibility suffices depends on how the vocabularies of the pages overlap. I record three facts of different strength.

Proposition 8.11 (One kernel: pairwise checking suffices). *Let $\{(\kappa^{\varepsilon_i}, \sigma_i)\}_i$ be finitely many claims about one kernel, with $\kappa^+ = \kappa$, $\kappa^- = \bar{\kappa}$ and $\varepsilon_i \in \{+, -\}$. They have a common model iff $\sigma_i \cap \sigma_j = \emptyset$ whenever $\varepsilon_i \neq \varepsilon_j$.*

Proof. $(\Rightarrow)$ is Proposition 4.16. $(\Leftarrow)$ Let $P = \bigcup_{\varepsilon_i=+} \sigma_i$ and $N = \bigcup_{\varepsilon_j=-} \sigma_j$. Then $P \cap N = \bigcup (\sigma_i \cap \sigma_j) = \emptyset$ over pairs of opposite polarity. Take $[\kappa]_M = P$, admissible as a union of admissible scopes, so that $N \subseteq \Omega \setminus P = [\bar{\kappa}]_M$. $\square$

This is the gluing (sheaf) condition for universal claims of Theorem 4.11: for a single kernel, local agreement on overlaps is all there is to global agreement. Two facts from the literature mark its limits; they are cited, not reproved here. For relational data, pairwise consistency of local tables implies global consistency exactly when the hypergraph of the tables' vocabularies is acyclic (Beeri, Fagin, Maier and Yannakakis, 1983). And for general systems of local sections, pairwise-compatible families with no global section arise routinely; this is the phenomenon studied as contextuality (Abramsky and Brandenburger, 2011). Cyclic overlap of vocabularies is the place where obstructions live.

8.4.2 Signed identifications, exactly

The case that matters most for pages about “the same” entity is the identification of one page’s referent with another’s.

Definition 8.12 (Signed identification). Let $v_1, \dots, v_n$ be binary variables, thought of as ± 1 -valued at each unit. A *signed identification* is a triple $e = (\{i, j\}, s_e, R_e)$ with $s_e \in \{+1, -1\}$ and R_e a scope, asserting that on every unit of R_e , $v_j = s_e v_i$: same value if $s_e = +1$, opposite if $s_e = -1$. A family $\mathcal{E}$ of such claims is *satisfiable* if some assignment $v_i : \Omega \rightarrow \{\pm 1\}$ satisfies each on its region. For a unit x let G_x be the signed multigraph on $\{1, \dots, n\}$ whose edges are the e with $x \in R_e$. A cycle is *negative* if the product of its signs is -1 , and G_x is *balanced* if it has no negative cycle.

Theorem 8.13 (Signed cycle criterion). *At each unit x there is an assignment $v \in \{\pm 1\}^n$ satisfying all edges of G_x iff G_x is balanced.*

Proof. ($\Rightarrow$) Let $i_1, \dots, i_m$ be a cycle with edges $e_1, \dots, e_m$ joining consecutive vertices and back. From $v_{i_{r+1}} = s_{e_r} v_{i_r}$ we get, going once around, $v_{i_1} = (\prod_r s_{e_r}) v_{i_1}$, and since $v_{i_1} = \pm 1 \neq 0$ the product is $+1$.

($\Leftarrow$) Suppose G_x is balanced. In each connected component choose a spanning tree and a root, and set $v_{\text{root}} = +1$. For any vertex u set v_u to the product of the signs along the tree path from the root. Tree edges are then satisfied by construction. A non-tree edge e joining u, w closes a cycle with the two tree paths; that cycle is positive, its sign being s_e times the product of the two path signs, that is $s_e v_u v_w = +1$. Hence $v_w = s_e v_u$, and e is satisfied. $\square$

Counterexample 8.14 (Locally fine, globally impossible). Take three variables and three identifications on a common region $R \neq \emptyset$: $e_{12} = (\{1, 2\}, +, R)$, $e_{23} = (\{2, 3\}, +, R)$ and $e_{31} = (\{3, 1\}, -, R)$. Any single edge is satisfiable, and so is any pair: two edges form a path, whose endpoints can always be assigned consistently. Every pair of pages is therefore compatible on every unit. But the triangle has sign product $(+1)(+1)(-1) = -1$, so at every unit of R there is no assignment, by Theorem 8.13. Nothing in the family is a pairwise contradiction, and no argument defeats another.

The example is the point of this section. Contradiction detection that examines claims two at a time cannot see it, and neither can the labeling of Chapter 7, which resolves pairwise defeat. Here every argument is unattacked and in at every unit of R , and the collection is unsatisfiable.

Definition 8.15 (Obstruction region). For a family $\mathcal{E}$ of signed identifications, the *obstruction region* is $\text{Ob}(\mathcal{E}) = \{x \in \Omega : G_x \text{ is unbalanced}\}$.

Theorem 8.16 (The obstruction region is a scope and characterizes satisfiability). *Let $\mathcal{E}$ be a finite family of signed identifications.*

- (a) $\text{Ob}(\mathcal{E}) = \bigcup_C \bigcap_{e \in C} R_e$, the union over negative simple cycles C of the multigraph of all edges; hence $\text{Ob}(\mathcal{E})$ is a scope.
- (b) $\mathcal{E}$ is satisfiable iff $\text{Ob}(\mathcal{E}) = \emptyset$.
- (c) Ob is monotone: shrinking any of the regions R_e (or deleting an edge) never enlarges it.

Proof. (a) G_x is unbalanced iff it contains a negative simple cycle. (Fundamental cycles are simple, so the proof of Theorem 8.13 shows that if all simple cycles are positive the graph is balanced; conversely a negative simple cycle is a negative cycle.) A simple cycle of the full multigraph lies in G_x iff $x \in R_e$ for each of its edges. Finitely many simple cycles exist, so Ob is a finite union of finite intersections of scopes, a scope by Proposition 4.4. (b) Units are independent: an assignment $v : \Omega \rightarrow \{\pm 1\}^n$ is a choice of assignment at each unit. By Theorem 8.13 a choice satisfying the edges active at x exists for every x iff every G_x is balanced. (c) The formula in (a) is monotone in each R_e . $\square$

8.4.3 The cohomological reading, and its limits

Identify ± 1 with $\mathbb{Z}/2$ by $s = (-1)^{\bar{s}}$. For fixed x the signs form a function $\bar{s}$ from the edges of the graph G_x to $\mathbb{Z}/2$, a 1-cochain. A choice of assignment v is a 0-cochain $\bar{v}$ on the vertices, and its coboundary is $(\delta\bar{v})(e) = \bar{v}_i + \bar{v}_j$. The edges are all satisfied iff $\bar{s} = \delta\bar{v}$. A graph has no 2-cells, so every 1-cochain is a cocycle and $H^1(G_x; \mathbb{Z}/2) = C^1 / \delta C^0$. Hence:

Proposition 8.17 (Cohomological form of the criterion). *G_x is balanced iff the class $[\bar{s}] \in H^1(G_x; \mathbb{Z}/2)$ vanishes.*

This is exact for the binary identification model of Theorem 8.13, and only for it. It should not be read as a general criterion. For sheaves of sets or groups other than this one, a nonzero class in a cohomology group of the cover may witness an obstruction; vanishing of H^1 is not by itself a guarantee that local sections glue; and families of pairwise compatible local sections that cannot glue exist for constraints that are not signed binary identifications, whose obstruction may live in a different ech group or in no group at all. The specification proves the signed case (Theorem 8.16), states the cohomology as its interpretation, and treats the wider machinery as an extension to be developed for each new kind of constraint.

8.4.4 What Adversaria does with an obstruction

Consider three pages P_1, P_2, P_3 , in different traditions or language editions, each discussing a figure known by one name. Let v_k record whether the figure on P_k has some specified property, and let identification claims relate them: P_1 asserts its figure is the same as that of P_2 , P_2 asserts the same of P_3 , and P_3 asserts that its figure is distinct from that of P_1 . This is exactly Counterexample 8.14. The example is schematic: it makes no assertion about what any actual page says.

The response of the system has three parts.

1. *Report, do not choose.* The dossier displays Ob and the negative cycles that witness it, the smallest ones first, as a first-class object. The system never selects an identity by fiat and never drops an edge to restore consistency.
2. *Count only standing claims.* The graph used is G_x over identification claims that are in at x ; claims that are und at x are reported separately as a potential obstruction. An objection that defeats one identification removes an edge from the graph at the units it defeats, and can dissolve the obstruction there.

3. *Offer distinction as the repair.* If the name is overloaded, the honest repair is a pooling declaration (Definition 8.1) that gives each sense its own kernel and confines each identification to the slices where it was meant.

Proposition 8.18 (Distinction can dissolve an obstruction and never creates one). *Let $\mathcal{E}'$ be obtained from $\mathcal{E}$ by replacing regions R_e with subregions $R'_e \subseteq R_e$, as happens when identifications are confined to sense slices. Then $\text{Ob}(\mathcal{E}') \subseteq \text{Ob}(\mathcal{E})$. If a negative cycle C has $\bigcap_{e \in C} R'_e = \emptyset$, that cycle contributes nothing to $\text{Ob}(\mathcal{E}')$.*

Proof. Both statements follow from the formula in Theorem 8.16(a). □

In the schematic example, if the first two identifications concern one sense of the name and the third concerns another, no unit lies in all three regions, and the obstruction disappears without any page having been rewritten and without any identification having been deleted. That is the sense in which local pages can be repaired without editing them: the repair is a new record that changes which claim means what where.

Chapter 9

The Ledger

Everything so far has been defined over a finite set of arguments, attacks and policies. This chapter says where that set comes from and what may be done to it. The answer is an append-only ledger of records, from which the argument graph is derived and from which the displayed dossier is derived in turn. Keeping the three apart is the central discipline:

$$\text{ledger} \longrightarrow \text{argument graph} \longrightarrow \text{epistemic presentation}.$$

The ledger records what happened. The argument graph represents inferential relations among what was admitted. The presentation is an interpretation of that graph under a stated policy. Many systems fail because a single interface label such as “accepted” is treated at once as a historical fact, a logical consequence and a community consensus. In Adversaria they are three different objects with three different sources of authority.

9.1 Records and the pipeline

Contributions pass through five distinct transitions:

$$\text{INSCRIBE} \rightarrow \text{VALIDATE} \rightarrow \text{ADMIT} \rightarrow \text{PROJECT} \rightarrow \text{ACTIVATE}.$$

Inscribe stores the submission in the ledger. *Validate* checks schema, integrity, references and anchoring. *Admit* decides whether the record is a recognizable argumentative act. *Project* decides whether it appears in a particular dossier. *Activate* permits it to influence labels, status and search ranking. The first is the only one every submission receives without conditions. None entails the next:

$$\text{stored}(r) \not\Rightarrow \text{admitted}(r) \not\Rightarrow \text{endorsed}(r).$$

This chapter formalizes the ledger, admission and activation. Projection and its governance belong to the protocol part; only what evaluation may depend on is fixed here (Section 9.8).

Definition 9.1 (Record). A *record* is a tuple $r = (\text{type}, \text{payload}, \text{refs}, \text{author}, \text{origin}, \text{trace})$. The type is one of claim, evidence, argument, policy, distinction, restriction, revision, equivalence, withdrawal, and so on. The payload carries the fields defined in Chapters 4–7. The references are identifiers of earlier records that the payload cites. The origin is one of {human, automated, assisted, imported}. The trace is defined in Section 9.2.

9.1.1 Three identifiers

A record needs several notions of sameness, and they must not be conflated.

Definition 9.2 (Identifiers). Let H be a hash function and canon a canonical encoding. The *content hash* of r is

$$H(\text{canon}(\text{type}, \text{payload}, \text{refs})).$$

The *record identifier* is $H(\text{content hash}, \text{author}, \text{sequence})$, where the sequence distinguishes repeated submissions by one author. The *family identifier* is the class of r under the equivalence relation generated by the equivalence claims that are in under the current policy.

The three answer different questions. The content hash supports integrity: has anything changed, and is this the same content? The record identifier names an assertion event: who inscribed it, in what order. The family identifier collects statements proposed to express one proposition; those proposals are themselves contestable claims and never automatic deduplications.

Proposition 9.3 (Identity). Assume H is injective on canonical encodings (the standard idealization of collision resistance, used as a modelling assumption and not proved).

- (a) Two records with identical type, payload and references have the same content hash. If their authors or sequence numbers differ, they have distinct record identifiers.
- (b) Changing any of type, payload or references changes the content hash and therefore the record identifier.
- (c) Records with the same content hash lie in the same family.

Proof. (a) and (b) are immediate from injectivity and from the definitions. (c) Identical content is the same proposition, so the equivalence relation of Definition 9.2 is taken to contain the relation “same content hash”. $\square$

Independent confirmations are counted by independence families (Chapter 5), never by record identifiers. Two authors who submit the same content produce one content hash, two record identifiers and, unless a separate, contestable claim of independence says otherwise, one independence family.

The family identifier is itself an instance of the gluing problem. Equivalence claims are identifications of sign $+1$ between records; a claim that two records express *different* propositions is an identification of sign -1 . A family is formed from in equivalence claims, and an odd cycle mixing the two kinds is exactly a negative cycle in the sense of Theorem 8.13. The obstruction region of Chapter 8 therefore flags families whose members cannot all be identified as asserted.

9.2 Trace completeness

A contribution attributed to an agent, human or automated, is a claim about an act. It is complete only when the act’s causal trace is resolved, and until then it is an attribution

and nothing more. An attribution of authorship does not establish that the contribution supports what it claims to support:

$$\text{ProposedBy}(a, c) \not\Rightarrow \text{Supports}(a, c).$$

Definition 9.4 (Trace). The *trace* of an act record has six components: the *sources transformed* (evidence items or earlier records the act used), the *warrant applied*, the *scope inferred*, the *earlier records that conditioned the output*, the *independence family*, and the *disposition* that completed the act. A component is *resolved* if it is present and, where it is a reference, points to a record in the ledger. An act is *trace-null*, written $\text{trace}(r) = \perp$, if any component is unresolved.

The null does not say that the act was worthless. It marks incomplete determination from the position of the ledger: there is a contribution whose derivation cannot yet be inspected. Such a record may be stored, routed for investigation and completed later by anyone able to supply the missing component; it cannot masquerade as completed evidence. The same rule covers text produced by a model with no recorded inputs, an anonymous assertion with no sources, and a bulk import with no methodology, all of which are trace-null for the same reason.

9.3 Dispositions and the active set

Every non-admission has a reason, and the reason is part of the record of what happened.

Definition 9.5 (Disposition). Given a valid ledger $\mathcal{L}$ and policy p , each record r receives a disposition

$$D_p(r, \mathcal{L}) \in \{\text{ADMITTED}, \text{DUPLICATE}, \text{MALFORMED}, \text{UNSCOPED}, \text{QUARANTINED}, \text{GATED}, \text{REFUSED}\}$$

by induction along the reference order: D_p depends only on the fields of r , on the dispositions of the records it references, on p , and on gate and capability records the policy names. In particular an argument is MALFORMED unless well-formed (W1)–(W5) and its fillers are admitted; a claim without scope or modality is UNSCOPED (Design rule 4.18); a trace-null act is QUARANTINED; a record whose content hash is shared with a record of smaller record identifier is DUPLICATE (a canonical tie-break, so that the disposition does not depend on arrival order; on an anchored surface the pair of ledger time and identifier is used, Chapter 16). The *active set* is

$$\text{Act}(\mathcal{L}, p) = \{r \in \mathcal{L} : D_p(r, \mathcal{L}) = \text{ADMITTED}\}.$$

Every non-admission must state the governing rule, the failing field or argumentative requirement, the evidence for the determination, the responsible decision mechanism and the repair or appeal path. A gated or quarantined record keeps its identity and can become admissible later without resubmission. The argument set $\mathcal{A}$ of Chapter 7 is the set of arguments in $\text{Act}(\mathcal{L}, p)$, and the labeling is computed on it and on nothing else. This is what “activation” means formally.

Proposition 9.6 (Inert records do not influence labels). *Let $\mathcal{L}, \mathcal{L}'$ be valid ledgers with $\text{Act}(\mathcal{L}, p) = \text{Act}(\mathcal{L}', p)$. Then the labelings coincide. In particular adding quarantined, gated, duplicate, malformed or unscoped records changes no label.*

Proof. The labeling is a function of the finite set of active arguments and of p (Definition 7.6). $\square$

9.4 Validity and merging

Definition 9.7 (Valid ledger). A finite set $\mathcal{L}$ of records is *valid* if (i) it is *closed*: every reference of every member is a member; (ii) identifiers agree with contents, so that records with equal identifiers are equal; and (iii) there is a linear order of $\mathcal{L}$ in which every record follows all the records it references.

Condition (iii) says that the reference graph is acyclic and is checkable in linear time. Under the hash idealization it is also unavoidable in practice, since a record cannot cite an identifier that depends on its own content; but validity is defined by (iii), and no proof below relies on hashing to obtain acyclicity.

Theorem 9.8 (Validity is preserved by appending and merging). (a) *If $\mathcal{L}$ is valid and $r \notin \mathcal{L}$ is a record whose references all lie in $\mathcal{L}$, then $\mathcal{L} \cup \{r\}$ is valid.*

(b) *If $\mathcal{L}_1, \mathcal{L}_2$ are valid and records with the same identifier in both are equal, then $\mathcal{L}_1 \cup \mathcal{L}_2$ is valid.*

(c) (Prefix stability.) *If $\mathcal{L} \subseteq \mathcal{L}'$ are valid, every record of $\mathcal{L}$ has the same content, references and ancestors in $\mathcal{L}'$ as in $\mathcal{L}$.*

Proof. (a) Place r last in a valid order. (c) Identifiers determine records, and by closure every ancestor of a record already lies in $\mathcal{L}$.

(b) Closure and identifier agreement are inherited from the two ledgers. For acyclicity, suppose the union's reference graph contained a cycle $r_1 \rightarrow r_2 \rightarrow \dots \rightarrow r_m \rightarrow r_1$, and say $r_1 \in \mathcal{L}_1$. Every edge leaving a record is determined by that record's references, and $\mathcal{L}_1$ is closed, so all the ancestors of r_1 , hence every r_i on the cycle, lie in $\mathcal{L}_1$, with their edges. The cycle lies inside $\mathcal{L}_1$, contradicting (iii) there. Any linear extension of the acyclic union graph is the required order. $\square$

9.5 Revision lineage

Claims change: scopes are narrowed after an objection, kernels are split, wording is repaired. The ledger never edits a claim. A revision is a new record that names what it supersedes.

Definition 9.9 (Revision and lineage). A *revision* is a record introducing a new claim version c' with one or more designated references $\text{supersedes}(c') = \{c_1, \dots, c_m\}$ to earlier claim versions. Restriction, distinction and constraint each produce their results as revisions. The *lineage graph* has claim versions as nodes and an edge $c' \rightarrow c_i$ for each supersession.

Several successors of one version are allowed (a fork, when a disputed claim branches into a broader formulation, a restricted one and an opposing one), and a revision with several predecessors is allowed (a merge). Nothing overwrites anything: a superseded version stays addressable, citable and objectable, and its status continues to be computed. “Superseded” is a fact about lineage, displayed as such, and is not a label of the defeat semantics.

Theorem 9.10 (Lineage integrity). *In a valid ledger:*

- (a) *The lineage graph is acyclic, and every version has at least one root ancestor (a version with no supersession references), so no version is orphaned.*
- (b) (History is preserved.) *Every ancestor of a claim version is present in the ledger, and remains present and unchanged in every valid ledger extending it.*
- (c) (No in-place change.) *A revision does not alter the earlier version, and therefore every record that refers to that version still refers to the same content.*

Proof. (a) Supersession references are references, so an edge of the lineage graph is an edge of the reference graph, which is acyclic by Definition 9.7(iii). A finite acyclic graph in which every node has finitely many out-edges reaches a node with none by following out-edges, and that node is a root. (b) Closure gives presence, and Theorem 9.8(c) gives stability. (c) The earlier version is an existing record; by (b) its content is unchanged in any extension, and references name it by identifier, which determines its content. $\square$

9.6 Withdrawal

A contributor may retract an argument. In an append-only ledger retraction cannot be deletion, so it is another record.

Definition 9.11 (Withdrawal). A *withdrawal* of an argument a is a record referencing a , admitted only if authorized (in the default policy, only the author of a may withdraw it). In the argument graph it contributes a node w_a with $\sigma(w_a) = \sigma(a)$ that attacks a on all of $\sigma(a)$, succeeds under every policy, and has no targets: no objection can be filed against a withdrawal.

Theorem 9.12 (Withdrawal is deletion, up to labeling). *Let $\mathcal{L}'$ extend $\mathcal{L}$ by a withdrawal of a . Write Z_x for the set consisting of a and every argument $d \in \mathcal{A}_x$ with $a \in \text{Sub}(d)$ and $x \in \sigma(a)$, for $x \in \sigma(a)$. Then:*

- (a) $L'(a, x) = \text{out}$ for every $x \in \sigma(a)$.
- (b) $L'(d, x) = \text{out}$ for every $d \in Z_x$.
- (c) *For every $x \in \sigma(a)$ and every argument $b \notin Z_x$, $L'(b, x)$ equals the label of b at x in the framework obtained from $\mathcal{A}_x(\mathcal{L})$ by deleting Z_x and all defeats involving its members. At units outside $\sigma(a)$ no label changes.*

Proof. (a) w_a has no defeaters, so it is in at every $x \in \sigma(a)$ by Theorem 7.7(c), and it defeats a , so a is out. (b) Apply Lemma 7.14(ii) to $a \in \text{Sub}(d)$. (c) By (a) and (b) every member of Z_x is out at x . Delete them one at a time. By Lemma 7.19 each deletion leaves

all other labels unchanged, and the remaining members of Z_x remain out (their labels do not change either). After all deletions, w_a defeats no remaining argument, since its defeats were on members of Z_x , so it is an isolated in node and does not affect the rest. The remaining framework is the original one with Z_x removed. For $x \notin \sigma(a)$, w_a is absent and the framework is unchanged. $\square$

An argument that stands on a withdrawn argument by POOL continues to stand at units its other fillers cover, and is out only where the withdrawn part was its only support. Withdrawal changes admissibility without rewriting history: the withdrawn record, and what everyone did with it, stay in the ledger.

9.7 Replay, merging and traceability

Write $\Pi(\mathcal{L}, p)$ for the projection consisting of the active set, the labeling and the regions computed from them.

Theorem 9.13 (Replay determinism and convergence). *(a) $\Pi(\mathcal{L}, p)$ depends only on the set $\mathcal{L}$ and on p , not on the order or multiplicity with which records arrived.*

(b) Merging is union: it is commutative, associative and idempotent on valid ledgers with consistent identifiers. Hence any two replicas holding the same set of records and the same policy hold the same projection, whatever the order of exchange.

Proof. (a) The disposition $D_p(r, \mathcal{L})$ is defined by recursion along the reference order and depends only on r , on the dispositions of its references, and on p ; the recursion is well-founded on the acyclic reference graph and returns the same value for any linear extension used to evaluate it. The duplicate rule uses record identifiers, not arrival order. The labeling is a function of the active set and p . (b) Union of sets is commutative, associative and idempotent, and it preserves validity by Theorem 9.8(b). $\square$

The convergence is of state, not of belief. Labels are not monotone in the ledger: filing an objection can turn an in argument und, and filing a reply can turn it in again (Section 7.8). Two replicas converge because they end with the same records, not because their statuses only ever move in one direction.

Theorem 9.14 (Federated divergence is traceable). *Let two members hold valid ledgers $\mathcal{L}_1, \mathcal{L}_2$ and policies p_1, p_2 .*

- (a) If $\mathcal{L}_1 = \mathcal{L}_2$ and $p_1 = p_2$ their projections are equal.*
- (b) Suppose $p_1 = p_2$ and $L_1(a, x) \neq L_2(a, x)$. Then the symmetric difference of the two active sets contains an argument in the backward closure $B_x(a)$ computed in the union of the two frameworks.*
- (c) Suppose the two active sets are equal, the admissibility relations are equal, and the two preferences agree on all pairs of active arguments. Then labels agree.*

Consequently every difference between two projections is attributed either to records or to a policy, and, when the policy is shared, to records in an identifiable neighbourhood of the disputed argument.

Proof. (a) is Theorem 9.13(a).

(b) With one policy, the defeat between two arguments is determined by the arguments themselves and p , independent of the rest of the ledger; so the frameworks F_1, F_2 at x are subframeworks of the union framework F and agree on every pair they share. Let $U = B_x(a)$ in F and suppose U contains no member of the symmetric difference, so $U \subseteq \mathcal{A}_1 \cap \mathcal{A}_2$ (if a itself were absent from one member it would be in the difference). All defeaters in F of members of U lie in U , hence in both frameworks, so U is the backward closure of a in F_1 and in F_2 , and the two frameworks agree on it. By Theorem 7.11 the labels of a agree, a contradiction.

(c) The frameworks at every x are then identical, so the labels are. $\square$

9.8 Evaluation depends on arguments only

The architecture separates four functions: storage, routing (which material reaches which person or workspace), admission and evaluation. They exchange information, and none silently determines another. The following is the precise form of that separation for evaluation.

Proposition 9.15 (Evaluation ignores non-argumentative metadata). *The labeling depends on the ledger only through $\text{Act}(\mathcal{L}, p)$ and on p . It does not depend on routing weights, view counts, notification state, search rank, contributor reputation, controversy flags or agreement tallies. Consequently:*

- (a) *If two states have the same active set and policy, their labelings coincide however their metadata differ.*
- (b) *A controversy flag, which may legitimately gate admission, has no bearing on status: flagging a claim does not make it false or unsupported, and unflagging it does not support it.*
- (c) *Passing a capability gate authorizes an act, that is, makes admission possible. It does not make the act's conclusion in: an admitted argument may be out or und under the same rules as any other.*

Proof. The labeling is a function of the active argument set and p (Definition 7.6), and Proposition 9.6 applies. (b) and (c) follow, using in (c) that a gated act's only effect is through D_p and hence through Act ; an example of an admitted argument left und is Proposition 7.16(a). $\square$

What a gate legitimately changes is *whether* a contribution takes part, and that change appears in the ledger as a disposition with a stated reason. It never changes *how* a participating contribution is evaluated. The consequence, developed in the protocol part, is that routing a claim to someone, or restricting who may change a shared surface, is not a truth procedure: relevance to a reader does not establish support, and support does not establish relevance.

Chapter 10

The Relation Algebra

The earlier chapters defined objects and a labeling. This chapter fixes the vocabulary of relations in which a dossier is written, gives it an algebra, and says which compositions are licensed and which are not. Two design decisions govern it. Every relation is *scoped*: it holds over a region, not simply or not at all. And composition is *deliberately incomplete*: chaining two relations yields a third only where the model can say why.

10.1 Ten relation types

Nodes are records of five sorts: evidence items (Ev), warrants (War), arguments (Arg), claims (Cl) and kernels (Ker). Most relations are *derived*: they are read off records already defined and cannot be inscribed independently of the structure that justifies them. A few are *inscribed* as acts.

relation	relates	region over which it holds	origin
SUPPORTS	$Ev \cup Cl \rightarrow Cl$	union of $\sigma(a)$ over active arguments a concluding the target with the source as a direct filler	derived
CONTRADICTS	$Cl \leftrightarrow Cl$	$\sigma(c) \cap \sigma(d)$ when the kernels are opposite	derived
UNDERCUTS_WARRANT	$Arg \rightarrow War$	$\sigma(b) \cap \sigma_w$ when $\kappa(\text{concl}(b)) = \text{valid}(w)$	derived
CONSTRAINS	$Arg \rightarrow Cl$	units where every argument for the target is defeated by the source (Chapter 11)	derived
QUALIFIES	$Cl \rightarrow Cl$	$\sigma(c')$, where c' has the kernel of c and $\sigma(c') \subsetneq \sigma(c)$	inscribed
DEPENDS_ON	$Cl \rightarrow Cl$	intersection of scopes along a sub-argument (Chapter 8)	derived
DISTINGUISHES	$Ker \rightarrow Ker$	the slice Ω_i for each sense i (Definition 8.1)	inscribed
REPLICATES	$Ev \rightarrow Ev$	$\sigma_e \cap \sigma_{e'}$, same kernel, different independence family	derived
FAILS_TO_REPLICATE	$Ev \rightarrow Ev$	$\sigma_e \cap \sigma_{e'}$, where e' declares an attempt to follow the protocol of e , its reported outcome kernel is not that of e , and the families differ	derived from the declaration
SUPERSEDES	$Cl \rightarrow Cl$	$\sigma(c)$ of the superseded version (Definition 9.9)	inscribed

Three remarks on the table. First, FAILS_TO_REPLICATE relates evidence to evidence and is not an objection. It records that a declared attempt (an optional reference in the evidence payload to the item whose protocol it follows) did not reproduce the specified result; it is not evidence for the opposite kernel, which needs its own item and warrant (Chapter 17). A failed replication defeats an argument only through an argument that says so: an argument concluding $\text{acc}(e)$ with e' among its leaves, which then undermines in the sense of Chapter 7. Second, QUALIFIES states a voluntary narrowing of a claim, and by Theorem 4.15 $c \models c'$ whenever c' qualifies c . It is distinct from CONSTRAINS, in which a narrowing is forced by a challenger. Third, every entry is region-valued, and the region is what makes the relation checkable: a relation whose

region is empty does not hold.

10.2 Scoped relations form an algebra

Let N be a finite set of nodes.

Definition 10.1 (Scoped relation). A *scoped relation* on N is a function $R : N \times N \rightarrow \mathcal{B}(\Omega)$. Say a is R -related to b at x if $x \in R(a, b)$. For each unit x the *slice* is the ordinary relation $R_x = \{(a, b) : x \in R(a, b)\}$. Define

$$\begin{aligned} (R \cup S)(a, b) &= R(a, b) \cup S(a, b), & (R \cap S)(a, b) &= R(a, b) \cap S(a, b), \\ R^\circ(a, b) &= R(b, a), & I(a, b) &= \begin{cases} \Omega & a = b \\ \emptyset & a \neq b, \end{cases} \\ (R; S)(a, c) &= \bigcup_{b \in N} R(a, b) \cap S(b, c). \end{aligned}$$

All values are regions, by Proposition 4.4, so the operations stay inside the algebra.

Theorem 10.2 (Slice compatibility). For every unit x : $(R \cup S)_x = R_x \cup S_x$, $(R \cap S)_x = R_x \cap S_x$, $(R^\circ)_x = (R_x)^\circ$, $I_x = \text{id}_N$, and $(R; S)_x = R_x; S_x$, the ordinary relational composite. Two scoped relations are equal iff all their slices are equal. Consequently:

- (a) composition is associative, $(R; S); T = R; (S; T)$, and I is a two-sided identity;
- (b) composition distributes over union on both sides: $R; (S \cup T) = R; S \cup R; T$ and $(R \cup S); T = R; T \cup S; T$;
- (c) composition is monotone in each argument, and converse reverses it: $(R; S)^\circ = S^\circ; R^\circ$.

Proof. For composition: $x \in (R; S)(a, c)$ iff there is b with $x \in R(a, b)$ and $x \in S(b, c)$, iff $(a, c) \in R_x; S_x$. The other identities are immediate. Two scoped relations agree at (a, b) iff the regions agree, iff they contain the same units, iff (a, b) lies in one slice exactly when it lies in the other, at every x . Parts (a)–(c) hold for ordinary finite relations, hence slice by slice, hence for scoped relations. $\square$

The theorem says a scoped relation algebra is nothing more than a family of ordinary relation algebras, one per unit, held together by regions. Every identity of the ordinary algebra therefore transfers, and no separate proof is needed for each new relation type.

Corollary 10.3 (Closure and acyclicity). Put $R^+ = \bigcup_{n \geq 1} R^n$ where $R^n = R; \dots; R$. Since N is finite, $R^+ = \bigcup_{n=1}^{|N|} R^n$, and $(R^+)_x$ is the transitive closure of R_x at each unit. The relation is acyclic if $R^+(a, a) = \emptyset$ for every a , that is, if no node is R -related to itself through any chain at any unit.

Proof. A chain of length $> |N|$ at a unit repeats a node and can be shortened. The rest is Theorem 10.2. $\square$

At the level of arguments, the sub-argument relation is acyclic by construction (Theorem 9.8). At the level of claims, `DEPENDS_ON` need not be, as Proposition 8.9 showed, and the closure gives the exact region where a claim-level cycle occurs, namely the scope

$$\bigcup_c \text{DEPENDS_ON}^+(c, c).$$

10.3 Composition is licensed, not assumed

The algebra of Theorem 10.2 composes any two relations on the same node set. That is a fact about the mathematics and not a licence to draw conclusions. Which composites carry meaning is decided by type. Here `USES_WARRANT` : `Arg` $\rightarrow$ `War` is the structural relation from an argument to its warrant, holding over the argument's scope.

Design rule 10.4 (Licensed compositions). Composition is defined only between relations whose sorts match, and only the following composites are given a meaning; every other composite is left undefined and produces no relation in the dossier.

composite	meaning
<code>DEPENDS_ON;DEPENDS_ON</code>	<code>DEPENDS_ON</code> , over the intersection of regions
<code>QUALIFIES;QUALIFIES</code>	<code>QUALIFIES</code> (repeated narrowing is a narrowing)
<code>SUPERSEDES;SUPERSEDES</code>	<code>SUPERSEDES</code> (lineage is transitive)
<code>UNDERCUTS_WARRANT;USES_WARRANT</code>	an attack on every argument that uses the warrant (Definition 7.2)
<code>REPLICATES;SUPPORTS</code>	contributes to the replication coordinate R of the target (Chapter 12); it does <i>not</i> yield <code>SUPPORTS</code>
<code>attack;attack</code>	a candidate reinstatement; realized only by labels, and never as a support relation
<code>SUPPORTS;SUPPORTS</code>	undefined

The table is a design decision, and no theorem claims it is the only sensible one. What the mathematics does supply is the reason the last line is undefined.

Proposition 10.5 (Support is not transitive). *The relation `SUPPORTS` of Section 10.1 satisfies neither $\text{SUPPORTS}; \text{SUPPORTS} \subseteq \text{SUPPORTS}$ nor its converse. Where a composite is nonempty, it marks the potential for a composite argument; it does not assert one, and the composite argument's conclusion can only be a claim over the region of the composite.*

Proof. For the first non-inclusion, let a_1 conclude c_1 from evidence e , and let a_2 conclude c_2 from a filler $a'_1 \neq a_1$ for c_1 . Then $(e, c_1), (c_1, c_2) \in \text{SUPPORTS}$ over their regions, but no argument for c_2 has e as a direct filler, so $(e, c_2) \notin \text{SUPPORTS}$. The composite is nonempty wherever both links apply. A composite argument $a_2[a_1]$, with a_1 placed in the slot, is well-formed only for conclusions of scope contained in $\sigma(a_1) \cap \sigma(a_2)$ by (W1), as in Theorem 6.6(a); outside that region no composite argument exists, and where the regions are disjoint the composite of relations is empty. For the converse non-inclusion, a claim supported by a single one-step argument from evidence has a `SUPPORTS` edge and no chain through an intermediate claim. $\square$

The role of `REPLICATES` is the reason the table differs for it. That e' replicates e and that e supports c does not put e' in any argument for c , and substituting e' for e silently

would create an argument no one inscribed with no warrant checked for e' 's scope. What it does establish is that the finding used in that argument has an independent duplicate, and that is information about the claim's standing, recorded in its status vector, not in a support edge.

Chapter 11

Constraint

Most scholarship neither proves a claim nor refutes it. It limits where, how, or under which measurement the claim can remain true. A relation for that case is the reason Adversaria has scoped claims at all. This chapter defines constraint precisely, proves that it behaves as a limit should (never enlarging a claim, independent of the order in which limits arrive, preserving what was already established), and identifies the part of a claim that survives challenge.

11.1 Constraint as region removal

Two notions must be kept apart. A *constraint on a scope* is an operation of set algebra: remove a region. A *constraint of one argument on a claim* is a fact about defeat: a challenger defeats every argument for the claim on some units. The first is exact and simple; the second is what the ledger supplies.

Definition 11.1 (Constrained claim). For a claim $c = (\kappa, \sigma, \dots)$ and a scope τ admissible for κ with $\sigma \setminus \tau \neq \emptyset$, the *constrained claim* is $c \setminus \tau = c|_{\sigma \setminus \tau}$, the restriction of c to $\sigma \setminus \tau$. It is a new claim version and c itself is untouched (Definition 9.9). If $\sigma \subseteq \tau$ nothing remains and the claim is *wholly defeated*.

Definition 11.2 (Constrains). Let $\mathcal{A}(c)$ be the active arguments concluding c ; each has scope $\sigma(c)$, by definition of the scope of an argument. For an argument b and policy p ,

$$\chi_p(b, c) = \{x \in \sigma(c) : \mathcal{A}(c) \neq \emptyset \text{ and } x \in D_p(b, a) \text{ for every } a \in \mathcal{A}(c)\},$$

the units where b defeats every argument for c . The relation $\text{CONSTRAINS}(b, c) = \chi_p(b, c)$ is a scope, by Proposition 4.4. It holds *effectively* on $\chi_p^{\text{in}}(b, c) = \{x \in \chi_p(b, c) : L(b, x) = \text{in}\}$.

The region χ is what the challenger reaches. The effective region is what it *achieves*, since an argument that is itself defeated or undecided at a unit does not constrain there. Section 11.4 relates the two.

11.2 Constraint on scopes is a commutative monoid

Theorem 11.3 (Constraint monotonicity and order independence). *Let σ be a scope admissible for κ , and for a scope τ admissible for κ let $\rho_\tau(\sigma) = \sigma \setminus \tau$.*

- (a) (Admissibility.) $\rho_\tau(\sigma)$ is admissible for κ .
- (b) (Monotone.) $\rho_\tau(\sigma) \subseteq \sigma$. For any sequence of constraints, $\sigma_{n+1} = \rho_{\tau_n}(\sigma_n) \subseteq \sigma_n$.
- (c) (Commutative.) $\rho_{\tau_1}\rho_{\tau_2} = \rho_{\tau_2}\rho_{\tau_1} = \rho_{\tau_1 \cup \tau_2}$.
- (d) (Idempotent, unital.) $\rho_\tau\rho_\tau = \rho_\tau$ and $\rho_\emptyset = \text{id}$.
- (e) (Antitone in the constraint.) $\tau \subseteq \tau' \Rightarrow \rho_{\tau'}(\sigma) \subseteq \rho_\tau(\sigma)$.

Hence the scope left after applying any finite set of constraints $\tau_1, \dots, \tau_n$ in any order, grouped in any way, is $\sigma \setminus (\tau_1 \cup \dots \cup \tau_n)$.

Proof. (a) Admissible scopes for κ form a Boolean subalgebra (Definition of admissible scope), closed under difference. (b) $\sigma \setminus \tau \subseteq \sigma$. (c) $(\sigma \setminus \tau_2) \setminus \tau_1 = \sigma \setminus (\tau_1 \cup \tau_2)$ is symmetric in the two constraints. (d) $(\sigma \setminus \tau) \setminus \tau = \sigma \setminus \tau$ and $\sigma \setminus \emptyset = \sigma$. (e) If $\tau \subseteq \tau'$ then $\sigma \setminus \tau' \subseteq \sigma \setminus \tau$. The final sentence follows from (c) and induction. $\square$

The constraints acting on a scope thus form a commutative idempotent monoid under union, and the surviving scope is a function of the *set* of constraints, not of the order in which challengers arrived. This is the first of the guarantees that make CONSTRAINS more useful than CONTRADICTS: a contradiction between claims has no such structure, and two rebuttals cannot be composed at all.

11.3 Preservation

Theorem 11.4 (Constraint preserves soundness, well-formedness and status). *Let c be a claim and τ a scope with $\sigma(c) \setminus \tau \neq \emptyset$ admissible.*

- (a) (Soundness.) For every model M , $M \models c$ implies $M \models c \setminus \tau$, and hence $M \models (c \setminus \tau_1) \setminus \tau_2$ for any further constraint.
- (b) (Composition.) $(c \setminus \tau_1) \setminus \tau_2$ and $c \setminus (\tau_1 \cup \tau_2)$ have the same core, and are therefore equivalent under $\models$ in both directions.
- (c) (Well-formedness.) For every well-formed argument a for c , $\text{RESTRICT}(a)$ is a well-formed argument for $c \setminus \tau$.
- (d) (Status.) Under restriction-coherent policies, for every $x \in \sigma(c) \setminus \tau$ and every $a \in \mathcal{A}(c)$, $L(\text{RESTRICT}(a), x) = L(a, x)$, and adding the restrictions changes no other label.

Proof. (a) Theorem 4.11(a), twice if needed. (b) Both cores are $(\kappa, \sigma \setminus (\tau_1 \cup \tau_2))$, by Theorem 11.3(c); equivalence is Theorem 4.15. (c) (W1) for RESTRICT asks only that the new scope lie inside the scope of the filler, and (W2)–(W5) are inherited. (d) Theorem 8.6. $\square$

Part (b) is the sense in which constraints compose: two successive constraints and one combined constraint express the same proposition, though they are different records

with different lineages. Part (d) says the constrained claim does not acquire standing from being restated, nor lose any it had.

Theorem 11.5 (Reversibility). *Suppose no argument for c has b among its sub-arguments, and let b be withdrawn. Then for every unit $x \in \sigma(b)$ the labels at x of all arguments not standing on b are those of the framework in which b and everything standing on it at x has been removed. In particular the standing of the arguments for c on $\chi(b, c)$ is restored to what it would have been without b .*

Proof. Theorem 9.12(c). □

Constraint is therefore non-destructive at every level. The claim is never edited; the constrained claim is a separate version; and the effect of the challenger is undone exactly by removing the challenger.

11.4 The surviving claim

A claim can be supported on some units, contested on others, and defeated on the rest. Since all arguments for c have scope $\sigma(c)$, the status of c at a unit is decided by the labels of one fixed set $\mathcal{A}(c)$.

Definition 11.6 (Claim regions). For c with $\mathcal{A}(c) \neq \emptyset$ put

$$\begin{aligned} \text{In}(c) &= \{x \in \sigma(c) : \text{L}(a, x) = \text{in for some } a \in \mathcal{A}(c)\}, \\ \text{Out}(c) &= \{x \in \sigma(c) : \text{L}(a, x) = \text{out for every } a \in \mathcal{A}(c)\}, \\ \text{Und}(c) &= \sigma(c) \setminus (\text{In}(c) \cup \text{Out}(c)). \end{aligned}$$

If $\mathcal{A}(c) = \emptyset$ the claim is *unsupported* on all of $\sigma(c)$. These are scopes by Theorem 7.8.

Proposition 11.7 (Bounds on the surviving region). *Let $\mathcal{A}(c) \neq \emptyset$.*

- (a) $\sigma(c) \setminus \bigcup_{a \in \mathcal{A}(c), b} D_p(b, a) \subseteq \text{In}(c) \subseteq \sigma(c) \setminus \text{Out}(c)$.
- (b) $\chi_p^{\text{in}}(b, c) \subseteq \text{Out}(c)$ for every argument b .

Proof. (a) If no argument defeats any argument for c at x , all are unattacked there, hence in. The upper inclusion holds because $\text{In}(c) \cap \text{Out}(c) = \emptyset$. (b) At $x \in \chi^{\text{in}}(b, c)$ the argument b is in and defeats every argument for c , so all are out. □

Proposition 11.8 (The static case). *Let a be the only argument for c and let $b_1, \dots, b_m$ be undermining or undercutting attackers of a that have no defeaters at any unit where they defeat a . Then $\text{In}(c) = \sigma(c) \setminus \bigcup_i D_p(b_i, a)$ and $\text{Out}(c) = \bigcup_i D_p(b_i, a)$.*

Proof. Outside every $D_p(b_i, a)$ the argument a has no defeater and is in. Inside $D_p(b_i, a)$ the attacker b_i is unattacked, so in, and a is out. □

In this case the constraint of Definition 11.1 with $\tau = \bigcup_i D_p(b_i, a)$ is exactly what the labeling delivers. In general the labeling is the arbiter: attackers may themselves be attacked, rebuttals are mutual, and preferences intervene, and then the survivor is read off the labels rather than computed by subtraction.

Theorem 11.9 (Restriction compatibility and the survivor). *Let policies be restriction-coherent, and let $c|_\rho$ be a restriction of c to an admissible nonempty $\rho \subseteq \sigma(c)$, whose arguments are the restrictions $\text{RESTRICT}_\rho(a)$ of the arguments $a \in \mathcal{A}(c)$.*

- (a) $\text{In}(c|_\rho) = \text{In}(c) \cap \rho$, $\text{Out}(c|_\rho) = \text{Out}(c) \cap \rho$, $\text{Und}(c|_\rho) = \text{Und}(c) \cap \rho$.
- (b) $c|_\rho$ is in at every unit of ρ iff $\rho \subseteq \text{In}(c)$.
- (c) If $\text{In}(c) \neq \emptyset$ is admissible for the kernel of c , then the survivor $\text{Surv}(c) = c|_{\text{In}(c)}$ is the unique largest restriction of c that is in throughout, and $\text{Surv}(\text{Surv}(c)) = \text{Surv}(c)$.

Proof. (a) At $x \in \rho$ the restrictions of the arguments for c are twins of the originals (Theorem 8.6), with equal labels. Hence some argument for $c|_\rho$ is in at x iff some argument for c is, and likewise for the other two regions. (b) By (a), $\text{In}(c|_\rho) = \rho$ iff $\rho \subseteq \text{In}(c)$. (c) By (b) a restriction to ρ is in throughout iff $\rho \subseteq \text{In}(c)$, so $\text{In}(c)$ is the largest such scope and $c|_{\text{In}(c)}$ the largest such restriction. Applying (a) with $\rho = \text{In}(c)$ gives $\text{In}(\text{Surv}(c)) = \text{In}(c)$, which is the whole scope of $\text{Surv}(c)$, so restricting again changes nothing. $\square$

The survivor is the answer to “what remains of this claim after everyone has had their say under this policy?”: the largest part of the original that is undefeated throughout. It is computed, not edited. Its admissibility is not automatic, but it follows from Proposition 7.9 whenever the ledger respects the grain of the kernel; a record that would cut an indivisible cell is not admitted as it stands, and its author is directed to state a finer kernel first.

Example 11.10. In the worked example of Section 7.8, at stage 2 the pooled claim c has $\text{In}(c) = \{AD, AL\}$, $\text{Und}(c) = \{KD\}$, and $\text{Surv}(c)$ is the claim for adults in both contexts. The unit KD is open, and is neither confirmed nor refuted. At stage 3 the reply restores $\text{In}(c) = \sigma(c)$ and $\text{Surv}(c) = c$. An author who accepts the survivor may inscribe it as a voluntary QUALIFIES revision; the system computes it regardless.

Chapter 12

The Status Vector

A dossier must tell a reader how things stand with a claim, and there is a strong pull to answer with one number: a score, a grade, a probability of truth. The design refuses that answer, and this chapter says what replaces it and proves why the refusal is principled rather than squeamish. The status of a claim is a *field* over its scope. At each unit it has several separate components, none derived from the others, and it comes with two distinct layers: an informational layer recording what has been argued, and a justificatory layer recording what survives.

12.1 Two layers: what is on the table, and what stands

Recall that the justificatory layer is the labeling of Chapter 7: an argument is in, out or und at a unit. It says nothing when no argument has been given. The informational layer says what has been offered, whatever its fate.

Definition 12.1 (Evidential state). Let $c = (\kappa, \sigma, \dots)$ be a claim and $x \in \sigma$. Say there is *pro* at x if some active argument for c is present at x , and *con* at x if some active argument for a claim of kernel $\bar{\kappa}$ is present at x . The *evidential state* $B(c, x)$ is

T (pro only), F (con only), B (both), N (neither).

This is Belnap's four values, read as records of what has been told and not as truth values. They carry two orders, as in Belnap's bilattice. In the *truth order*, F lies below N and B, which are incomparable, and both lie below T. In the *information order*, N lies below T and F, which lie below B. A claim can gain support and opposition together as the evidence base grows, moving upward in the information order and sideways in the truth order; neither "both" nor "neither" is an error state. "Both" means the ledger contains argument on each side. It does not mean the store is corrupt, and it does not license any conclusion (Corollary 7.12).

The evidential state depends only on which arguments are active, and the labels also depend on the policy. Their relation is governed by a guarantee that the informational layer may be inconsistent while the justificatory layer never is.

Proposition 12.2 (Consistency of what stands). *At no unit are an argument for a claim of kernel κ and an argument for a claim of kernel $\bar{\kappa}$ both in.*

Proof. Let a and b be such arguments, both present at x . Their conclusions have opposite kernels and x lies in both scopes, so they rebut each other with an active region containing x . By Definition 7.4 the sets $\langle b \prec a \rangle$ and $\langle a \prec b \rangle$ are disjoint, so x lies in at most one of them, and at least one of the two rebuttals succeeds at x . Then one of a, b defeats the other at x . But the set in_x is conflict-free (Theorem 7.7(b)); so they are not both in. $\square$

Because the two layers are separate, situations that a single “disputed” flag would blur are told apart.

Proposition 12.3 (Four situations, four signatures). *Each of the following occurs, and no two share the same pair (evidential state, labels of the arguments for the claim) at a unit.*

situation	evidential state	labels
lack of evidence	N	no argument present
balanced conflict	B	pro and con arguments both und (mutual rebuttal, no tie-breaker)
unresolved methodological attack	T or F	the argument is und because an undermining or undercutting attacker is und
scope mismatch	T at some units, F at others, never B	pro and con arguments exist but their scopes are disjoint

Proof. Witnesses: for lack of evidence, a claim with no arguments; for balanced conflict, the two arguments of Proposition 7.16(a); for an unresolved methodological attack, an argument a and an undercutter u with a defeater u' that u in turn defeats, both und (a symmetric pair of undercutters); for scope mismatch, Proposition 7.17 with $\sigma \cap \tau = \emptyset$. The signatures are visibly distinct. $\square$

12.2 The vector

At a unit $x \in \sigma(c)$ with $\mathcal{A}(c) \neq \emptyset$, let $\mathcal{I}(c, x) = \{a \in \mathcal{A}(c) : L(a, x) = \text{in}\}$ be the arguments that stand there.

Definition 12.4 (Status vector). The *status of c at x* is $S(c)(x) = (E, R, C, G, A, U)$ with the following components; the whole status is the field $x \mapsto S(c)(x)$.

- *Evidence profile* $E(c)(x)$: the set of evidence kinds occurring among the leaves of arguments in $\mathcal{I}(c, x)$.
- *Replication* $R(c)(x) = (r^+, r^-)$: let L be the set of leaves e of arguments in $\mathcal{I}(c, x)$ with $x \in \sigma_e$. Then r^+ is the number of independence families among L together with all e' such that $\text{REPLICATES}(e', e)$ holds at x for some $e \in L$, and r^- is the number of independence families among the e' with $\text{FAILS_TO_REPLICATE}(e', e)$ at x for some $e \in L$.
- *Causal identification* $C(c)(x)$: the best, over arguments $a \in \mathcal{I}(c, x)$, of the weakest rung of any ordinary warrant in the tree of a ; $\perp$ if $\mathcal{I}(c, x) = \emptyset$.
- *Generalizability* $G(c)$: the tuple of projections $\pi_d(\text{In}(c)) \subseteq U_d$ for $d \in D$, the values of each dimension over which the claim currently stands.

- *Agreement* $A(c) = (\text{Acc}, \text{Rej})$: the sets of actors whose recorded stance on c is acceptance or rejection.
- *Unresolved defeaters* $U(c)(x)$: the set of arguments b with $L(b, x) = \text{und}$ that defeat some argument in $\mathcal{A}(c)$ at x .

Each component is a definition, and each embodies a choice the specification flags as such. Two choices deserve remark. In C , a chain is as strong as its weakest ordinary warrant, and alternatives are as strong as the best of them; this is the usual reading of causal inference chains and is a design decision, not a theorem. In R , independence is counted by families, never by records (Proposition 5.5), so that one dataset analysed ten times contributes once. The count r^- is a count of families of declared failed replication attempts; it is not a count of arguments against the claim, and it opposes nothing by itself.

Proposition 12.5 (Agreement has no defeat power). *The component A is displayed but is not an input to the labeling. Recording acceptances or rejections, in any number, changes no label.*

Proof. Stance records are not arguments and are not in the active argument set; the labeling is a function of that set and the policy (Proposition 9.15). $\square$

The consequence is that a vote cannot defeat an argument, and popularity cannot pass for support. Agreement is informative, since it tells a reader who has looked at the claim, but it is separated from evidence by construction.

Proposition 12.6 (Components are scopes and fields). *For each value v of E , C or U , and each (r^+, r^-) , the set of units at which the component takes that value is a scope. $G(c)$ is determined by $\text{In}(c)$.*

Proof. Each component at x depends on x only through the labels of finitely many arguments at x and the membership of x in finitely many scopes, so it depends on the pattern of x , as in Theorem 7.8. Each value class is a union of pattern classes, hence a scope. $\square$

12.3 Dominance

Two claims can be compared at a unit where both are defined. The comparison the model supports is the coordinatewise one: a status is at least as good as another when it is at least as good on every component that bears on standing.

Definition 12.7 (Dominance). Let c, c' be claims and $x \in \sigma(c) \cap \sigma(c')$. Write $c \preceq_x c'$, read “ c' dominates or equals c at x ”, when all of the following hold, for $S(c)(x) = (E, R, C, G, A, U)$ and $S(c')(x) = (E', R', C', G', A', U')$:

- (i) $E \subseteq E'$;
- (ii) $r^+ \leq r'^+$ and $r^- \geq r'^-$;
- (iii) $C \leq C'$ in the rung ladder, with $\perp$ lowest;
- (iv) $G(c)_d \subseteq G(c')_d$ for every dimension $d \in D$;

(v) $|U| \geq |U'|$.

The coordinate A is excluded. Write $c \prec_x c'$ when $c \preceq_x c'$ and not $c' \preceq_x c$, and say c, c' are *incomparable at x* when neither $c \preceq_x c'$ nor $c' \preceq_x c$.

The exclusion of A is the content of Proposition 12.5 carried into comparison. Agreement has no defeat power, so it must not have promotion power either, or a count of acceptances would move a claim upward in an order that is meant to reflect what stands.

Design rule 12.8 (Agreement is not a coordinate of dominance). No comparison of statuses that the system computes or displays as a ranking uses A . Views that show A show it beside the comparison and not inside it.

Proposition 12.9 (Dominance is a preorder). *For each unit x , $\preceq_x$ is reflexive and transitive on the claims defined at x , and $\prec_x$ is irreflexive and transitive.*

Proof. Each of (i)–(v) is a reflexive and transitive condition, being inclusion, $\leq$ or $\geq$ on a coordinate, and a conjunction of such conditions is again reflexive and transitive. For $\prec_x$: irreflexivity is immediate. If $c \prec_x c' \prec_x c''$ then $c \preceq_x c''$ by transitivity; if also $c'' \preceq_x c$ then $c' \preceq_x c'' \preceq_x c$, contradicting $c \prec_x c'$. $\square$

The preorder is not total, and this is where the specification departs from every scalar reputation scheme. The point is stated for a schematic pair, with the realizability stated explicitly.

Example 12.10 (An incomparable pair). Let c_1 and c_2 share a unit x and be supported at x as follows, with no unresolved defeaters at x in either case. The claim c_1 has one interventional trial as its only leaf, so $E = \{\text{trial}\}$, $r^+ = 1$, $r^- = 0$ and $C = \text{interventional}$. The claim c_2 has three observational studies from three independence families as leaves, so $E = \{\text{observational}\}$, $r^+ = 3$, $r^- = 0$ and $C = \text{observational}$. Then $C(c_2) < C(c_1)$ while $r^+(c_1) < r^+(c_2)$, and the two evidence profiles are incomparable under inclusion. Hence c_1 and c_2 are incomparable at x .

The example is realizable by a valid ledger: each claim is the conclusion of an argument built from one warrant and the stated leaves, with the leaves anchored to sources of the stated kinds. Nothing in it depends on the particular numbers, only on the two facts that replication and identification strength are separate coordinates and that neither is derived from the other.

12.4 Scalar summaries are not determined by the data

A scalar summary is a function s from statuses to the real numbers. Two properties are asked of it in practice: it should respect dominance, and it should order all claims. The next theorem says that the first is easy, that the second is available only by making a choice that the dossier does not contain, and that the choice can go either way.

Fix a finite set V of status values with the preorder $\preceq$ and its strict part $\prec$; for a fixed ledger, and a fixed unit, the values realized by the finitely many claims form such a set.

Definition 12.11 (Monotone scalar; representation). A function $s : V \rightarrow \mathbb{R}$ is *monotone* if $u \prec v$ implies $s(u) < s(v)$. It *represents* the preorder if $u \preceq v \Leftrightarrow s(u) \leq s(v)$ for all $u, v \in V$.

Theorem 12.12 (Scalar inadequacy). *Let $(V, \preceq)$ be a finite preorder.*

- (a) *A monotone scalar exists.*
- (b) *If $u, v \in V$ are incomparable, there exist monotone scalars s_1, s_2 with $s_1(u) < s_1(v)$ and $s_2(v) < s_2(u)$.*
- (c) *Some scalar represents $\preceq$ if and only if $\preceq$ is total, that is, if and only if there is no incomparable pair.*

Consequently, when V contains an incomparable pair (as it does for the statuses of Example 12.10), no scalar represents dominance. A monotone scalar must nevertheless assign every incomparable pair a numerical comparison, possibly a tie, that dominance does not determine; moreover, for every such pair, monotone scalars exist giving either strict orientation.

Proof. For a strict partial order $\triangleleft$ on a finite set define the height $h_{\triangleleft}(v)$ as the greatest n for which there is a chain $v_0 \triangleleft v_1 \triangleleft \cdots \triangleleft v_n = v$. This is well defined because the set is finite and $\triangleleft$ is irreflexive and transitive. If $u \triangleleft v$ then every chain ending at u extends to a chain ending at v , so $h_{\triangleleft}(v) \geq h_{\triangleleft}(u) + 1$: the height is strictly monotone for $\triangleleft$.

(a) Take $\triangleleft$ to be $\prec$, a strict partial order by Proposition 12.9, and $s = h_{\prec}$.

(b) Define $\triangleleft_1$ on V by $a \triangleleft_1 b$ if $a \prec b$, or if $a \preceq u$ and $v \preceq b$. Let T be its transitive closure. It is enough to show that T is irreflexive, for then T is a strict partial order containing $\prec$, and $s_1 = h_T$ is monotone for $\prec$ and satisfies $s_1(u) < s_1(v)$, since $u \triangleleft_1 v$ (take $a = u, b = v$).

Suppose T had a cycle, and take a cycle $a_0 \triangleleft_1 a_1 \triangleleft_1 \cdots \triangleleft_1 a_m = a_0$ of $\triangleleft_1$ steps. If no step uses the second clause, the cycle lies in $\prec$, which is irreflexive and transitive, a contradiction. If a step $a_i \triangleleft_1 a_{i+1}$ uses the second clause, then $a_i \preceq u$ and $v \preceq a_{i+1}$. Every other step in the cycle either is a $\prec$ step, which preserves $\preceq v$ when following a_{i+1} (since $v \preceq a \prec a'$ gives $v \preceq a'$), or uses the second clause, which would require the current element to satisfy $\preceq u$. Follow the cycle from a_{i+1} around to a_i . If all steps in between are $\prec$ steps, then $v \preceq a_{i+1} \preceq a_i \preceq u$, so $v \preceq u$, contradicting incomparability. If some step in between uses the second clause, take the first such step after a_{i+1} ; up to that step every step is a $\prec$ step, so its source a_j satisfies $v \preceq a_j$ and, by the second clause, $a_j \preceq u$, hence $v \preceq u$ again, a contradiction. Thus T is irreflexive. The scalar s_2 is obtained by exchanging the roles of u and v .

(c) Suppose s represents $\preceq$. For any u, v either $s(u) \leq s(v)$ or $s(v) \leq s(u)$, so $u \preceq v$ or $v \preceq u$; hence $\preceq$ is total. Conversely, if $\preceq$ is total, take $s = h_{\prec}$. If $u \preceq v$ and $v \preceq u$, any chain ending at one can have its last element replaced by the other, so $h_{\prec}(u) = h_{\prec}(v)$; if $u \prec v$ then $h_{\prec}(u) < h_{\prec}(v)$; so $u \preceq v$ implies $s(u) \leq s(v)$. If $u \not\preceq v$, totality gives $v \prec u$ and $s(v) < s(u)$, so $s(u) \not\leq s(v)$. $\square$

Part (b) is the substantive statement. A scalar that is faithful to every relation the dossier does establish, namely strict dominance, still has to order every incomparable pair, and the dossier contains nothing that says how. A published scalar therefore carries

information that is not in the status: a decision about the exchange rate between replication, identification strength, coverage and the rest. The design refuses to store that decision as if it were a property of the claim.

Corollary 12.13 (Deliberate absence of a global rank). *No function of the status vector that is used as the standing of a claim, in the sense that its order is treated as the order of standing, can be defined without a choice on incomparable pairs that lies outside the ledger. The specification therefore defines no such function, and its store contains none.*

12.5 Weighted sums

The most common scalar is a weighted sum of numerical codings. The verdict of such a sum on an incomparable pair depends on the weights, and the dependence is not a technicality.

Proposition 12.14 (Weight dependence). *Let $\varphi : V \rightarrow \mathbb{R}^k$ be any coding, and for a weight vector $w \in (0, \infty)^k$ put $s_w(v) = w \cdot \varphi(v)$. If $u, v \in V$ are such that $\varphi(u) - \varphi(v)$ has a strictly positive entry and a strictly negative entry, then there are weights w, w' with $s_w(u) > s_w(v)$ and $s_{w'}(u) < s_{w'}(v)$.*

Proof. Let $d = \varphi(u) - \varphi(v)$, let P be the set of indices with $d_i > 0$ and N the set with $d_i < 0$, both nonempty. Put $\delta = \min_{i \in P} d_i > 0$ and $\beta = \sum_{i \in N} |d_i|$, and let $w_i = M$ for $i \in P$ and $w_i = 1$ otherwise. Then $w \cdot d \geq M\delta - \beta > 0$ for $M > \beta/\delta$. Exchanging P and N gives w' with $w' \cdot d < 0$. $\square$

For Example 12.10 with the coding $\varphi = (|E|, r^+, -r^-, \text{runc}(C), -|U|)$, the difference is $(0, -2, 0, +k, 0)$ for a positive integer k equal to the rung gap between an interventional and an observational warrant. So the trial is ranked above the three studies exactly when the weight on identification strength is more than $2/k$ times the weight on replication. Both positive weightings are equally available, and the dossier states neither.

The three-way comparison also has a geometric side, and that is where weighted sums fail in a second way.

Counterexample 12.15 (A frontier point no positive weighting selects). Suppose two coordinates coded as reals, and three claims with codings $u_1 = (1, 0)$, $u_2 = (0, 1)$ and $u_3 = (0.4, 0.4)$. None of the three dominates another: u_3 is below u_1 in the second coordinate and below u_2 in the first. Yet for every $w = (w_1, w_2)$ with $w_1, w_2 > 0$,

$$w \cdot u_3 = 0.4(w_1 + w_2) \leq 0.8 \max(w_1, w_2) < \max(w_1, w_2) = \max(w \cdot u_1, w \cdot u_2),$$

so u_3 is never a maximizer of a positive weighted sum. Ranking by any such sum silently discards a claim that is undominated.

The claim discarded by the sum may be exactly the balanced one: moderately replicated, moderately identified, with moderate coverage, which is a defensible thing for a dossier to record and a poor thing for a ranking to delete.

12.6 The Pareto frontier

What the model can offer in place of a ranking is a set that discards only what is strictly worse.

Definition 12.16 (Frontier). For a finite set X of claims defined at x , the *frontier* $\text{Fr}_x(X)$ is the set of $c \in X$ with no $c' \in X$ such that $c \prec_x c'$.

Proposition 12.17 (The frontier is nonempty and covers). *If X is finite and nonempty, then $\text{Fr}_x(X) \neq \emptyset$, and for every $c \in X$ there is $c' \in \text{Fr}_x(X)$ with $c \preceq_x c'$.*

Proof. Given $c \in X$, if c is not in the frontier there is c_1 with $c \prec_x c_1$; repeat from c_1 . By Proposition 12.9 the relation $\prec_x$ is irreflexive and transitive, so the chain never revisits an element, and since X is finite it stops, at an element c' of the frontier. Transitivity gives $c \preceq_x c'$. $\square$

The frontier is an answer to the question “which of these has nothing strictly better”, and it is not an answer to “which of these is best”. A frontier of size greater than one is a finding: the claims differ in ways that dominance cannot settle, and the view shows the coordinates on which they differ.

12.7 Policy-indexed evaluation

Labels depend on the policy (Theorem 7.18), and so does the status vector, since $\mathcal{I}(c, x)$ is defined through labels. Two very different things can make a reader uncertain about a claim, and the model keeps them apart.

Definition 12.18 (Policy-indexed status). For a finite family $\mathcal{P}$ of policies over one ledger, the *indexed status* of c at x is the set

$$S_{\mathcal{P}}(c)(x) = \{S_p(c)(x) : p \in \mathcal{P}\}.$$

The claim is *policy-robust* at x over $\mathcal{P}$ if $S_{\mathcal{P}}(c)(x)$ is a singleton. Robust dominance is $c \preceq_x^{\mathcal{P}} c'$ if $c \preceq_x^p c'$ for every $p \in \mathcal{P}$.

Uncertainty about the claim is what a single status vector records: the state N , the state B , unresolved defeaters, thin replication. Disagreement about evaluation is the size of $S_{\mathcal{P}}(c)(x)$: the same ledger, evaluated under rules that different communities would each accept, produces different standing. The first is reduced by adding arguments to the ledger. The second is not reduced by any addition to the ledger, since the ledger is the same for all members of $\mathcal{P}$.

Proposition 12.19 (Properties of indexed status). *Let $\mathcal{P}' \subseteq \mathcal{P}$.*

- (a) *If c is policy-robust over $\mathcal{P}$ then it is policy-robust over $\mathcal{P}'$; and $|S_{\mathcal{P}'}(c)(x)| \leq |S_{\mathcal{P}}(c)(x)|$.*
- (b) *If every argument for c present at x has the same label under every $p \in \mathcal{P}$, and the defeats among them at x are the same under every p , then c is policy-robust over $\mathcal{P}$ at x .*
- (c) *Robust dominance is a preorder, contained in each $\preceq_x^p$, and it has at least as many incomparable pairs as any $\preceq_x^p$.*

- (d) There are ledgers and policies p, q for which $S_{\{p,q\}}(c)(x)$ has two elements.
- (e) The component A is the same under every policy.

Proof. (a) The image of a subset is a subset. (b) The components other than A are functions of the labels of arguments and of the defeat relation among them, and, in a fixed ledger, of nothing else. When the labels and the defeats agree, so do the components. (c) An intersection of preorders is a preorder, and it is contained in each of them; a pair comparable under the intersection is comparable under every member, so the intersection has no fewer incomparable pairs. (d) Theorem 7.18 supplies a ledger and two policies under which some argument for c has different labels at x ; then $\mathcal{I}(c, x)$ differs, and with it at least one of E, R, C or U in a suitable instance. (e) Stance records are not evaluated (Proposition 12.5). $\square$

The cost of robust dominance is stated in (c): asking for agreement across evaluation rules removes comparabilities. That is the honest price of not choosing a rule.

12.8 Scoreboards and commensurability

The results above are mathematical. What follows is a design argument, and is not offered as a theorem.

Any public display that orders claims or contributors by a single figure teaches its audience which figure to raise. A contributor who optimizes the figure will optimize it along its cheapest coordinate, and a figure that had a use as a summary will cease to have one once it is a target. The problem is general to measures that become goals. The specific form it takes here is that a scalar standing invites the manufacture of whichever coordinate contributes most cheaply: replicated-looking evidence from one family, agreement from a friendly audience, scope claims narrow enough to be uncontroversial. Some of these are answered by other parts of the design, with families counted rather than records (Proposition 5.5) and agreement having no defeat power (Proposition 12.5). The remaining defence is not to publish a figure that can be a target.

A vector display changes the incentive rather than removing it. Raising one coordinate at the expense of another is visible as a trade, and a reader can see which coordinate was raised. A contributor who wants a better vector must improve some coordinate without lowering others, which is what dominance measures, and the ways of doing so, namely more independent replication, stronger identification, wider scope over which the claim stands and fewer open defeaters, are the ways of doing better work.

A comparison is always made relative to an exchange rate. In a competition the organizers legislate that rate, and the participants then optimize against what they legislated. The specification takes the position that the rate should be declared and not discovered. A *view* is a named, versioned and public function or preorder that extends dominance, and is itself a contestable object: it can be proposed, criticized on stated grounds, and replaced. Every view that is monotone in the sense of Definition 12.11 agrees with dominance where dominance speaks and disagrees with other monotone views only on the pairs that Theorem 12.12(b) shows to be undetermined. The disagreement between views is thereby located precisely: it lies in the incomparable pairs, and nowhere else.

Design rule 12.20 (Views are declared, versioned and separate from status). A ranking presented to a reader is presented as the output of a named view $\text{view}(v, \cdot)$ with its version and its declared weights or order. Views are never stored in the status of a claim and never enter admission or labeling. Every ranked display shows the frontier and names the view that produced the order within it.

There remains a question of what a ranked display should do when no view is acceptable to the person asking. The system may return the frontier and no order. That is a legitimate result, and is the correct one when a query presupposes a commensuration that nobody has declared: a contest with no agreed rule for trading one dimension against another has no winner, and the response that says so is more accurate than one that produces a name.

An example from competitions. A competition marks itself as a game when it tests the admissibility of its own framing assumptions. Take a speed-solving contest for a puzzle cube. Two variants expose what an ordinary scoreboard hides. A *dud* cube is a configuration that violates the cube's orientation or permutation invariants and cannot be solved. The right response is a refusal with an obstruction witness, the violated invariant, and not a long failed attempt; this is the analogue of a disposition with a reason (Chapter 16). A *gradient* cube has stickers whose tones overlap, so that classification cannot be settled sticker by sticker and must be revised until the inferred state satisfies the global invariants; classification comes before transformation, as the choice of kernel comes before argument (Chapter 15), and a locally plausible assignment that is globally impossible is a local-to-global obstruction of the kind of Chapter 8. Scored by time alone, both variants collapse into the ordinary one. Scored by a vector of time, refusal accuracy and classification accuracy, they do not, and any single ranking over that vector introduces a commensuration nobody declared (Theorem 12.12).

Part IV

Protocol

Chapter 13

Routing Is Not Adjudication

Part III defined what stands and why. The remaining chapters specify the protocol that carries records to that computation and results back to readers. The first step is to separate two functions that systems often merge: deciding what a reader should see, and deciding what stands. The first is routing. The second is adjudication. This chapter states the separation generically, so that it applies to any system that has both, and then derives what the formal model of Part III implies for it.

13.1 Four functions

A shared knowledge system performs four functions, and any given system may implement them together or apart.

function	question it answers	may depend on
storage	what has been recorded, and is it intact	content, references
routing	what should reach this reader or workspace, in what order	reader context, ledger, anything about relevance, including labels
admission	does this record take part in evaluation on this surface	record, records it references, policy, the authorization projection
evaluation	what stands, at which units	the active set and the policy, nothing else

A *routing architecture* allocates material to people: by prerequisite structure, by estimated relevance, by notification. Its output is a subset of the material and an order. An *adjudication architecture* determines which contributions stand: its output is a status for each contribution. The two answer different questions, and each can be implemented well or badly without reference to the other. The common error is to let the routing output serve as an adjudication output, so that what is shown prominently is taken to be what stands, or what is hidden is taken to have been rejected.

Definition 13.1 (Routing function). Let R_d be a set of reader contexts. A *routing function* is a map ρ assigning to each reader context $r \in R_d$ and each valid ledger $\mathcal{L}$ and policy p a view $V = \rho(r, \mathcal{L}, p) \subseteq \text{Act}(\mathcal{L}, p)$ together with a linear order on V . A routing function may depend on any information whatever.

The definition is permissive on purpose. Whatever a routing function does, the question is what its output is allowed to mean.

13.2 What routing can and cannot change

Proposition 9.15 says the labeling depends only on the active set and the policy. It follows that routing, which is not an input to the labeling, cannot change any label. The remaining question is what happens if a system computes its labels *from the routed view*, as a system does that evaluates only what it has routed. Write L_V for the grounded labeling of the sub-framework of Act induced on V .

Proposition 13.2 (Local evaluation on a view). *Let $V \subseteq \text{Act}(\mathcal{L}, \mathbf{p})$, let $a \in V$ and $x \in \sigma(a)$, and let $B_x(a)$ be the backward closure of a in the framework on Act (Theorem 7.11).*

- (a) *If $B_x(a) \subseteq V$ then $L_V(a, x) = L(a, x)$.*
- (b) *If $L(a, x) \neq \text{in}$ then there is a view V with $a \in V$ and $L_V(a, x) = \text{in}$. In particular, one can take $V = \{a\}$.*

Proof. (a) The frameworks on V and on Act agree on every defeater of every member of $B_x(a)$, since all these defeaters lie in V . So the backward closure of a in V is $B_x(a)$ and the two defeat graphs agree on it. Theorem 7.11 gives equality of labels. (b) In the framework on $\{a\}$, the argument a has no defeater, so it is in by Theorem 7.7(c). $\square$

Part (b) is the reason routing must not compute status. Every argument that fails to stand appears to stand under some view, namely the view that omits its objections. A routing function is free to choose such a view, and one that ranks by engagement or by predicted agreement will often choose one by accident.

Counterexample 13.3 (A routed view that flips a label). Let a be an argument for c and b an objection that rebuts it under a policy that prefers b , so that $b \triangleright a$ and b has no defeater. Then $L(a, x) = \text{out}$ and $L(b, x) = \text{in}$ at $x \in \sigma(a) \cap \sigma(b)$. A reader whose view is $V = \{a\}$, because the objection was routed elsewhere or ranked below the fold, sees a labeled in under L_V .

The specification therefore fixes two rules. Routing may be arbitrary. Displays of status may not be computed from a view.

Design rule 13.4 (Status is computed on the active set). The status shown for an argument or claim at a unit is $L(\cdot, x)$ computed on $\text{Act}(\mathcal{L}, \mathbf{p})$ for the surface's policy. Routing selects and orders what is shown and does not enter the computation. When some member of $B_x(a)$ is not in the reader's view, the display marks the status as *not closure-complete* and states how many members of the closure are hidden.

Proposition 13.2(a) gives the exact condition under which a view is faithful: it contains the backward closure of what it displays. The mark in Design rule 13.4 costs nothing to compute, since the closure is part of the proof object (Section 14.4).

13.3 Acyclic dependence

Routing may legitimately use labels. A view that shows arguments which stand before those which do not, or shows first the unresolved defeaters of a claim that a reader follows, uses labels to decide order and inclusion, and this is useful. The dependence goes one way.

Proposition 13.5 (One-way dependence of routing on evaluation). *Define the dependence relation on the four functions of Section 13.1: F depends on G if the output of F can change when the output of G changes while its other inputs stay fixed. Then evaluation depends on admission and on nothing else; admission depends on storage; routing may depend on all three; and neither storage, admission nor evaluation depends on routing. In particular the relation is acyclic.*

Proof. Evaluation depends only on Act and p (Proposition 9.15), and Act is the output of admission. Admission depends on the record, on the dispositions of the records it references and on the authorization projection (Definition 16.5); these are stored records or functions of them. No definition of these three functions takes a reader context or a view as input. Routing is defined to take any input. $\square$

A consequence for reputation and capability, which are derived from labels, is stated in Chapter 16: capability enters admission only through records, so that the dependence of admission on labels goes through the ledger and does not create a cycle.

13.4 What each architecture is for

The separation is not a claim that routing is second-rate. The two answer different questions, and neither answers the other's.

routing architecture	adjudication architecture
allocates attention among readers and workspaces	assigns standing to contributions
succeeds when material reaches those to whom it is useful	succeeds when standing follows the arguments
fails by sending the wrong thing, or nothing	fails by counting what should not count, or ignoring what should
output: a subset and an order	output: a labeling and a status field
legitimately uses prerequisites, interest, novelty, load	legitimately uses records, warrants, objections, a declared policy

Two consequences follow, and both are choices of the specification. Relevance does not establish support: a claim routed to a thousand readers is not thereby better supported than one routed to ten. Support does not establish relevance: a claim that stands may be routed to no one. Neither inference is licensed. A system that routes well can be useful and dangerous at once, useful because material reaches readers who need it, and dangerous because the prominence of what it routes can be mistaken for the standing of what it contains. The specification gives routing a place and a rule, and never lets it write to the labeling.

13.5 Routing as a surface

The place routing has is the *surface*: a workspace, a feed, a search result, a notification, a default dossier. Each surface has a policy (Chapter 16) and a view. What a surface contains is a routing matter; what a surface's contents are labeled is an evaluation matter for

that surface's policy. Gating access to shared surfaces is distinct from gating inscription in the ledger, and Chapter 16 shows that inscription is never blocked.

Chapter 14

The Pipeline

This chapter specifies how a submission passes from being stored to affecting what readers see. The path has five stages, each a distinct state transition, and each with a stated dependence. Nothing here adds a rule to the semantics of Part III; the chapter fixes the boundary at which each existing rule applies.

14.1 Records

The ledger contains records of a fixed set of types. Every record has a type, a payload, a list of references to earlier records, an author identifier, a sequence number, a *claimed time* (provenance only, never consulted by admission) and a declared origin mode. Ledger time is not a field; it is derived from anchor records (Section 16.1).

type	payload	defined in
EVIDENCE	an evidence item	Ch. 5
WARRANT	a warrant with backing	Ch. 6
CLAIM	a claim: kernel, scope, modality, provenance, version	Ch. 4
ARGUMENT	a tree over a warrant with fillers and a conclusion	Ch. 6
EQUIVALENCE	a signed identification, including independence-family claims	Ch. 8
DISTINCTION	a pooling declaration	Ch. 8
REVISION	a new claim version with supersession references	Ch. 9
WITHDRAWAL	a reference to an argument authored by the actor	Ch. 9
STANCE	acceptance or rejection of a claim by an actor	Ch. 12
CAPABILITY, REVOCATION, INVALIDATION, RESTRICTION, CLEARANCE	gate records	Ch. 16
BATCH	a declared set of records with a common origin	Ch. 17
ANCHOR	a commitment ordering a set of records into ledger time	Ch. 16

Objections are not a separate type: an objection is an ARGUMENT whose conclusion denies a component of its target (Chapter 7). The absence of a separate type is deliberate,

since it prevents a class of records that defeat without arguing.

14.2 Five stages

Definition 14.1 (Stages). A submission passes through the following, in order, and can stop at any of them.

1. INSCRIBE: the record is appended to the ledger, with a receipt (its content hash and record identifier).
2. VALIDATE: structural checks. References resolve, identifiers agree with contents, the reference order is acyclic (Definition 9.7), the payload conforms to its type's schema, and no executable payload is present in a field that must be inert.
3. ADMIT: the disposition $D_p(r, \mathcal{L})$ of Definition 9.5 is computed for a given surface's policy. The record takes part in evaluation on that surface if and only if it is ADMITTED.
4. PROJECT: a dossier is chosen: a claim family, a scope and a query, over the surface's active set.
5. ACTIVATE: a consumer other than evaluation, namely a ranking, a notification, or a credit computation, is permitted to consume the record.

The stages differ in what they may depend on, and it is the dependence and not the label that carries the guarantee.

stage	depends on	does not depend on
inscribe	the record	policy, gates, labels
validate	the record and the records it references	policy, labels
admit	record, references' dispositions, policy, authorization projection, ledger time	routing, reader, view counts, claimed time
project	the surface's active set, the query, and L	the reader context
activate	any consumer input, including labels	no label depends on it

Proposition 14.2 (Stored, admitted and endorsed are different). *There are valid ledgers with records that are stored and not admitted; admitted and not in; and in and not endorsed by any actor.*

Proof. A trace-null act is stored and QUARANTINED (Definition 9.5). An admitted argument may be out or und, as in Proposition 7.16(a) and Counterexample 13.3. An unattacked argument is in (Theorem 7.7(c)) whether or not any actor has recorded a stance (Proposition 12.5). $\square$

14.3 Every submission receives a disposition

Proposition 14.3 (Totality and stable receipt). *In a valid ledger every record has exactly one disposition for each policy. A record's content hash and record identifier are fixed at inscription and do not depend on its disposition or on any later record, and its ledger time is fixed once the anchor that first includes it is inscribed.*

Proof. D_p is defined by induction along the reference order, which is acyclic in a valid ledger, and assigns one of the seven values to every record. The hashes are functions of content, type, author and sequence. $\square$

The proposition is what underwrites the *submission contract*. A submitter is owed, whatever the load or the policy: a receipt, retention of the record or a declared term of retention, exportability of the ledger, and an explicit disposition with a reason (Chapter 16). A submitter is not owed immediate indexing, review, inference or prominent placement. The ledger has no quota on the number of claims per actor; the scarce things, namely computation, attention and review time, are governed elsewhere in the pipeline and allocated by the rules of Chapters 16 and 17.

Design rule 14.4 (Content addressing). Payloads are stored once, keyed by content hash, and records carry references to payloads. The storage cost of n submissions of identical content is that of one payload plus n fixed-size record headers. Large payloads may be held in cold storage, and indexing of a payload is decoupled from its inscription.

14.4 Proof objects and certificates

Every status shown to a reader has a bounded object that justifies it, and the object is worth as much as what certifies its completeness.

Definition 14.5 (Proof object). For an argument a and a unit x , the *proof object* of the label $L(a, x)$ is the tuple consisting of the argument tree of a , a set B claimed to be the backward closure $B_x(a)$ with, for each pair in it, the defeat region and the policy clause that produced it, the policy version, the ontology versions and mapping claims used by any argument in B , and the labels of the members of B . It is *complete* if $B = B_x(a)$, the backward closure in the framework on the active set of the surface.

Proposition 14.6 (A complete proof object can be rechecked locally). *Given a complete proof object for $L(a, x) = \lambda$, a checker that reads only the proof object and the policy can verify that λ is the grounded label of a at x , by recomputing the least fixed point on the induced framework of B .*

Proof. By Theorem 7.11, $L(a, x)$ is determined by the defeat graph restricted to $B_x(a)$. Since $B = B_x(a)$, the proof object contains that graph and each defeat's region and policy clause, from which membership of x in each region is checked against the policy. The recomputation of Φ_x by iteration terminates in at most $|B|$ rounds. $\square$

The proposition is conditional on completeness, and a checker that reads only the proof object cannot verify completeness. A producer that omitted one standing defeater would supply an induced graph that yields a different label. Completeness therefore needs a separate certificate.

Definition 14.7 (Closure certificate). An anchor record (Chapter 16) may carry a *commitment* root to the surface's active argument set at its ledger prefix and to an index that lists, for each active argument b and each unit pattern, all arguments defeating b there under the policy, the commitment being a Merkle-style hash of the sorted entries. A *closure certificate* for (a, x) consists of the anchor, and for each member $b \in B$ a membership

proof of b and a membership proof of the index entry listing all defeaters of b at x , each verifiable against root.

Proposition 14.8 (Certified labels). *Assume the hash is collision-resistant and that root commits to the true active set and defeat index of its prefix. If a proof object comes with a valid closure certificate against that root, and B is closed under the listed defeaters, then $B = B_x(a)$ and the recomputation of Proposition 14.6 yields the grounded label of a at x in the committed prefix.*

Proof. Each membership proof ties an index entry to the root, and by collision resistance the entry is the committed one. Closure of B under the listed defeaters means no defeater of a member is omitted, so $B_x(a) \subseteq B$; every member of B is reached from a through listed defeaters, so $B \subseteq B_x(a)$. Proposition 14.6 then applies. $\square$

The proposition is relative to the commitment being correct. The index is computable from the active set and the policy, so a party that can recompute it can audit the root; a checker that cannot recompute it relies on the anchoring authority, and the specification says so. The display of a proof object states which certificate, if any, accompanies it, so that “proof object” does not come to mean an excerpt that persuades. A query may be given a budget, and when it is exceeded the display says the object is truncated; the label is unaffected, since it is computed on the full active set (Design rule 13.4).

14.5 Idempotence and order

Proposition 14.9 (Ingestion is idempotent and order-independent). *Inscribing a record already in the ledger, and inscribing records in any order that respects references, yield the same ledger and, for any policy, the same projection.*

Proof. A ledger is a set of records with consistent identifiers, and union is idempotent and commutative. The projection is a function of the set and the policy (Theorem 9.13). $\square$

Chapter 15

Ontologies and Mappings

A kernel carries an ontology version and operational meanings (Chapter 4). The specification does not choose a vocabulary. This chapter says how claims stated in different vocabularies can bear on one another without either being merged into the other, and it proves the soundness and the defeasibility of the one mechanism that connects them.

15.1 Plural ontologies

An *ontology version* is a versioned ledger object that fixes a vocabulary and, for each term, the operational meanings by which it is applied. Kernels are interpreted relative to one ontology version. Two ontology versions may define the same word differently, may define different words for one construct, or may draw the same boundary in different places. The specification does not require, and does not provide, a canonical ontology.

Design rule 15.1 (No silent identification of kernels). No record identifies, subsumes or merges two kernels except a mapping claim of the kind defined below. In particular, neither an editor nor an automated process may treat two kernels as synonymous because their statements look alike, and no evaluation rule treats two kernels as equal unless they are the same object.

The rule is what separates the mechanism here from a synonym table. A synonym table asserts an identity outside the argument graph. A mapping claim asserts it inside.

15.2 Mapping claims

Two kernels κ, κ' over the same unit space give rise to two derived kernels whose extensions in a model M are determined by those of κ and κ' :

$$[\text{EQ}(\kappa, \kappa')]_M = \Omega \setminus ([\kappa]_M \triangle [\kappa']_M), \quad [\text{IMP}(\kappa, \kappa')]_M = \Omega \setminus ([\kappa]_M \setminus [\kappa']_M).$$

A *mapping claim* is a claim whose kernel is $\text{EQ}(\kappa, \kappa')$ or $\text{IMP}(\kappa, \kappa')$, with a scope τ over which it is asserted. It is an ordinary claim: it has arguments, objections and a status. The kinds of mapping in ordinary use are expressed as follows.

- *Equivalent* over τ : $\text{EQ}(\kappa, \kappa')$ over τ .

- *Narrower*: $\text{IMP}(\kappa, \kappa')$, every case of κ is a case of κ' .
- *Broader*: $\text{IMP}(\kappa', \kappa)$.
- *Operationally substitutable* over τ : EQ over τ , with the operational meanings named in the arguments.
- *Disputed*: a mapping claim with an objection, or with arguments both for and against it, whose status is not settled at some unit.
- *Historically related*: not a mapping claim; a relation in the dossier that licenses nothing.

A mapping is a graded, scoped statement. It is asserted over τ and nowhere else, and two vocabularies may be equivalent for adults in a laboratory and not for children in a classroom.

15.3 Transport

Consider an argument that concludes κ over a scope. It bears on a claim about κ' only if some warrant connects them, and here the connecting warrant is an ordinary one.

Definition 15.2 (Transport warrants). For kernels κ, κ' the warrants

$$\text{TR}_{\text{EQ}} = (\kappa, \text{EQ}(\kappa, \kappa') \Rightarrow \kappa'; \Omega; \text{none}; \emptyset), \quad \text{TR}_{\text{IMP}} = (\kappa, \text{IMP}(\kappa, \kappa') \Rightarrow \kappa'; \Omega; \text{none}; \emptyset)$$

are the *transport warrants*. An argument that fills the premise $\text{EQ}(\kappa, \kappa')$ (or $\text{IMP}(\kappa, \kappa')$) with an argument for a mapping claim, and the premise κ with an argument for κ , concludes κ' over the intersection of their scopes.

The transport warrants are not structural exceptions like RESTRICT, POOL and LIFT. They are ordinary warrants and so are subject to (W1)–(W5). Their validity is proved, not stipulated.

Theorem 15.3 (Transport is sound). *Let $\mathcal{M}_{\text{map}}$ be the class of models in which mapping kernels are interpreted as above. Every model in $\mathcal{M}_{\text{map}}$ validates TR_{EQ} and TR_{IMP} . Consequently every well-formed argument built on a transport warrant, with fillers sound in a model $M \in \mathcal{M}_{\text{map}}$, is sound in M (Theorem 6.4).*

Proof. Let $M \in \mathcal{M}_{\text{map}}$ and $x \in [\kappa]_M \cap [\text{EQ}(\kappa, \kappa')]_M$. Since $x \notin [\kappa]_M \triangle [\kappa']_M$ and $x \in [\kappa]_M$, we get $x \in [\kappa']_M$. The argument for IMP is the same, since $x \in [\kappa]_M$ and $x \notin [\kappa]_M \setminus [\kappa']_M$ give $x \in [\kappa']_M$. The second statement is Theorem 6.4 for ordinary warrants. $\square$

Theorem 15.4 (Transport is exactly as defeasible as its mapping). *Let d be an argument built on a transport warrant, with an argument m for the mapping claim as a filler. Then at every unit $x \in \sigma(d)$:*

- if $L(m, x) = \text{out}$ then $L(d, x) = \text{out}$;*
- any defeater of m at x defeats d at x , by hereditary attack (Definition 7.2);*
- a defeated mapping, one whose arguments are all out at x , leaves at x no transported argument that stands.*

Proof. (a) $m \in \text{Sub}(d)$ and $x \in \sigma(d) \subseteq \sigma(m)$ by (W1). Apply Lemma 7.14(ii). (b) This is the definition of hereditary attack through sub-arguments. (c) Every transported argument has a mapping argument among its sub-arguments; by (a) each of them is out at x when its mapping argument is. $\square$

Corollary 15.5 (Chained mappings intersect scopes). *If m_1 maps κ_1 to κ_2 over τ_1 and m_2 maps κ_2 to κ_3 over τ_2 , then an argument transporting κ_1 to κ_3 through both has scope contained in $\tau_1 \cap \tau_2 \cap \sigma$, where σ is the scope of the source argument. Where $\tau_1 \cap \tau_2 = \emptyset$, no such argument exists.*

Proof. Two applications of (W1); this is Theorem 6.6(a) applied to the chain. $\square$

Mappings are therefore not closed under composition in any implicit sense. A chain of mappings yields a mapping only over the intersection of the scopes that were separately argued, and only if each link is present and stands. The dossier for a claim in one vocabulary can show which arguments in others reach it, through which mapping claims, and over what region.

15.4 Consequences

Proposition 15.6 (Evaluation needs no canonical ontology). *The labeling is defined for any set of admitted arguments whatever ontology versions their kernels use. Two members that use different ontology versions for the same subject may share a ledger, and their projections differ exactly where their policies or admitted mapping claims differ.*

Proof. The definition of attack (Definition 7.2) compares kernels by equality and negation only, and the labeling uses only attacks, regions and policy. No definition refers to the content of an ontology. Differences between projections are attributed to records or policy by Theorem 9.14, and mapping claims are records. $\square$

Two things follow for pages. A dispute that arises only because two pages use one word for two operational meanings is not a rebuttal in the model, since the kernels are different objects and neither is the negation of the other; it appears as a gluing question, where a signed identification claim links them and a distinction dissolves it (Chapter 8, Proposition 8.18). And a bot that judges two terms synonymous cannot make them so; it can inscribe a mapping claim, and then the claim is open to objection like any other.

Chapter 16

Gates, Restrictions and Dispositions

Open inscription and governed projection are compatible. Any valid record can be inscribed. What a surface allows a record to do, whether to enter its default dossier, to alter its status displays, to trigger notifications, or to change an established mapping, is decided by its policy, and the policy may gate acts. This chapter specifies gates so that they test what an act requires and never what the actor believes, so that time cannot be chosen by the contributor whose act it governs, so that every restriction is finite and explained, and so that the gate mechanism cannot be used to protect a conclusion from objection.

16.1 Ledger time

One timestamp cannot carry every temporal meaning, and a contributor-controlled one must never decide admission. The specification separates four notions.

- *Ledger time*: the position of a record in the accepted order of a surface. It is derived, not declared (below), and is the only time that admission consults.
- *Effective interval*: a half-open interval of ledger time, $[e_0, e_1)$, carried by governance records (capabilities, restrictions) and fixed by the admitted record that carries it.
- *Claimed time*: external time asserted by a contributor or source about when something happened. It is provenance, is evidence like any other, and is never consulted by admission.
- *Unit-space time*: the time dimension of Ω , which states what a claim is about (Chapter 4). It is a matter of scope and has nothing to do with the order of records.

Definition 16.1 (Anchors and ledger time). A surface s names an anchoring authority in its policy. An *anchor record* of s references its predecessor anchor (the first anchor has none) and a set of records, and carries a commitment to the active set of the prefix it closes (Definition 14.7). Anchors of s form a chain $\text{anchor}_1, \text{anchor}_2, \dots$. The *prefix* $\mathcal{L}_{\leq n}$ is the set of records in the reference closure of anchor_n , and the *ledger time* of a record is

$$t_s(r) = \min\{n : r \in \mathcal{L}_{\leq n}\}, \quad t_s(r) = \infty \text{ if there is no such } n.$$

If two distinct anchors claim the same index, the surface is *forked* beyond that index;

acts whose first anchoring is ambiguous have disposition GATED with the mechanism equivocation, and the fork is itself reported in the divergence report of Chapter 18.

Proposition 16.2 (Properties of ledger time). *On a surface whose anchors form a chain:*

- (a) t_s is a function of the set of records and of s .
- (b) If r references q then $t_s(q) \leq t_s(r)$.
- (c) The prefix $\mathcal{L}_{\leq n}$ is a fixed set, and $\mathcal{L}_{\leq n} \subseteq \mathcal{L}_{\leq m}$ for $n \leq m$. A record with $t_s(r) > n$ does not belong to $\mathcal{L}_{\leq n}$.
- (d) The contributor of a record cannot change t_s of it by any field of the record, including its claimed time.

Proof. (a) The chain and the closures are functions of the set. (b) The closure of an anchor is closed under references, so a record in it brings its references with it. (c) The closure of anchor_n is contained in that of anchor_{n+1} because the latter references the former. (d) Ledger time depends only on which anchor's closure first contains the record, and claimed time is not an input to any closure. $\square$

The reference invariant of (b) is weak in one respect: a record and what it references may share an anchor, and the effective intervals of governance records may overlap. A projection is indexed by the prefix, and the prefix fixes the evaluation time:

$$G_n = \Pi(\mathcal{L}_{\leq n}, p).$$

Proposition 16.3 (Historical projections are immutable). *The projection G_n is a function of the fixed set $\mathcal{L}_{\leq n}$ and of p , so its answer to “what stood at prefix n ” never changes. A later projection G_m , $m > n$, is a different object and differs from G_n only through records of ledger time greater than n .*

Proof. By Proposition 16.2(c) and Theorem 9.13. Records of $\mathcal{L}_{\leq m} \setminus \mathcal{L}_{\leq n}$ have ledger time in $(n, m]$. $\square$

A later reconstruction may therefore include information that was not available at n , and it says so by carrying its own index. It cannot rewrite the earlier answer. Expiration needs no scheduler: an effective interval is half-open, and a record with ledger time outside it is simply outside the authority. Early termination is an explicit record.

Design rule 16.4 (Ledger time is derived). No field supplied by a contributor determines the ledger time of a record. Admission, capability validity, revocation and restriction applicability are evaluated at ledger time and never at claimed time. Duplicate detection on an anchored surface breaks ties by the pair (ledger time, record identifier), so that an earlier record is never demoted by a later one with a smaller identifier.

16.2 Two planes

Governance records and argumentative records are kept in separate planes so that authorization can never become standing.

Definition 16.5 (Authorization and argument planes). The *authorization plane* consists of the anchor, capability, revocation, invalidation, restriction and clearance records. Its *projection* $C(\mathcal{L}, p)$ is the set of these records with disposition ADMITTED, where the disposition of an authorization record depends only on its own fields, on authorization records that it references, on the root authorities the policy names, and on ledger time. The *argument plane* consists of all other records. Its admitted members form the active set $\text{Act}(\mathcal{L}, p)$ of Chapter 9. The disposition of an argument-plane record reads C for its gate outcome and reads the dispositions of the argument-plane records it references, and reads nothing else.

The layering is

$$\text{ledger} \longrightarrow \begin{cases} C & \text{authorization projection} \\ \text{Act} & \text{argument projection (reads } C) \end{cases} \longrightarrow \text{defeat labeling of } \text{Act}.$$

Proposition 16.6 (Planes). (a) $C(\mathcal{L}, p)$ is a function of the authorization records of $\mathcal{L}$ and of p ; it does not depend on any argument-plane record or on any label.

(b) $\text{Act}(\mathcal{L}, p)$ is a function of $\mathcal{L}$ and p and is the single set of arguments that the defeat semantics evaluates.

(c) No authorization record is a node of the defeat graph.

Consequently Proposition 9.6, Theorem 9.13 and Proposition 13.5 hold as stated.

Proof. (a) By Definition 16.5 an authorization record's disposition reads only authorization records, roots and ledger time, and the recursion is on the reference order among them. (b) Argument-plane dispositions are computed by recursion along the reference order, reading C , which is fixed by (a). (c) The defeat graph is defined on Act , which contains argument-plane records only. $\square$

Credit and reputation stay outside both planes, and enter the ledger only when someone inscribes a capability record on their basis. That is what closes what would otherwise be a cycle: capability is derived, in practice, from what has survived, that is from labels; labels are computed from Act ; Act reads C . Labels influence which capability records are inscribed, but a capability record, once inscribed, is data, and the disposition of an authorization record does not read the argument plane.

Proposition 16.7 (Admission does not read labels). The disposition $D_p(r, \mathcal{L})$ of any record is a function of its fields, of $C(\mathcal{L}, p)$, of the dispositions of the argument-plane records it references, and of p . It is not a function of any label.

Proof. Immediate from Definition 16.5 and Proposition 16.6(a). $\square$

16.3 Capabilities

An *act family* is a class of records that call for a common competence: adding archival evidence, supplying a warrant for a causal inference, raising an objection, proposing a mapping claim, restricting a claim, withdrawing an argument. A competence for one family does not transfer to another.

Definition 16.8 (Capability, revocation, invalidation). A *capability record* is $\kappa = (u, \alpha, \tau, [e_0, e_1], g)$: an actor u is authorized to perform acts of family α over scope τ during the effective interval $[e_0, e_1]$ of ledger time, on grounds g . A *revocation record* references a capability record. An *invalidation record* references a capability record and is admitted only if its author holds a capability of the audit family, and cites the finding on which it rests. An act record r of family α and act scope τ_r , by author u , holds κ if r references κ , $\kappa \in C$, $\text{author}(r) = u$, $\tau_r \subseteq \tau$, $e_0 \leq t_s(r) < e_1$, no admitted revocation ρ of κ has $t_s(\rho) \leq t_s(r)$, and no admitted invalidation of κ exists.

The definition reads only records and ledger time. In particular a capability is not a computed reputation. A community may decide by any rule, including one based on the survival of earlier contributions, whether to issue a capability, but the issuing is an act, recorded with grounds. The grounds are checked structurally, as references to the grantee's own acts or to a verification record, and their labels are not read.

Proposition 16.9 (Revocation is prospective). *If a revocation ρ of κ is admitted, every act r that holds κ with $t_s(r) < t_s(\rho)$ keeps its disposition. Acts with $t_s(r) \geq t_s(\rho)$ do not hold κ . Only an invalidation record changes the disposition of acts made earlier, and it does so explicitly, by making them REFUSED with the mechanism invalidated capability.*

Proof. Both statements are read off Definition 16.8: revocation enters the test of holding only through the inequality $t_s(\rho) \leq t_s(r)$, and invalidation enters through its existence. $\square$

Backdating an act into an expired interval, or postdating it past a restriction's end, is not available to a contributor, since the ledger time of an act is not a field of it (Proposition 16.2(d)). What a contributor can do is delay or hasten submission, and the anchoring authority decides when records are anchored; a surface that distrusts its authority audits the anchors (Section 14.4).

16.4 Gates authorize acts, not conclusions

Definition 16.10 (Gate). A *gate* of a policy p for a surface s is a predicate $G_s(r)$ on act records that is true if r holds a capability of the required family and scope, or satisfies an alternative demonstration named by the gate, or is the object of a valid clearance record. A record r that passes its base checks, falls under the gate and fails it has disposition GATED on s .

Dispositions are relative to a surface. A record that is GATED on the principal dossier may be ADMITTED in a personal or group projection whose policy has no such gate. It is stored in either case and has one identifier.

Proposition 16.11 (Capability confers no standing). *Passing a gate makes a record eligible for admission. It does not change the label of any argument, does not change any argument's defeat relations to others, and does not defeat anything. A capable actor's argument is in, out or und by the same rules as any other.*

Proof. This is Proposition 9.15(c) together with Proposition 16.6(c): authorization records are outside the defeat graph, and an argument's admission affects the labeling only through its membership in Act. $\square$

Proposition 16.12 (Inscription is never blocked by a gate). *For every finite set of records that is closed, has consistent identifiers and an acyclic reference order, the set is a valid ledger under every policy, whatever gates the policy contains.*

Proof. Validity (Definition 9.7) is a condition on the set alone and mentions no policy. Gates enter only in D_p . $\square$

The specification states the consequence as a distinction:

freedom to inscribe $\neq$ authority to modify a shared projection.

An ungated actor can still inscribe a claim, cite evidence, prepare an objection, keep a personal or group projection, fork a dossier, and request review. What is gated is the contribution to a shared surface: the principal dossier, default status displays, high-ranking search results, notifications, and established mappings.

Clearance. A gated record is not discarded. It keeps its identifier and can become admissible without resubmission. A *clearance record* references a gated record r and belongs to the authorization plane. It is admitted if its author holds a capability for the family of clearing acts at $t_s(\text{clearance})$. When an admitted clearance references r , the gate outcome of r is treated as passed, and r , if its other checks pass, is ADMITTED and belongs to Act like any other argument. The clearance itself is not an argument and enters no defeat graph.

Proposition 16.13 (Clearance keeps one active set). *With clearance, $\text{Act}(\mathcal{L}, p)$ remains a function of $\mathcal{L}$ and p , and remains the only set evaluated by the labeling. A cleared argument has, at each unit, the label it would have had if it had passed the gate. Revoking the clearing capability later does not change the disposition of records already cleared; an invalidation of that capability does.*

Proof. The gate outcome of an argument-plane record reads C , and the admitted clearances are members of C by Definition 16.5; the recursion of Proposition 16.6 is unchanged. The label of an admitted argument is a function of Act and p , whether it entered directly or by clearance. The last statement is Proposition 16.9 applied to the clearing capability. $\square$

16.5 Testing capability without testing belief

A gate is legitimate only if what it tests is a competence that the act requires. A gate that asks whether the actor accepts the community's conclusion tests belief and is excluded by the following requirement.

Definition 16.14 (Position-neutral gate). For a claim $c = (\kappa, \dots)$ let $\bar{c}$ denote the claim with kernel $\bar{\kappa}$ and the same scope. A gate is *position-neutral* if for every actor u , act family

α and scope τ , the gate is passed by an act of family α on c over τ if and only if it is passed by the corresponding act on $\bar{c}$ over τ , the acts differing only in the exchange of c and $\bar{c}$.

The gate the specification recommends for contested subjects is a reconstruction gate. It uses no notion of a strongest argument, since the model defines no scalar strength (Theorem 12.12); it uses the frontier of the status preorder. For a kernel κ and a unit x , let $X_\kappa(x)$ be the set of claims of kernel κ defined at x , and $\text{Fr}_x(X_\kappa(x))$ its frontier (Definition 12.16). To modify the default projection of a claim c over τ , an actor must inscribe a *reconstruction record* that references

1. for each unit $x \in \tau$ at which claims of kernel κ exist, an admitted argument for some claim on $\text{Fr}_x(X_\kappa(x))$, or for a claim named by the surface's dossier policy in its place;
2. the same for the kernel $\bar{\kappa}$;
3. a claim stating the principal empirical uncertainty,
4. a distinction or scope claim stating the principal scope distinction, and
5. for each of the two positions, an observation that would count against it.

Items (1) and (2) are checked by computation and not by judgment. Items (3) to (5) are graded by graders who are not the author, on whether they name the actual open uncertainty, distinction and observations in the dossier.

Proposition 16.15 (Reconstruction gates are position-neutral). *The reconstruction gate is position-neutral. An actor who passes it may act on c and on $\bar{c}$ at τ , and one who fails may act on neither.*

Proof. The conditions (1)–(5) depend on the pair $\{\kappa, \bar{\kappa}\}$ and are unchanged when κ and $\bar{\kappa}$ are exchanged; the frontier computation is the same construction applied to each kernel. The grading is by the same criteria on both sides. Hence the predicate takes the same value on c and $\bar{c}$. $\square$

The proposition guarantees a structural property of the gate, and no more. It does not guarantee that graders are unbiased, and it does not guarantee that the tests are well chosen; those are matters of the invariants of Section 16.8. What it excludes is a gate whose conditions depend on which side of the dispute the actor takes. An actor may pass the gate while opposing the prevailing view, and may fail it while agreeing with it.

Design rule 16.16 (Tests are derived from the dossier). The comprehension tests and reconstruction tasks are generated from the current dossier and not from a fixed list held by administrators. Grading rules and answers become auditable after the fact, while the selection of items remains unpredictable enough to resist rote preparation. Equivalent competencies have alternative demonstrations, such as a reviewed record of contributions, reproduction of a result, or externally verifiable expertise recorded as a verification record.

16.6 Restrictions and controversiality

A topic is not locked. What a restriction can lock is a surface.

Definition 16.17 (Restriction). A *restriction record* is a tuple

$$q = (\text{rule}, \text{ev}, s, \tau, \mathcal{F}, e_0, e_r, e_1, j)$$

consisting of a named rule from the policy, references ev to records showing that the rule's observable trigger holds, a surface s , a scope τ , a set $\mathcal{F}$ of act families that are gated, an effective interval $[e_0, e_1)$ of ledger time with a review point e_r , where $e_0 < e_r \leq e_1 < \infty$, and a reference j to a *justification argument* concluding that the restriction is warranted. Acts of a family in $\mathcal{F}$ on surface s over a scope within τ with ledger time in $[e_0, e_1)$ fall under the gate, and no others do.

Observable triggers that a policy may name include unusually high attack density, repeated reversion or supersession, a rapid influx of new actors, credible risk of personal harm, legal or medical actionability, conflict among qualified projections, and evidence of coordinated manipulation. The policy names the rules. The restriction record must cite records that show the trigger. A restriction that a moderator dislikes a topic is not of this form.

Proposition 16.18 (Restrictions are finite and expire). *Every restriction has finite duration. A restriction has no effect on any act with ledger time outside $[e_0, e_1)$, and needs no record to end it, unless a renewal record, itself a restriction record with its own justification, covers the act.*

Proof. By Definition 16.17, $e_1 < \infty$ and acts outside $[e_0, e_1)$ do not fall under the gate; the interval is a property of the record and needs no further event. A renewal is a new restriction record. $\square$

Design rule 16.19 (Justifications are contestable and stratified). The justification argument j of a restriction is an ordinary argument and can be objected to by ordinary objections. A restriction is evaluated on a *meta surface* whose policy contains no restrictions; the label of j and of the objections to it are computed there. Consequently no restriction can gate an objection to its own justification, and no act of withdrawal, appeal or request for review is ever gated by the restriction it concerns. Emergency restrictions may take effect immediately with a short interval; a renewal requires that the justification argument not be out on the meta surface at the renewal's own ledger time, computed on the prefix that precedes it.

The stratification is what prevents circularity. Were the label of the justification computed on the restricted surface, a restriction could gate the objections that would defeat it, so that the restriction's own standing depended on its own effect. On the meta surface that dependence is absent, and Proposition 16.7 keeps admission on the restricted surface independent of labels; the renewal consults the meta-surface label only as an input to an authorized act, which is inscribed as a record.

16.7 Dispositions with reasons

Every non-admission is explained. The explanation is computed, so that two replicas agree on it.

Definition 16.20 (Reason). For a record r with $D_p(r, \mathcal{L}) \neq \text{ADMITTED}$, the *reason* $q_p(r, \mathcal{L})$ is the tuple (governing rule, failing field or requirement, references to the records evidencing the failure, deciding mechanism, repair and appeal path), where the mechanism is one of: schema check, disposition rule, gate, restriction, clearance, equivocation, invalidated capability, or an explicit decision record.

Proposition 16.21 (Reasons are determined). $q_p(r, \mathcal{L})$ is a function of the set $\mathcal{L}$ and of p . Two replicas holding the same records and the same policy compute the same reason for every record.

Proof. The disposition is a function of the set and the policy (Theorem 9.13(a)); the reason records which clause of the definition of D_p produced the value, with the references that clause used, and is therefore a function of the same inputs. $\square$

A consequence is that gate statistics, meaning who is refused under which rule and how often, are computed from the ledger by replay, so that audits for disparate exclusion require no privileged data.

Appeal. An appeal is an argument, filed on the meta surface, that a stated rule was misapplied or is itself unwarranted. It uses the ordinary machinery of Chapter 7.

Design rule 16.22 (A successful appeal repairs the rule). An appeal whose argument stands on the meta surface yields a revision of the gate or rule concerned, a new version with a supersession reference (Definition 9.9), and the clearance of similarly situated records follows from the revised rule and not from a one-off exception. The earlier version stays in the lineage (Theorem 9.10).

16.8 Gate capture

The capability system is the most valuable point of attack in the design, and its invariants are listed here with the way each is enforced.

	invariant	enforced by
1	No actor authors, grades and finally approves the same test.	validation: the three role fields of a grant record are distinct actors
2	No viewpoint is required unless the capability is about representing that viewpoint.	Proposition 16.15 for reconstruction gates; design elsewhere
3	Every failure maps to a declared competency.	validation: a GATED reason names a declared competency identifier
4	Equivalent competencies have alternative demonstrations.	design: a gate specification lists at least two routes
5	Gate creation and renewal require public reasons.	validation: a restriction record without a justification reference is malformed
6	Gate statistics are auditable for disparate exclusion.	Proposition 16.21
7	A successful appeal repairs precedent.	Design rule 16.22
8	Administrators are subject to the gate when performing gated acts.	construction: gate predicates refer to actors only through capability records
9	Capability cannot substitute for evidence.	Proposition 16.11
10	Capability cannot become hereditary through endorsement alone.	validation: the grounds of a grant must reference the grantee's own acts or a verification record, and never stances of others

Three of the ten hold as propositions of the model, four are enforced by validation, and three depend on how a community specifies its gates and are design requirements. The table says which is which, so that no design requirement is read as a guarantee.

Design rule 16.23 (Blind grading and reversed items). For high-conflict topics grading is blind to the actor's identity and stated position, and tests are periodically seeded with structurally equivalent items whose polarity is reversed, so that a bias in either direction becomes visible in the audit of Proposition 16.21.

Chapter 17

Imports, Automated Origin and Negative Results

Large systems are built partly by machines: bulk imports, extraction pipelines and generated contributions. They raise three problems for a claim graph. Volume can look like consensus. Machine production can pass for independent human confirmation. And what is measured, imported or generated tends to be positive findings, because those are what sources publish. This chapter states how each is treated, and proves the properties that follow from the model.

17.1 Origin modes and batches

Every act record declares an origin mode: human, automated, assisted or imported. The declaration is part of the trace (Definition 9.4), and a record without one is trace-null and quarantined. The mode is displayed and can be filtered in queries. It does not enter the labeling (Proposition 9.15).

Definition 17.1 (Batch). A *batch record* references a set M of records, states the procedure that produced them, the source or sources it drew on, and the origin mode, and asserts a *common-origin claim*: that the members of M lie in one independence family. A record is a *member* of the batch if the batch references it.

The common-origin claim is an equivalence claim in the sense of Chapter 8. It is an ordinary claim, and it is open to objection, but the burden lies in the direction the design intends: members of a batch are one family unless a distinction claim with grounds says otherwise.

Design rule 17.2 (Batches are one family by default). The default family assignment for members of a batch is a single family. Assigning members to distinct families requires a distinction claim stating why their origins are independent, filed as an ordinary act and open to objection. Records of different batches are in different families only if the procedures or sources are declared to differ.

Proposition 17.3 (An import is one family). *Let a batch have n members, each of which is an evidence item, and suppose the common-origin claim of the batch stands. Then the members lie in*

one independence family, and, for any claim, their joint contribution to the replication coordinate r^+ or r^- of the status vector is at most 1, whatever n is.

Proof. By definition the family identifier is the class under the equivalence generated by equivalence claims that stand, and the common-origin claim identifies all members. The coordinates $r^\pm$ count independence families and not records (Definition 12.4), and Proposition 5.5 shows that further members of an existing family do not raise the count. $\square$

The proposition states, for a batch, what Chapter 2 states for automated origin generally: independence, replication and contributor count are never inferred from record count.

17.2 Reversible activation

An import is activated on a surface by being admitted there. Its withdrawal is a set of withdrawals of its arguments, and this is as clean as a single withdrawal.

Corollary 17.4 (Batch withdrawal is deletion up to labeling). *Let $a_1, \dots, a_k$ be arguments and let $\mathcal{L}'$ extend $\mathcal{L}$ by a withdrawal of each. For $x \in \sigma(a_i)$ let $Z_x^{(i)}$ be the set of Theorem 9.12 for a_i , and let $Z_x = \bigcup_{i: x \in \sigma(a_i)} Z_x^{(i)}$. Then every member of Z_x is out at x in $\mathcal{L}'$, and every other argument has at x the label it has in the framework obtained from $\mathcal{A}_x(\mathcal{L})$ by deleting Z_x and all defeats involving its members. At units in no $\sigma(a_i)$ no label changes.*

Proof. The proof of Theorem 9.12 applies verbatim with Z_x in place of the single set. Each w_{a_i} has no defeaters, so it is in and defeats a_i ; hence every a_i is out at units of $\sigma(a_i)$, and every argument having some a_i as a sub-argument is out there by Lemma 7.14(ii). All members of Z_x are therefore out. Delete them one at a time; by Lemma 7.19 no other label changes and the remaining members stay out. The withdrawal nodes then defeat nothing that remains and are isolated in nodes. $\square$

A defective import can therefore be withdrawn as a unit, and the graph afterwards is what it would have been had the import never entered, at the level of labels. The record of the import and of its withdrawal stays in the ledger, along with everything anyone did in the interim.

17.3 Negative and failed work

Knowledge bases favor positive assertions because sources do. The specification makes negative and failed work first-class by giving each kind an object in the model.

kind of negative work	object in the model
failed replication	FAILS_TO_REPLICATE from a declared attempt whose outcome does not reproduce the specified result (Chapter 10); counted in r^- ; supports no opposing kernel
null detection	an evidence item whose kernel states that a test under its design did not detect the effect; opposes nothing by itself
bounded effect	an evidence item with kernel $\bar{\kappa}$, “the effect is below the declared bound”, from an equivalence or noninferiority design with adequate precision
opposite effect	an evidence item supporting a contrary directional kernel; reaches $\bar{\kappa}$ through an entailment warrant
withdrawn argument	a withdrawal record; the argument stays addressable (Theorem 9.12)
abandoned operationalization	a revision superseding the kernel, with the earlier version preserved (Theorem 9.10)
defeated warrant	an undercutting argument that stands, with its region
unsuccessful search	an evidence item reporting the protocol, corpus and date, with kernel stating that no match was found; an inference to a claim of absence needs a warrant of search adequacy, with backing, open to undercutting
expected evidence not found	the same, with the expectation stated as a claim

Four things are called a negative result and they are not the same thing. Let κ be a kernel of the form “the effect is at least δ ”, with the bound δ part of its operational meaning, and let $\det(\kappa, D)$ be the kernel “a test of κ under design D detects it”.

Definition 17.5 (Negative outcomes). A study of design D over a scope σ_e reports one of the following, each as an evidence item with the kernel shown.

- (i) *Null detection*: the kernel $\overline{\det(\kappa, D)}$. The test did not reject its null under the design.
- (ii) *Bounded effect*: the kernel $\bar{\kappa}$. The reported interval excludes effects of size δ or more, which is a claim the design warrants only if it is an equivalence or noninferiority design with adequate precision at that bound.
- (iii) *Opposite effect*: a kernel $\kappa^\downarrow$ stating an effect of the opposite sign of size at least δ .
- (iv) *Failed replication*: an evidence item that declares an attempt to follow the protocol of an earlier item e_0 and whose outcome kernel is not that of e_0 . It records that the specified result was not reproduced.

Proposition 17.6 (What each negative outcome can support). (a) *A null detection item has a kernel that is neither κ nor $\bar{\kappa}$. It is therefore not the conclusion kernel of any argument for $\bar{\kappa}$, and no attack arises between it and any argument for κ . An argument from it to $\bar{\kappa}$ exists only through an ordinary warrant, for instance that a design of adequate power would have detected an effect of size δ if present, with a backing that states the power, and that warrant is open to undercutting.*

- (b) *A bounded-effect item supports arguments for $\bar{\kappa}$ directly, and an opposite-effect item supports them through an entailment warrant, valid in all models in which $\kappa^\downarrow$ has its stated meaning, from $\kappa^\downarrow$ to $\bar{\kappa}$.*

- (c) A failed replication contributes to the replication coordinate r^- and to nothing else; it is not evidence for any kernel.

Proof. (a) Attack is defined from equality and negation of kernels (Definition 7.2), and $\det(\kappa, D)$ is neither equal to nor the negation of κ . The warrant is an ordinary warrant with premise kernel $\det(\kappa, D)$ and conclusion $\bar{\kappa}$ and is subject to (W1)–(W5). (b) The first is by kernel equality. For the second, if $x \in [\kappa^\downarrow]_M$ then the effect at x is of the opposite sign, hence below δ , so $x \in [\bar{\kappa}]_M$ in every model with that meaning. (c) By the definition of r^- (Definition 12.4) and of FAILS_TO_REPLICATE. $\square$

The absence of a positivity bias in the semantics is a precise property.

Proposition 17.7 (Evaluation is symmetric under exchange of a kernel and its negation). *Let ι be a bijection on kernels that commutes with negation, that is, $\iota(\bar{\kappa}) = \overline{\iota(\kappa)}$. Extend ι to a ledger $\mathcal{L}$ by renaming the kernels of all claims, evidence items and warrants. Then the argument frameworks of $\mathcal{L}$ and $\iota\mathcal{L}$ are isomorphic under the renaming, and $L(\iota a, x) = L(a, x)$ for every argument a and unit x , when the policy is renamed correspondingly.*

Proof. Attack (Definition 7.2) is defined from scopes, from the structure of argument trees, and from whether two kernels are equal or are negations of each other. All three are preserved by ι , which is a bijection commuting with negation and leaving scopes untouched. Denial kernels of targets are renamed consistently. Preferences and admissibility refer to arguments and evidence kinds and are transported by the renaming. The renamed framework is then isomorphic, and the grounded labeling, being defined from the graph alone, is preserved. $\square$

In particular, taking $\iota(\kappa) = \bar{\kappa}$ on all kernels exchanges every finding with its opposite. A bounded-effect finding is exactly as strong, in the semantics, as a positive one of the same scope, kind and independence. A null detection is not the mirror image of a positive detection; it is a different kernel, and Proposition 17.6(a) says what it takes to use it. Any asymmetry a dossier shows comes from the records, that is, from what has been inscribed, and none from the rules.

Example 17.8 (An unsuccessful search). An evidence item reports a search: protocol P , corpus C , date T , and finds no item matching the query. Its kernel states that no match exists in C at T , over the scope of units C covers. To conclude that a phenomenon is absent from the units C represents, an argument needs a warrant of search adequacy, that a search of this kind would have found the phenomenon if present, with a backing that names the search's sensitivity. That warrant can be undercut, and its scope is σ_w . The dossier thereby records both the search and the assumption that turns it into a finding of absence, and a reader can see which is contested.

A failed attempt does not by that fact lower an actor's standing. A well-executed failure is among the most valuable acts in the system, and the credit dimensions below treat it so.

17.4 Credit as a vector

Credit for argumentative acts is not a scalar, for the reason status is not (Theorem 12.12). Each act a receives a *credit vector*

$$\rho(a) = (q, \delta, r, v, s),$$

whose components are evidential quality (how well the act's leaves are anchored and how strong the rung they support), decisiveness (how much of the label of the target the act changes), repair value (how much of a defeated claim the act saves, as a survivor), verification (survival under independent challenge or confirmation), and scope precision (agreement between the scope the act asserts and the scope that ultimately stands). Credit attaches *after* downstream events: an objection may earn submission credit on inscription and verification credit only once it survives challenge.

Definition 17.9 (Independent validating event). A validating event for an act a is a confirmation of it, a failed attempt to defeat it, or a replication of its finding. It is *independent* if the record that constitutes it lies in an independence family different from that of a .

Proposition 17.10 (No self-validation). *The verification component $v(a)$ of the credit of a counts only independent validating events. Consequently adding any number of records in the family of a , whether by the author of a or by any other actor, leaves $v(a)$ unchanged.*

Proof. By Definition 17.9 an event in the family of a is not independent and is not counted. $\square$

The guarantee is only as strong as the family structure that supports it. Family formation is by equivalence claims, which are themselves arguable, so that a ring of coordinated actors that has not been shown to depend on one another counts as independent until someone files and sustains the equivalence claim. The specification does not pretend otherwise, and Chapter 16 lists the audits that make such a claim easier to file.

Credit is used for capability decisions and for the allocation of attention. It is derived from labels, and it enters no label (Proposition 13.5).

Remark 17.11 (Redundant acts). It is natural to propose that credit be proportional to the informational change an act makes to the accepted graph, with diminishing credit for structurally redundant acts. The specification adopts this as a governance heuristic and does not define the measure. No theorem in this monograph depends on it, and none should until an information measure on argument graphs is defined that supports one.

17.5 Attention is the scarce resource

Spam is damaging when it consumes other people's attention, so the scarce thing to charge for is promotion into shared attention surfaces, not speech. The design is unbounded inscription with bounded attention, in place of bounded inscription with unaccountable discretion.

Design rule 17.12 (Attention controls). A surface may combine the following controls, each stated in its policy and each visible in dispositions and reasons: review bonds returned when a contribution survives screening; sponsorship by an actor holding a capability for the relevant family; indexing bandwidth earned progressively; duplicate detection by content hash; bundling of repetitive submissions into a batch; random audits; and reductions of credit for acts that systematically waste review effort. None of them gates inscription (Proposition 16.12), and none changes a label (Proposition 16.11).

Chapter 18

Federation, Queries and the Constitution

The final chapter of the protocol part specifies how communities share records while keeping their own evaluation, what questions the system answers about arguments and scopes, and how the commitments of the whole design fit together. It closes with the limits of what has been specified.

18.1 Shared events, divergent projections

A federation member i holds a valid ledger $\mathcal{L}_i$ and a policy p_i , and computes the projection $G_i = \Pi(\mathcal{L}_i, p_i)$. Members exchange records. Merging is union (Theorem 9.13(b)), so members that hold the same records hold the same ledger, whatever the order of exchange. Convergence is claimed at the event layer and nowhere else.

Proposition 18.1 (Convergence is of records, not of projections). (a) If $\mathcal{L}_i = \mathcal{L}_j$ and $p_i = p_j$ then $G_i = G_j$.
(b) If $\mathcal{L}_i = \mathcal{L}_j$ and $p_i \neq p_j$ it is possible that $G_i \neq G_j$.

Proof. (a) is Theorem 9.13(a). (b) Theorem 7.18 gives a ledger and two policies under which some argument has different labels at some unit. $\square$

The second part is a feature and not a defect. It lets disagreement be inspected: two dossiers that differ, on a shared ledger, differ because their policies differ, and the difference can be located.

Definition 18.2 (Divergence report). For members i, j the *divergence report* is the triple consisting of the symmetric difference $\text{Act}(\mathcal{L}_i, p_i) \triangle \text{Act}(\mathcal{L}_j, p_j)$, the set of pairs on which Adm_{p_i} and Adm_{p_j} differ, and the set of pairs (b, a) of arguments active in both on which the preferences $\langle b \prec_{p_i} a \rangle$ and $\langle b \prec_{p_j} a \rangle$ differ.

Proposition 18.3 (Divergence is explained by the report). If the projections of i and j assign different labels to some argument at some unit, then the divergence report is nonempty. If moreover $p_i = p_j$, the symmetric difference of the active sets contains an argument in the backward closure of the disputed argument, computed in the union of the two frameworks.

Proof. The first statement is the contrapositive of Theorem 9.14(c): if all three components were empty, the active sets would be equal, the admissibility relations equal, and the preferences would agree on all pairs of active arguments, and then the labels would agree. The second statement is Theorem 9.14(b). $\square$

Every difference between two dossiers is thus attributed either to records or to a policy, and, under a shared policy, to records in an identifiable neighbourhood of the disputed argument. What a member cannot do is claim that the graphs differ for reasons not visible in the record.

A member may adopt another's policy by reference, since policies are ledger objects, and may publish its own. Dispositions are member-relative: a record refused by one member may be admitted by another without either violating a rule of the protocol. Nothing about any member's gates binds any other member's surfaces.

18.2 Queries over arguments and scopes

A query is a function of the projection and its parameters. Its result depends on the ledger, the policy, a scope, and the query text.

Definition 18.4 (Query semantics). A query Q is evaluated over $(\mathcal{L}, p, \sigma)$, where $\sigma \in \mathcal{B}(\Omega)$ restricts the units considered, and yields a finite set of records, units or regions. The labeling used is the grounded labeling of Definition 7.6. Other labelings are extensions and are not specified.

Proposition 18.5 (Queries are replica-independent). *The result of a query depends only on the set $\mathcal{L}$, the policy p , the scope and the query. Two members holding the same records and policy return the same result.*

Proof. The query is a function of the projection, and the projection is a function of $\mathcal{L}$ and p (Theorem 9.13). $\square$

The specification defines the following built-in queries, each in terms of objects already in the model.

query	definition
strongest surviving version of c	$\text{Surv}(c) = c _{\text{In}(c)}$, the unique largest restriction of c that is in at every unit (Theorem 11.9)
smallest unresolved defeater	for $x \in \text{Und}(c)$, the members of $U(c)(x)$ that are minimal under inclusion of their active regions
regions without standing	$\sigma(c) \setminus \text{In}(c) = \text{Out}(c) \cup \text{Und}(c)$
claims depending on a withdrawn source	for an evidence item e , the claims with an argument having e among its leaves, with the regions where those arguments are out after the withdrawal (Theorem 9.12)
conclusions that change under another policy	$\{(c, x) : S_p(c)(x) \neq S_{p'}(c)(x)\}$ (Section 12.7)
disputes caused only by operational non-equivalence	families of signed identifications with $\text{Ob}(\mathcal{E}) \neq \emptyset$ for which some confinement to sense slices gives $\text{Ob}(\mathcal{E}') = \emptyset$ (Proposition 8.18)

A query in the surface language combines conditions on evidence kind, scope, origin mode, labeling and policy. The example from the design notes reads: show the claims supported by peer-reviewed evidence, within the population of adults, excluding arguments defeated by a retraction, including unresolved scope objections. Each condition is a predicate over records or regions, and their conjunction is again a function of $(\mathcal{L}, p, \sigma)$.

18.3 The dossier

The presentation layer assembles, for a claim and a surface, what the earlier chapters define: the lineage of claim versions (Chapter 9); the arguments grouped by label at each region; the regions In, Out and Und (Chapter 11); the status vector as a field, without a scalar (Chapter 12); the unresolved defeaters; any obstruction reported for a family of pages (Chapter 8); the indexed status over the policies the reader chooses to compare; and a proof object for any displayed label (Section 14.4). It is a function of $(\mathcal{L}, p, \text{query})$ and stores nothing of its own.

18.4 The ten decisions

The design can be summarized in ten commitments. The table gives, for each, where it is realized, and whether the realization is a proved property of the model, an enforced validation, or a design requirement.

	commitment	realized in	kind
1	The ledger records acts, not truth.	Def. 9.7; Prop. 12.5; Thm. 9.13	proved
2	Claims are scoped, typed and versioned.	Thm. 4.11; Rule 4.18; Thm. 9.10	proved; design

	commitment	realized in	kind
3	Arguments, not triples, carry epistemic force.	Thms. 6.4, 6.5, 6.6	proved
4	Ontologies are plural and mappings are contestable.	Thms. 15.3, 15.4; Rule 15.1	proved; design
5	Contradictions are preserved without explosion.	Cor. 7.12; Prop. 12.2	proved
6	Inference is local, budgeted and accompanied by proof objects.	Thm. 6.6; Prop. 14.6	proved; design (budget)
7	Inscription is open and shared projection is governed.	Props. 16.12, 9.15	proved
8	Capabilities authorize acts, never opinions.	Props. 16.11, 16.15	proved for the stated gate; design otherwise
9	Restrictions are explicit, temporary and appealable dispositions.	Props. 16.18, 16.21; Rules 16.19, 16.22	proved; design
10	No stored claim gains authority through volume, repetition, automation or status.	Props. 5.5, 17.3, 12.5, 17.10, 16.11	proved, given the family structure

18.5 Limits of the specification

The following are open, and are stated so that nothing above is read as settling them.

1. *Family structure is contestable.* Every count of independent sources depends on the equivalence claims that stand. A coordinated ring not yet shown to be dependent counts as independent.
2. *The admissibility table and the rung ladder are design choices.* The proofs use their form, not their content.
3. *No information measure is defined.* Credit heuristics that appeal to informational change remain heuristics (Remark 17.11).
4. *The obstruction theory is exact only for signed binary identifications.* Broader sheaf-theoretic obstructions are an extension for each new kind of constraint (Chapter 8).
5. *Gate properties depend on gate design.* Position-neutrality is proved for the reconstruction gate; the quality of its graders and tests is an institutional matter.
6. *Costs are not analyzed.* The specification states finite computations and proof objects of bounded size, and does not analyze the computational cost of labeling at scale.
7. *Grounded labeling is the only labeling specified.* Other semantics from the argumentation literature are possible extensions.

Part V

Cases

Chapter 19

Three Pages and One Name

The first case is the one that motivated the gluing analysis of Chapter 8: separate pages, in separate traditions or language editions, each about a figure known by one name, each locally coherent, with identification claims that cannot all hold. The case is schematic. It fixes a unit space, three pages and three identification claims, and does not assert anything about what any actual page says. Its purpose is to show, step by step, what the specification computes, what it reports, and which acts repair the situation.

19.1 Setup

Let the unit space be $\Omega = A \times S$ with $A = \{a_1, a_2, a_3, a_4\}$ the aspects of the figure's account that pages discuss, and $S = \{s_1, s_2\}$ the senses in which the name may be used. No record yet distinguishes senses, so every claim below is asserted over all of S . Three pages P_1, P_2, P_3 each make an identification claim about the figure they discuss, using the signed model of Definition 8.10. Let v_k record whether the property in question holds of the figure of page P_k .

- $e_{12} = (\{1, 2\}, +, R_{12})$ with $R_{12} = \{a_1, a_2, a_3\} \times S$: over these aspects the figure of P_1 is that of P_2 .
- $e_{23} = (\{2, 3\}, +, R_{23})$ with $R_{23} = \{a_2, a_3, a_4\} \times S$: over these aspects the figure of P_2 is that of P_3 .
- $e_{31} = (\{3, 1\}, -, R_{31})$ with $R_{31} = \{a_3, a_4\} \times S$: over these aspects the figure of P_3 is distinct from that of P_1 .

Each identification is the conclusion of an argument. The leaf is an evidence item quoting the page's statement, with the extent of the quotation recorded and the kind textual testimony; the warrant is an ordinary warrant that a page's explicit identification of its figure with another is to be read as a claim of sameness or difference over the aspects it treats. The evidence is anchored, the arguments are well-formed, and no argument attacks another. At every unit of its region each is in by Theorem 7.7(c).

19.2 What page-by-page checking sees

The active edges at an aspect are those whose region contains it. Their sign product decides balance (Theorem 8.13).

aspect	active identifications	G_x
a_1	e_{12}	balanced (one edge)
a_2	e_{12}, e_{23}	balanced (a path)
a_3	e_{12}, e_{23}, e_{31}	unbalanced: sign product $(+)(+)(-) = -$
a_4	e_{23}, e_{31}	balanced (a path)

Every pair of the three identifications is satisfiable on its own, at every aspect. No claim rebuts another, since the kernels are not opposite; no argument is attacked; every argument stands. A store that examines claims two at a time, and a labeling that resolves pairwise defeat, both report nothing. This is Counterexample 8.14 with regions.

19.3 What the specification reports

By Theorem 8.16(a), the obstruction region is the union over negative simple cycles of the intersection of their regions. The graph of all three edges has a single simple cycle, the triangle, and it is negative. Hence

$$\text{Ob}(\mathcal{E}) = R_{12} \cap R_{23} \cap R_{31} = \{a_3\} \times S,$$

a scope, in agreement with the table. The dossier for the family shows this region, the cycle that witnesses it, the three identification claims with their status at each unit of the region, and the evidence behind each. The system does not choose which identification to give up and does not remove any. The three pages are unedited; the collection has a property that none of them has.

19.4 Ways the obstruction can change

Each of the following is an ordinary act in the ledger.

An objection stands. A contributor files an undercutter against the warrant used for e_{31} : the passage in P_3 is a translation choice and does not assert distinction. The undercutter u has active region $\{a_3\} \times S$ where the warrant is used and the argument is present, and by the rules of Chapter 7 an undercut always succeeds. If u has no defeater it is in, so the argument for e_{31} is out at a_3 ; at a_3 the graph G_{a_3} over identifications that stand has the two edges e_{12}, e_{23} , which is balanced, and $\text{Ob} = \emptyset$. At a_4 the edge e_{31} still stands and G_{a_4} is still a path. If instead u is itself contested and und, the identification e_{31} is und at a_3 ; it is not counted, and the dossier reports the obstruction at a_3 as *potential*, resting on a claim whose standing is unresolved.

A distinction is made. A contributor files a pooling declaration Δ (Definition 8.1) stating that the name has two senses, with the operational meaning of each, and confines the identifications: e_{12} and e_{23} to the slice of sense s_1 , and e_{31} to the slice of sense s_2 . The regions become

$$R'_{12} = \{a_1, a_2, a_3\} \times \{s_1\}, \quad R'_{23} = \{a_2, a_3, a_4\} \times \{s_1\}, \quad R'_{31} = \{a_3, a_4\} \times \{s_2\}.$$

No unit lies in all three, so the triangle contributes nothing, and $\text{Ob}(\mathcal{E}') = \emptyset$ by Proposition 8.18. No identification was deleted and no page was edited. The distinction is itself a claim with arguments, open to objection; the dossier shows whether it stands, and, where it does not, the obstruction returns at the units where it does not.

Nothing is done. The obstruction stays visible, with its region. Readers see that the three pages cannot be jointly satisfied at a_3 , and the pages themselves say nothing about it.

19.5 What the case shows

The case exercises four results. The first is the exactness of the signed criterion, which is the only reason the report can state the obstruction as a region and not as a suspicion (Theorem 8.16). The second is the separation of attack from identification: nothing in the graph defeats anything, so no labeling would have found the problem. The third is that repair is by records: an objection changes which edges stand, a distinction changes what the edges mean where, and both act on the graph and leave the pages as they were. The fourth is that the built-in query of Chapter 18 for disputes caused only by operational non-equivalence returns this family, since a confinement to sense slices exists that removes the obstruction. What the case does not show is which repair is correct. That is decided, in the dossier, by the arguments for and against the undercutter and the distinction, and by the policy under which they are evaluated.

Chapter 20

A Contested Effect

The second case follows one claim through the machinery of Parts III and IV: pooling, an import, an objection, a reply, two policies, an equivalence result, and a withdrawal. It is schematic. The names are placeholders and the numbers exercise the definitions; no finding is reported. Labels were computed by the grounded labeling and cross-checked by an independent implementation on the frameworks shown.

20.1 Setup

Let $\Omega = \{CS, CC, AS, AC\}$: child or adult participants, in school or clinic settings. Let κ be the kernel “the training raises mean reading scores by at least the declared bound δ ”, with a fixed operational meaning, and write $c = (\kappa, \{CS, CC, AC\})$ for the claim of interest.

item	kind	scope	family	remark
e_{rct}	randomized trial	$\{CS, CC\}$	F_1	kernel κ
$e_1, \dots, e_{50}$	registry export	$\{CS\}$	F_2	kernel κ ; one batch, common origin
e_{coh}	cohort study	$\{AC\}$	F_3	kernel κ
e_{eq}	equivalence trial, margin δ , powered	$\{AS\}$	F_4	kernel $\bar{\kappa}$ (effect below δ)

There are three arguments for κ and one against. The argument a_1 concludes κ over $\{CS, CC\}$ from e_{rct} under an ordinary warrant w_{rct} of rung interventional. The argument a_3 concludes κ over $\{AC\}$ from e_{coh} under an ordinary warrant w_{coh} of rung observational. The argument p applies POOL to a_1 and a_3 and concludes c over $\{CS, CC, AC\}$. The argument n concludes $d = (\bar{\kappa}, \{AS\})$ from e_{eq} under an equivalence-design warrant, of rung interventional, whose backing states the power at the margin δ . The registry items are the leaves of no argument for c . They enter its status only through the relation of replication.

20.2 Status before any objection

At every unit of $\sigma(c)$ the argument p is in, and $\text{In}(c) = \sigma(c)$. The status vector (Definition 12.4) is as follows, at the three units.

unit	E	$R = (r^+, r^-)$	C	U
CS	{randomized trial}	(2, 0)	interventional	$\emptyset$
CC	{randomized trial}	(1, 0)	interventional	$\emptyset$
AC	{cohort study}	(1, 0)	observational	$\emptyset$

At CS the leaf e_{rct} lies in F_1 , and the fifty registry items, each of kernel κ with a scope overlapping $\{CS\}$, replicate it. They lie in one family F_2 , since the batch's common-origin claim stands, so $r^+ = 2$ and not 51 (Proposition 17.3). The registry is displayed as information, and is worth one family.

20.3 An objection

A contributor files an undercutter u against w_{coh} , from a sensitivity analysis showing that unmeasured confounding of the size plausible in the cohort could account for the estimate; u has region $\{AC\}$, the scope where w_{coh} is used. The undercut attacks a_3 at AC and, by hereditary attack, attacks p there. Nothing attacks u , so it is in.

At AC: u is in, a_3 is out, and p is out. At CS and CC nothing changes. Hence

$$\text{In}(c) = \{CS, CC\}, \quad \text{Out}(c) = \{AC\}, \quad \text{Und}(c) = \emptyset,$$

and the strongest surviving version of c is the restriction $\text{Surv}(c) = c|_{\{CS, CC\}}$ (Theorem 11.9). The broad claim did not disappear. What remains is its restriction to the units where it still stands, and the record shows what removed the rest. The status vector at AC now has $E = \emptyset$, $R = (0, 0)$ and $C = \perp$.

20.4 A reply, and two policies

A second contributor files r , an argument from a preregistered sensitivity analysis concluding that w_{coh} is valid at AC. Its conclusion is the opposite kernel to that of u , both have scope $\{AC\}$, and they rebut each other.

policy	u	r	a_3, p at AC	$\text{In}(c)$
p: no preferences	und	und	und	$\{CS, CC\}$
p': $\langle u \prec r \rangle = \{AC\}$	out	in	in	$\sigma(c)$

Under p the mutual rebuttal is symmetric and no argument is preferred, so u and r stay und and, since u defeats a_3 and p while itself unresolved, so do a_3 and p at AC. Then $\text{Und}(c) = \{AC\}$ and $U(c)(AC) = \{u\}$. Under p', which prefers the preregistered analysis at AC, the rebuttal of r by u fails and that of u by r succeeds; r is unattacked and in, u is out, and a_3 and p are in.

Two members holding the same ledger and adopting p and p' respectively compute different dossiers. The indexed status $S_{\{p,p'\}}(c)(AC)$ has two elements: under p it is $(\emptyset, (0,0), \perp, \{u\})$, under p' it is $(\{\text{cohort study}\}, (1,0), \text{observational}, \emptyset)$, and at CS and CC the two agree. The divergence report (Definition 18.2) has an empty symmetric difference of active sets, equal admissibility, and one pair on which the preferences differ, (u, r) . That pair, at that unit, is the whole explanation (Proposition 18.3). The disagreement is not about the evidence, and no further evidence in the ledger would resolve it; it is about whether preregistration is to be preferred to an unregistered sensitivity argument, which is a question a policy answers.

20.5 An equivalence result

The argument n concludes $d = (\bar{\kappa}, \{AS\})$. Its scope is disjoint from $\sigma(c)$, so the two claims, of opposite kernels, are jointly satisfiable, by a model with $[\kappa] = \{CS, CC, AC\}$ (Proposition 4.16). No rebuttal arises. In the dossier for κ over Ω the four units show four situations: standing support at CS and CC, contested support at AC (under p), and standing opposition at AS, with none of them a contradiction. The equivalence finding is as strong in the semantics as a positive one of the same kind, scope and independence (Proposition 17.7); its effect is confined to the units it covers. Had the trial only failed to detect an effect, its kernel would state that a test under its design did not detect κ . That kernel is neither κ nor $\bar{\kappa}$, opposes no argument, and leaves AS with no standing on either side; it would appear in the dossier as evidence, and only a warrant of adequate power, open to undercutting, could turn it into an argument for $\bar{\kappa}$ (Proposition 17.6).

20.6 A withdrawal

Suppose, under p' , the authors of the trial withdraw a_1 . By Theorem 9.12 the argument a_1 is out on $\{CS, CC\}$ and so is every argument having a_1 as a sub-argument at those units, namely p . At AC nothing changes: p stands there through a_3 alone. So

$$\text{In}(c) = \{AC\}, \quad \text{Out}(c) = \{CS, CC\},$$

and $\text{Surv}(c) = c|_{\{AC\}}$. The claim has narrowed to the region where its only remaining support lives, and the region that had the strongest evidence has none, because the withdrawn argument was the only thing standing there. The trial's record, the fifty registry items and the withdrawal remain in the ledger, and the withdrawn argument can still be cited and objected to.

20.7 What the case shows

The case exercises the pieces in the order a reader meets them: scoped evidence and the status vector as a field; an import counted as one family; a hereditary attack through a pooled argument, confined to the unit where the weak component was used; the survivor of a partially defeated claim; symmetric defeat left unresolved; a policy difference located by the divergence report; an equivalence result whose conflict with a broader

claim is localized to the empty intersection; and a withdrawal that is deletion up to labeling. No step used a scalar. At the end, a reader who asks how well supported the training claim is receives a field over four units and a list of records, and is offered, if wanted, a named view (Design rule 12.20) that ranks it against other claims, with the frontier beside the order.

Appendix A

Inventory of Results

The table lists the principal formally stated results of Part III (theorems, with the propositions and corollaries that carry a structural guarantee), with their labels and the properties they guarantee. Definitions, design rules, counterexamples and design arguments are not listed: they record choices, not guarantees.

guarantee	result	content
scope monotonicity	Thm. 4.11	narrowing a scope preserves satisfaction; additivity over disjoint scopes
conflict localization	Prop. 4.16, 4.17	opposite-kernel claims conflict exactly on the intersection of their scopes
no implicit support transitivity	Thm. 6.6, Prop. 10.5	claims alone entail nothing; chaining intersects scopes
soundness and tightness of arguments	Thm. 6.4, 6.5	conclusion scope is exactly what the grounds license
defeat-label consistency	Thm. 7.7	grounded labels are conflict-free, minimal and unique
non-explosion	Cor. 7.12	inconsistent argument does not license arbitrary conclusions
policy-relative defeat	Thm. 7.18	defeat is a function of policy; labels can differ across policies
sense decomposition	Thm. 8.2, 8.3	pooled claims decompose by sense; no generalization across senses
gluing obstruction (signed case)	Thm. 8.13, 8.16	a page family is satisfiable iff no negative cycle; obstruction region is a scope
ledger validity and stability	Thm. 9.8	valid ledgers are closed and prefix-stable
lineage preservation	Thm. 9.10, 9.12	revision history is acyclic and immutable; withdrawal is deletion up to labeling
replay determinism	Thm. 9.13	projection depends on the record set and policy only
federated divergence traceability	Thm. 9.14	every difference is attributed to records or to policy
evaluation separation	Prop. 9.15	labeling ignores non-argumentative metadata

Appendix A. Inventory of Results

guarantee	result	content
slice compatibility	Thm. 10.2	the relation algebra is a family of ordinary algebras
constraint monotonicity, order independence	Thm. 11.3	constraints compose monotonically and commutatively
preservation under constraint composition	Thm. 11.4, 11.5, 11.9	soundness and status survive composition; reversible; unique survivor
scalar inadequacy	Thm. 12.12, Cor. 12.13	no scalar represents a nontotal dominance preorder; monotone scalars can disagree on every incomparable pair
transport of arguments along mappings	Thm. 15.3, 15.4	transport warrants are sound; transport is exactly as defeasible as the mapping
routing and evaluation	Prop. 13.2, 13.5	a routed view can manufacture apparent standing; evaluation does not depend on routing
ledger time and authorization	Prop. 16.2, 16.3, 16.6, 16.9	ledger time is not contributor-controlled; prefix projections are immutable; authorization is outside the defeat graph; revocation is prospective
gates and restrictions	Prop. 16.11, 16.15, 16.18	capability confers no standing; reconstruction gate is position-neutral; restrictions expire
proof objects	Prop. 14.6, 14.8	complete objects are locally checkable; certified against a commitment
negative outcomes	Prop. 17.6, 17.7	null detection opposes nothing by itself; evaluation has no positivity bias
batches and imports	Prop. 17.3, Cor. 17.4	an import is one family; batch withdrawal is deletion up to labeling
federation	Prop. 18.1, 18.3	convergence of records only; divergence is explained by a report